



Aggregating Falcon Signatures with LaBRADOR

Marius Aardal, Diego F. Aranha, Katharina Boudgoust, Sebastian Kolby, Akira Takahashi

► To cite this version:

Marius Aardal, Diego F. Aranha, Katharina Boudgoust, Sebastian Kolby, Akira Takahashi. Aggregating Falcon Signatures with LaBRADOR. CRYPTO 2024 - 44th International Cryptology Conference, Aug 2024, Santa Barbara, United States. pp.71-106, <10.1007/978-3-031-68376-3_3>. <hal-04700114>

HAL Id: hal-04700114

<https://hal.science/hal-04700114v1>

Submitted on 17 Sep 2024

HAL is a multi-disciplinary open access archive for the deposit and dissemination of scientific research documents, whether they are published or not. The documents may come from teaching and research institutions in France or abroad, or from public or private research centers.

L'archive ouverte pluridisciplinaire **HAL**, est destinée au dépôt et à la diffusion de documents scientifiques de niveau recherche, publiés ou non, émanant des établissements d'enseignement et de recherche français ou étrangers, des laboratoires publics ou privés.



HAL Authorization

Aggregating Falcon Signatures with LaBRADOR

Marius A. Aardal¹, Diego F. Aranha¹, Katharina Boudgoust^{2*}, Sebastian Kolby¹, and Akira Takahashi^{3**}

¹ Aarhus University, Denmark

² CNRS, Univ Montpellier, LIRMM, France

³ J.P.Morgan AI Research & AlgoCRYPT CoE, United States

August 9, 2024

Abstract. Several prior works have suggested to use non-interactive arguments of knowledge with short proofs to aggregate signatures of Falcon, which is part of the first post-quantum signatures selected for standardization by NIST. Especially LaBRADOR, based on standard structured lattice assumptions and published at CRYPTO’23, seems promising to realize this task. However, no prior work has tackled this idea in a rigorous way. In this paper, we thoroughly prove how to aggregate Falcon signatures using LaBRADOR. We start by providing the first complete knowledge soundness analysis for the *non-interactive* version of LaBRADOR. Here, the multi-round and recursive nature of LaBRADOR requires a complex and thorough analysis. For this purpose, we introduce the notion of *predicate special soundness (PSS)*. This is a general framework for evaluating the knowledge error of complex Fiat-Shamir arguments of knowledge protocols in a modular fashion, which we believe to be of independent interest. We then explain the exact steps to take in order to adapt the non-interactive LaBRADOR proof system for aggregating Falcon signatures and provide concrete proof size estimates. Additionally, we formalize the folklore approach of obtaining aggregate signatures from the class of hash-then-sign signatures through arguments of knowledge.

Changelog (August 9, 2024)

Technical:

- In our instantiation of LaBRADOR, where we previously recommended the use of a high-splitting ring, we now recommend using a 2-splitting ring bringing us closer to the original parameters in [BS23]. Through the lifting approach from [CHK⁺21], this allows both shorter proofs and more efficient ring computations. We report our improved aggregate signature sizes in Section 7.
- We highlight how our approach can be adapted to the same synchronized model as Squirrel [FSZ22] and Chipmunk [FHSZ23], giving sublinear sizes by eliminating the need for salts.

Editorial:

- Restructured and improved the exposition for predicate special soundness. Section 4 gives preliminaries on multi-round special soundness, and exemplifies its limitations with a worked example. Section 5 introduces predicate special soundness definitions and applies them to the example from Section 4.
- Section 7 now has a greater emphasis on concrete aggregate signature sizes and comparisons to existing schemes.
- Various minor editorial changes.

* Work partially done while affiliated with Aarhus University.

** Work partially done while affiliated with the University of Edinburgh.

Table of Contents

1	Introduction	4
1.1	Our Contributions	5
1.2	Technical Overview	5
2	Preliminaries	8
2.1	Notation	8
2.2	Power-of-Two Cyclotomic Rings	8
2.3	Module-SIS	9
2.4	Aggregate Signatures, Falcon, and SNARKs	9
2.5	LaBRADOR Proof System	9
3	Choice of Ring and Challenge Space	10
3.1	Well-Spread Challenge Spaces	11
3.2	Variant of Schwartz-Zippel Lemma	11
4	Special Soundness and its Limitations	12
4.1	(Multi-Round) Special Soundness	12
4.2	Limitations of Special Soundness: A Toy Example	13
5	Predicate Special Soundness	15
5.1	Challenge predicates	15
5.2	Commitment predicates	16
5.3	Predicate Special Sound Protocols and their Security	19
5.4	Extending to Coordinate-Wise PSS	20
5.5	Knowledge Soundness of LaBRADOR under the Fiat-Shamir Transform	20
6	Adapting LaBRADOR for Aggregation	21
6.1	Changing the Modulus	21
6.2	Norm Checks in the First Iteration	21
6.3	Reformulating the Constraints for a Better Recursion	23
6.4	Working over Subring	23
7	Estimates and Comparison	23
7.1	Estimates and Comparison of Proof Sizes	23
7.2	Estimates of Running Times for Polynomial Arithmetic	24
A	Related Work	29
A.1	Aggregate Signatures from Lattices	29
A.2	Concrete Analysis of Fiat-Shamir	30
B	Additional Preliminaries	30
B.1	Signatures	30
B.2	Aggregate Signatures	31
B.3	Falcon Signature Scheme	32
B.4	SNARKs	33
B.5	Coordinate-Wise Special Soundness	34
B.6	Details of LaBRADOR	35
C	From SNARKs to Aggregate Signatures	40
C.1	Hash-then-Sign Signatures	40
C.2	Snarky Aggregate Hash-then-Sign Signatures	40
D	Omitted Details of Section 3	43
E	Missing Details on Estimates	45
F	Full Description of Padded Falcon Aggregation Constraints	46
F.1	Reformulating Constraints for Better Recursion	46
F.2	Final constraints before moving to subring	47
G	Coordinate-Wise PSS	50
H	Knowledge Soundness Proof for PSS	52
H.1	Abstract Sampling Game	53
H.2	Knowledge Extractor	56
H.3	Coordinate-Wise Extension	59
I	Proof of Non-Interactive Knowledge Soundness of LaBRADOR	63
I.1	Analysing Levels	63

I.2	PSS of LaBRADOR.....	67
I.3	Binding Relation	67
I.4	Knowledge Soundness	68
I.5	Recursive Composition	69
I.6	Last Iteration Optimizations	69

1 Introduction

In 2022, the US National Institute of Standards and Technology (NIST) announced the first protocols, deemed secure even in the presence of quantum computers, for standardization.⁴ Falcon [PFH⁺22], whose security relies on structured lattice problems, is one of the three signature protocols selected by NIST. A natural question now is whether Falcon can be used in more advanced cryptographic settings. In this work, we study the question of *aggregating* many Falcon signatures into a single one.

Aggregate signatures (AS), introduced by [BGLS03], allow to combine N individual signatures, on possibly distinct messages and public keys, into one aggregated signature σ_{agg} . This feature is beneficial whenever a large amount of signatures have to be sent and bandwidth is a bottleneck. It gained a lot of attention in the past few years as aggregate signatures are used on a large scale in blockchains. As an example, Ethereum 2 is currently using aggregate signatures based on pairings [BDN18], as detailed in the annotated specifications.⁵ Preferably, the aggregation does not require the interaction of the signing parties. This is desirable especially if many parties are involved. As many of the currently deployed cryptographic protocols, and in particular pairing-based protocols, become insecure in the presence of large scale quantum computers, it is an important research question to search for presumably quantum-resistant solutions. Lattice-based cryptography has been shown to be one of the most promising directions.

Recently, there have been multiple proposals of aggregating lattice-based signatures, cf. Appendix A for a detailed related work discussion. However, the only known AS tailored to Falcon are *sequential* and thus require some form of interaction between signers [EB14, WW19]. Although [WW19] explicitly instantiates their scheme with Falcon, it turned out to be susceptible to a simple forgery attack [BT23]. Moreover, the size of an aggregate signature is still linear in the number N of signatures involved, and [BT23] reports the compression rate (i.e. the size of AS divided by the size of N signatures) is only about 60% even if the GPV-based scheme of [EB14] is adapted to Falcon.

When it is challenging to design an AS tailored to a specific signature scheme, a natural question is whether *generic* solutions exist. One generic approach for aggregation proposed in the literature is to use non-interactive arguments of knowledge (AoK) [ACL⁺22, DGKV22, WW22].⁶ Given N signatures issued for possibly distinct public keys and messages, we set as witness w the signatures and as statement x the corresponding public keys and messages, i.e., $w := \{\sigma_i\}_{i \in [N]}$ and $x := \{\text{pk}_i, m_i\}_{i \in [N]}$. The signature scheme defines the relation $R = \{(x, w) : \text{Ver}(\text{pk}_i, m_i, \sigma_i) = 1 \ \forall i \in [N]\}$, where $\text{Ver}(\cdot, \cdot, \cdot)$ is the verification algorithm of the signature. Any non-interactive argument of knowledge Π for all NP languages can be used to produce for a statement x a proof π of a corresponding witness w fulfilling the relation R . In particular, if Π is *succinct* (i.e. its proof size is at most polylogarithmic in the size of the statement and witness, henceforth SNARK) [ACL⁺22] or a rate-1 *batch* argument (i.e. its proof size is independent of N the number of NP statements, henceforth BARG) [DGKV22, WW22], one can construct a compact AS scheme by setting the proof as an aggregate signature.

Concretely, the idea of aggregating Falcon using SNARKs was sketched in a recent lattice-based SNARK [ACL⁺22, Sec. 7.2]. In particular, they observed that the verification equation of Falcon can be expressed in the native language of their SNARK (i.e. without converting the equation into general circuit constraints). However, they left it open to rigorously realize the idea. Moreover, their proof system relies on a non-standard lattice-based knowledge assumption, which was recently broken by [WW23].

With the recent introduction of the lattice-based SNARK LaBRADOR [BS23], a significantly more efficient proof system whose security relies on standard structured lattice problems was proposed. Further, the native language of LaBRADOR seems even more suitable for aggregating Falcon signatures. Again, the idea of using LaBRADOR to aggregate signatures was sketched in [TS23b] without providing security proofs and concrete estimates for the computation times and proof sizes for any particular signature scheme. Moreover, their approach requires translating the verification conditions of the signature scheme into R1CS, which does not natively support the relation for batch-proving Falcon signatures.⁷ Given all this, we are motivated to ask the following question:

Can LaBRADOR be used to aggregate Falcon signatures while providing (1) a rigorous security proof, and (2) concrete estimates?

⁴ <https://csrc.nist.gov/Projects/post-quantum-cryptography/selected-algorithms-2022>

⁵ <https://github.com/ethereum/annotated-spec/blob/master/phase0/beacon-chain.md>

⁶ To be more precise, only a relaxed version of knowledge soundness called *somewhere extractability* is sufficient for constructing AS.

⁷ In an updated version of this preprint [TS23a], LaBRADOR is used to aggregate a specific signature scheme, derived from [CLMQ21] and different to Falcon, without going through R1CS.

1.1 Our Contributions

In this work, we give a positive answer to this question. Along the way, we develop a number of technical tools that we believe to be of independent interest. First, we observe that for signature aggregation a *non-interactive* version of LaBRADOR is required. However, the soundness analysis of a non-interactive version of LaBRADOR is not covered by existing frameworks, such as [AFK22], due to its extensive use of probabilistic tests that cannot be captured by a simple rewinding process (cf. Section 4). In Section 5, we propose a new soundness notion for multi-round public-coin protocols, called *predicate special soundness (PSS)*. Our PSS notion leads to a more general framework for analyzing knowledge soundness of the Fiat-Shamir transform [FS87], implying the first complete knowledge soundness analysis for the *non-interactive* version of LaBRADOR in the random oracle model (Theorem 5.2). We highlight Section 5 as a versatile toolkit allowing future protocols designers to derive concrete knowledge error of (possibly more complex) Fiat-Shamir AoK protocols in a modular fashion. As an example of the flexibility of our framework, we show that it applies to different algebraic settings such as low and high splitting rings (cf. Section 3). To complete the construction of aggregate signatures, we explain in Section 6 the concrete steps required in order to adapt the LaBRADOR proof system for aggregating Falcon signatures (leaving a complete specification to Appendix F). As the modulus q used in the Falcon signature scheme is optimized to be as small as possible, it does not give enough room to use LaBRADOR. Thus, we have to use another larger modulus q' in the proof system, requiring additional checks to guarantee that no wrap-around was caused and that the norms are in the right bounds. Moreover, Falcon operates on rings with large degrees. By using techniques from [LNPS21] to move to subrings of smaller degrees when instantiating LaBRADOR, we significantly reduce the proof sizes, and hence the AS sizes. Lastly, in Section 7 and E we provide concrete estimates for proof sizes and detailed comparison with other lattice-based AS. As a side contribution, we formalize the folklore approach of obtaining aggregate signatures from the class of hash-then-sign signatures through SNARKs (cf. Appendix C).

1.2 Technical Overview

Falcon is an instantiation of the GPV framework [GPV08] for lattice-based hash-then-sign signatures over the NTRU [HPS98] class of structured lattices. It works over a power-of-two cyclotomic ring $\mathcal{R}$ modulo q , denoted by $\mathcal{R}_q$. Let us quickly recap the hash-then-sign paradigm. A key pair consists of pk defining a *preimage sampleable function (PSF)* [GPV08] $F_{\text{pk}} : \text{Do} \rightarrow \text{Ra}$, and sk that allows one to invert F_{pk} . Upon receiving a message m to be signed, the signer first generates its hash $y = \text{H}(r, m) \in \text{Ra}$, where $r \in \{0, 1\}^k$ is a freshly sampled random salt, and then use sk to sample a signature $\sigma = x \in \text{Do}$ following some distribution $\mathcal{D}(F_{\text{pk}}^{-1}(y))$. If instantiated with Falcon, $\text{pk} = \mathbf{h} \in \mathcal{R}_q$ defines an NTRU-module $\Lambda = \{(\mathbf{u}, \mathbf{v}) \in \mathcal{R}^2 : \mathbf{u} + \mathbf{h}\mathbf{v} = \mathbf{0} \bmod q\}$ and sk contains a secret (short) basis of Λ that allows sampling module elements following a discrete Gaussian distribution defined over an arbitrary coset $\Lambda_{\mathbf{t}} = \{(\mathbf{u}, \mathbf{v}) \in \mathcal{R}^2 : \mathbf{u} + \mathbf{h}\mathbf{v} = \mathbf{t} \bmod q\}$. Thus, the signing algorithm of Falcon first hashes m to $y = \mathbf{t} \in \mathcal{R}_q$, uses the secret basis to obtain a preimage $x = (\mathbf{s}_1, \mathbf{s}_2) \in \Lambda_{\mathbf{t}}$, and outputs $\sigma = (\mathbf{s}_1, \mathbf{s}_2, r)$ as a signature. The verification conditions are simply (1) $\mathbf{s}_1 + \mathbf{h}\mathbf{s}_2 = \text{H}(r, m) \bmod q$, and (2) $\|(\mathbf{s}_1, \mathbf{s}_2)\|_2 \leq \beta \ll q$, where β is determined by a Gaussian parameter.

Aggregation of Falcon signatures amounts to batch-proving knowledge of N Gaussian samples $(\mathbf{s}_{i,1}, \mathbf{s}_{i,2})_{i \in [N]}$ and salts $(r_i)_{i \in [N]}$ meeting the above verification conditions w.r.t. a list $(m_i, \mathbf{h}_i)_{i \in [N]}$ of (potentially distinct) messages and public keys, respectively. However, generating proof of correct hash computation is not only costly, but also leads to heuristic security guarantees: an aggregator may need a concrete description of H as a hash function, while Falcon has only been proven secure if H is modeled as a random oracle.⁸ We therefore opt to let the aggregator include salts r_i in the aggregated signature and generate a proof for $\mathbf{t}_i = \mathbf{s}_{i,1} + \mathbf{h}_i \mathbf{s}_{i,2} \bmod q$ for a public statement $\mathbf{t}_i$, and let the verifier compute $\mathbf{t}_i = \text{H}(r_i, m_i)$ locally. Although this approach sacrifices the asymptotic compactness of the resulting aggregate signature, the size of salt r_i is much smaller than $(\mathbf{s}_{i,1}, \mathbf{s}_{i,2})$ in practical parameter regimes. We empirically show that aggregating $(\mathbf{s}_{i,1}, \mathbf{s}_{i,2})_{i \in [N]}$ already reduces the size of signature significantly compared to the naive concatenation of N Falcon signatures. To realize an asymptotically compact scheme, our approach can easily be adapted to (not-to-be-standardized) variants of Falcon: one could remove the salt and make the scheme deterministic, or one could move to a synchronized model, where the salt is replaced by a common time period. In the latter model, only signatures issued for the same time period are going to be aggregated.

⁸ Analogously, the security proofs for IVC [Val08] and PCD [BCMS20] often face the same problem.

Adapting LaBRADOR for Aggregating Falcon Signatures. We instantiate the SNARK with LaBRADOR, a highly efficient sublinear argument based on Module-SIS. We first recall the *principal relation* for which LaBRADOR has been designed. The relation R consists of witness vectors $\vec{w}_1, \dots, \vec{w}_r \in \mathcal{R}_q^n$ and a statement $x = (\mathcal{F}, \beta)$, where $\mathcal{F}$ is a collection of dot product constraints and β is a norm bound check, over $\mathcal{R}_q$ and $\mathcal{R}$ respectively. We call n the *rank* and r the *multiplicity* of the witnesses. Each dot product constraint $f \in \mathcal{F}$ is defined by a function of the form

$$f(\vec{w}_1, \dots, \vec{w}_r) := \sum_{i,j=1}^r \mathbf{a}_{i,j} \langle \vec{w}_i, \vec{w}_j \rangle + \sum_{i=1}^r \langle \vec{\varphi}_i, \vec{w}_i \rangle - \mathbf{b},$$

where $\mathbf{a}_{i,j}, \mathbf{b} \in \mathcal{R}_q$ and $\vec{\varphi}_i \in \mathcal{R}_q^n$, such that $\mathbf{a}_{i,j} = \mathbf{a}_{j,i}$ for all i, j . We say that the principal relation is satisfied if $\forall f \in \mathcal{F}, f(\vec{w}_1, \dots, \vec{w}_r) = \mathbf{0}$ and $\sum_{i=1}^r \|\vec{w}_i\|_2^2 \leq \beta^2$.

At first glance, the verification equation and norm bound of a single Falcon signature seem quite compatible with the principal relation of LaBRADOR. To aggregate N signatures, one might then try to extend the statement to contain a verification equation for each signature and a combined norm bound: $\forall i = 1, \dots, N, \mathbf{s}_{i,1} + \mathbf{h}_i \mathbf{s}_{i,2} - \mathbf{t}_i = \mathbf{0} \pmod{q}$ and $\sum_{i=1}^N \sum_{j=1}^2 \|\mathbf{s}_{i,j}\|_2^2 \leq N\beta^2$. If LaBRADOR was instantiated over the ring used by Falcon then $(\mathbf{s}_{i,1}, \mathbf{s}_{i,2})_{i \in [N]}$ could be used directly as witness vector of multiplicity $r = 2N$ of rank $n = 1$ in $\mathcal{R}_q^n$. However, there are several problems with this approach.

The first problem is that the norm check in LaBRADOR is both approximate and with respect to the entire witness. This introduces a degree of slack that grows with the number of signatures N . When reducing the unforgeability of our aggregate signature scheme to the unforgeability of Falcon, we need that the knowledge extractor outputs a witness consisting of valid Falcon signatures (see details of generic AS construction from SNARK in Appendix C). In particular, we need to be able to guarantee that they have ℓ_2 -norm at most β . In Subsection 6.2, we therefore modify the first iteration of the LaBRADOR protocol to use the approach of [GHL22] for an exact proof of smallness. The norm checks in the subsequent iterations are only for the binding of the commitments, so no modifications have to be made there.

The second problem we encounter appears when we look closer at how exactly LaBRADOR performs its consolidated norm check. Recall, to verify the norm of the witness LaBRADOR uses an (approximately) distance preserving projection to compute a much smaller vector, to send to the prover. Specifically, the modular Johnson-Lindenstrauss projections (see Lemma 2.2) are used, which require that the norm bound b of the statement satisfies the inequality $\sqrt{\lambda}b \leq q/C_1$ for security level λ and some corresponding constant C_1 . For both Falcon parameter sets this is not satisfied, even when restricting b to the norm of just a single signature. Therefore, if we wish to use the Johnson-Lindenstrauss projection, we need a larger modulus. In Section 6.1, we reformulate the statement and witness so that the LaBRADOR protocol uses a separate modulus q' , different from q .

The last problem with this formulation is that the number of initial witness elements $r = 2N$ (actually, r is even bigger due to the aforementioned modifications) and their rank $n = 1$ is quite unbalanced. For the performance of LaBRADOR, the relation between the multiplicity r and the rank n is important (cf. Appendix B.6). In Section 6.3, we present an alternative formulation of the constraints that achieves a better balance between these parameters. This new formulation gives us better runtimes for the prover and verifier and slightly shorter aggregation proofs. For a full list of the final set of constraints, see Appendix F.2.

In a first instantiation of LaBRADOR with the constraint system described above, we maintained the same ring $\mathcal{R}$ as for Falcon. However, we obtained surprisingly large proof sizes and found out that the large ring degrees of Falcon $d \in \{512, 1024\}$ were the main reason for this. As detailed out in Section 6.4, we were able to significantly compress the proof sizes by using existing techniques from [LNPS21] to move to subrings $\mathcal{S}$ of smaller degrees d' .

Choice of Ring and Challenge Space. As explained before, Falcon operates over a power-of-two cyclotomic ring $\mathcal{R}$ of degree d modulo q , while our modified LaBRADOR operates over $\mathcal{S}$ of degree d' modulo q' . For a given degree d' , we can vary the modulus q' in order to obtain different mathematical properties of $\mathcal{S}_{q'}$. In particular, the relation between d' and q' defines how well $\mathcal{S}$ is *splitting* into *CRT-slots*. Without going into the mathematical details here, we mainly distinguish two settings: low-splitting (with few CRT-slots) and high-splitting (with many CRT-slots) regimes. When invoking lattice-based SNARKs, one makes use of a subset $\mathcal{C} \subset \mathcal{S}_{q'}$, which is called the challenge space.

LaBRADOR [BS23] opted for a low-splitting regime which allows to design a simple yet useful challenge space $\mathcal{C}$ for their protocol. In particular, they exploit the facts that in the low-splitting regime 1) every non-zero element of small enough norm is invertible [LS18] and 2) the size of each of the few CRT-slots is exponentially big in the ring degree d' .

Although in Section 7 we conclude that the two-splitting case gives rise to optimal performances and proof sizes for our particular instantiation of aggregate signatures, we do not restrict our analysis to one

choice of ring and challenge space. In future use cases, it might be useful to go to a high-splitting ring. We thus propose a generalization of LaBRADOR allowing for both low and high-splitting regimes. This comes with some additional technical challenges as we detail out in the following. First and foremost, not every small enough ring element is still invertible. By designing suitable challenge spaces, however, one can show that the probability that a challenge element (and the difference of two distinct challenges) is non-invertible is as small as $2^{-\lambda}$ for a targeted security level λ [ALS20, ESZ22, ESLR23]. The previous approaches have been quite ad-hoc and we think it is of independent interest to abstract away the needed property of $\mathcal{C}$ to prove knowledge soundness of most lattice-based AoKs. To this end, we introduce the concept of *well-spread* challenge spaces in Section 3.1. At a high level, well-spreadness of $\mathcal{C}$ directly links the splitting behavior of $\mathcal{S}_{q'}$ to the probability of invertibility of elements in $\mathcal{C}$.

LaBRADOR (implicitly) also used the fact that every small enough ring element is invertible in the two-splitting regime in order to apply a variant of the Schwartz-Zippel lemma over low-splitting rings [BCPS18]. In Section 3.2, we prove the first generalization of Schwartz-Zippel to the high-splitting regime by connecting it to the previously introduced well-spreadness.

Soundness Analysis of Non-Interactive LaBRADOR. LaBRADOR is a multi-round public-coin protocol with a structure partially resembling Bulletproofs-style [BCC⁺16, BBB⁺18] recursive protocols. Attema *et al.* [AFK22]⁹ recently proved that $(2\mu + 1)$ -round interactive protocols with $\mathbf{K} = (k_1, \dots, k_\mu)$ -tree special soundness give rise to Fiat-Shamir non-interactive AoK with knowledge error $\kappa \in O(Q \cdot \kappa')$ in the random oracle model, where Q is the number of RO queries made by a cheating prover and κ' is a knowledge error of the underlying interactive protocol. Unfortunately, the result of [AFK22] doesn't immediately allow us to derive a concrete knowledge error for Fiat-Shamir LaBRADOR because (interactive) LaBRADOR doesn't satisfy the $\mathbf{K}$ -tree special soundness, as we explain in Section 4. There are three main technical hurdles when adapting AFK: (1) If the challenge space is imperfect (i.e. not every challenge difference is invertible in $\mathcal{R}_{q'}$), a tree with edges labeled by distinct challenges does not necessarily allow for extraction. Thus, we must take into account the probability that the extractor fails by hitting a bad challenge. (2) AFK only covers the case where extraction of a valid witness (or a solution to some computational problem) is always successful once a tree of accepting transcripts is given, whereas in LaBRADOR only a candidate witness is obtained and to check its validity the extractor must additionally perform probabilistic checks w.r.t. this fixed candidate, using a freshly sampled challenge from other rounds. (3) AFK only considers a tree of transcripts where each node is labeled by the prover's message, and each edge is labeled by a single challenge value. Its requirement of successful extraction is simply that the edges linked to the same node have distinct labels, whereas to extract a (candidate) witness in LaBRADOR, one needs a tree of transcripts where each edge is labeled by a *vector* of challenges in the amortization round and those vectors have to be distinct coordinate-wise.

To resolve the issues (1) and (2) altogether, in Section 5 we extend the $\mathbf{K}$ -tree special soundness notion with predicate system Φ , dubbed $(\mathbf{K}, \Phi)$ -*predicate-special-soundness (PSS)*. On a high-level, Φ is a collection of predicates defined for every level of a given $\mathbf{K}$ -tree of transcripts, describing “well-formedness” of sub-trees bottom-up. We then consider two types of predicates: *challenge predicates* which enforce special properties on the challenges for sibling nodes, and *commitment predicates* which enforce properties on committed values for sub-trees, helping to extract a valid witness. We then define *failure density* for every predicate to bound the number of bad challenges for an arbitrary fixed context, and use failure density to derive a knowledge error of Fiat-Shamir-transformed $(\mathbf{K}, \Phi)$ -PSS protocols (formally stated in Theorem 5.1). Like in the base result of AFK, we conclude that the concrete knowledge error is still linear in Q . To illustrate the usefulness of our PSS in an accessible manner, we use a bare-bones version of LaBRADOR (Protocol 1) as a running example. Finally, to address the issue (3) we further generalize the PSS notion by incorporating *coordinate-wise special soundness (CWSS)* of Fenzi and Nguyen [FMN23] in Section 5.4, applying the resulting framework to obtain the knowledge error of LaBRADOR in Section 5.5.

Estimates. Putting the aforementioned techniques together, we are able to provide parameter sets for our AS instantiated with LaBRADOR for Falcon-512 or Falcon-1024. In Section 7 and Appendix E, we compare our AS with (1) the GPV-based AS of [JRS23], and (2) the Merkle-tree-based AS of [FSZ22, FHSZ23]. We provide our program code in the repository at <https://github.com/dfaranha/aggregate-falcon>. We observe that for both parameter regimes starting from ca. 110 signatures, our AS is shorter than the trivial solution. For example, if $N = 2000$, our AS (with salts) achieves a compression rate of less than 13%. For $N = 8192$, it even goes down to less than 8%. Without random salts, e.g. for deterministic or

⁹ In an independent and concurrent work Wikström achieved a similar result, using alternate techniques [Wik21]. We will focus on the result of Attema *et al.* as we take departure from their approach. In Appendix A.2 we compare our approach to other related works on concrete analysis of Fiat-Shamir, including [LNP22, BF23].

synchronized Falcon variants, the compression rates further drop to less than 7% and 2%, respectively. We also show that our AS is significantly shorter than [JRS23] and [FSZ22]. Although [FHSZ23] outputs slightly smaller aggregate signatures, our approach has advantages in that it is provably secure under the standard security notion for AS (in contrast to the “synchronized” and limited-life-cycle model of [FSZ22, FHSZ23]), and that it is compatible with the standardized Falcon scheme. If we move to the synchronized model as [FSZ22, FHSZ23], we can avoid including the salt resulting in aggregate signature sizes which are significantly smaller than [FSZ22, FHSZ23] for all proposed parameters. We also provide concrete estimates of computation times for polynomial arithmetic, and confirm that our instantiation of LaBRADOR is optimal for our use case of aggregation.

From SNARKs to Aggregate Signatures. Although it might be tempting to conclude the security of SNARK-based aggregate signatures AS assuming knowledge soundness of the argument system Π and EU-CMA security of the signature scheme S , there is a subtle gap that was already pointed out in a different context [FN16].

An adversary $\mathcal{A}$ forging an aggregate signature outputs a valid aggregate signature in the form of a valid proof π for the argument system Π . Reducing to EU-CMA of S requires extracting a witness for π , which contains a forged signature σ . For the proof systems which we are interested in knowledge soundness allows the extractor to rewind the prover, our adversary, during extraction. Here we encounter our problem, the prover may alter its behavior when rewound, making different queries to the signing oracle. For the reduction to succeed we must be able to answer these queries, without making additional signing queries in the EU-CMA, which could result in the extracted signatures no longer being forgeries.

Fiore and Nitulescu [FN16] in fact showed the existence of powerful signing oracles that completely undermine knowledge soundness for *any* SNARK (in the standard model), assuming universal one-way hash functions. This means that, one cannot prove the security of generic SNARK-based AS from arbitrary Π and S ; instead, one has to prove that a certain class of signing oracles do not interfere with witness extraction for Π and thus successful reduction to EU-CMA security of S . To this end, we extend one of the positive results of [FN16] to prove that the reduction indeed succeeds even if $\mathcal{A}$ has access to a signing oracle for, both salted and deterministic, hash-then-sign-type schemes (in the random oracle model). In our theorem, the only additional requirement for Π is that its knowledge soundness holds against an adversary receiving auxiliary input consisting of random elements in the PSF range and their corresponding preimages; this requirement is rather mild since the elements are generated independently of the CRS and/or random oracle used by an argument system Π . Our general results (see Appendix C) are not tailored to specific instantiations of hash-then-sign and argument systems, and thus they may be of independent interest for future designers of SNARK-based AS constructions.

2 Preliminaries

2.1 Notation

Let q be an odd prime and d a power of 2. The $2d$ -th cyclotomic ring is defined as $\mathcal{R} = \mathbb{Z}[X]/\langle X^d + 1 \rangle$. Throughout this paper, we work over $\mathcal{R}$ modulo q , denoted by $\mathcal{R}_q = \mathbb{Z}_q[X]/\langle X^d + 1 \rangle$. We use bold letters for polynomials $\mathbf{r} \in \mathcal{R}_q$ to differentiate them from integers $r \in \mathbb{Z}_q$. This pattern extends to vectors $\vec{\mathbf{v}} \in \mathcal{R}_q^n$, $\vec{v} \in \mathbb{Z}_q^n$ and matrices $\mathbf{A} \in \mathcal{R}_q^{m \times n}$, $A \in \mathbb{Z}_q^{m \times n}$. For $\mathbf{r} = \sum_{i=0}^{d-1} r_i X^i \in \mathcal{R}_q$, we use $\text{ct}(\mathbf{r})$ to denote its constant coefficient r_0 . Vector concatenation is denoted $\vec{v}_1 \parallel \vec{v}_2$. Several norms are of interest in this paper. For $\vec{v} \in \mathbb{Z}_q^n$ and $p \in \{1, 2, \infty\}$, we define $\|\vec{v}\|_p$ as the ℓ_p -norm of its unique representative in $[\pm \frac{q-1}{2}]^n := [-\frac{q-1}{2}, \dots, +\frac{q-1}{2}]^n$. We define norms over $\mathcal{R}_q$ with respect to the coefficient embedding $\tau : \mathcal{R}_q \rightarrow \mathbb{Z}_q^d$, mapping $\mathbf{r}$ to the vector $(r_0, \dots, r_{d-1})$. Thus, for $\mathbf{r} \in \mathcal{R}_q$, $\|\mathbf{r}\|_p := \|\tau(\mathbf{r})\|_p$. This can naturally be generalized to norms over $\mathcal{R}_q^n$ by extending the coefficient embedding to $\tau : \mathcal{R}_q^n \rightarrow \mathbb{Z}_q^{nd}$, $\vec{\mathbf{v}} \mapsto \tau(\mathbf{v}_1) \parallel \tau(\mathbf{v}_2) \parallel \dots \parallel \tau(\mathbf{v}_n)$. Additionally, we define the operator norm of $\mathbf{r} \in \mathcal{R}_q$ as

$$\|\mathbf{r}\|_{\text{op}} = \max_{\mathbf{s} \in \mathcal{R}_q} \frac{\|\mathbf{r}\mathbf{s}\|_2}{\|\mathbf{s}\|_2}.$$

We write $r \leftarrow \mathcal{D}$ to denote that r was sampled from the distribution $\mathcal{D}$. When sampling uniformly at random from a set S , we use the shorthand $r \xleftarrow{\$} S$. For a positive integer n we let $[n] = \{1, \dots, n\}$.

2.2 Power-of-Two Cyclotomic Rings

Let d be a power-of-two and $l \mid d$ such that $q = 2l + 1 \pmod{4l}$. This condition ensures that $\mathbb{Z}_q$ contains a primitive $2l$ -th root of unity ζ , but no element of order a larger power of 2. Then modulo q , $X^d + 1$ factors

as the product of l irreducible polynomials $X^\delta - \zeta_i$ of degree $\delta = d/l$, where $\zeta_i = \zeta^{2^{i-1}}$ for $i \in \{1, \dots, l\}$ [LS18, Cor. 1.2]. By the Chinese remainder theorem (CRT), it follows that the ring $\mathcal{R}_q = \mathbb{Z}_q[X]/\langle X^d + 1 \rangle$ splits into the product of l residue fields

$$\mathcal{R}_q \cong \mathbb{Z}_q[X]/\langle X^\delta - \zeta_1 \rangle \times \dots \times \mathbb{Z}_q[X]/\langle X^\delta - \zeta_l \rangle.$$

We call l the *split factor* and δ the *split ratio* of $\mathcal{R}_q$.

For a given degree d , we can vary how to choose the modulus q in order to achieve different splitting behaviors, which in turn affect the mathematical properties of $\mathcal{R}_q$. There are mainly three splitting regimes studied in the literature. We call $\mathcal{R}_q$ *fully-splitting* if $l = d$ and thus $\delta = 1$. We call it *almost-fully-splitting* if $l = d/2^c$ and thus $\delta = 2^c$ for a small positive integer c . In this paper, we focus on $c \in \{2, 3\}$. Lastly, we say that $\mathcal{R}_q$ is *two-splitting* if $l = 2$ and thus $\delta = d/2$.

For an element $\mathbf{r} \in \mathcal{R}_q$, we call $\mathbf{r} \bmod (X^\delta - \zeta_i)$ the i -th CRT-slot of $\mathbf{r}$. An element is invertible in $\mathcal{R}_q$ if and only if all of its CRT-slots are non-zero. We denote by $\mathcal{R}_q^\times$ the set of invertible ring elements.

Power-of-two cyclotomic rings enjoy a special popularity in the design of cryptographic schemes as they come with nice properties. As detailed out in [LNPS21, Sec. 2.8], it is possible to nicely work over subrings of $\mathcal{R}$. More precisely, there is a norm-preserving bijection $\phi: \mathcal{R} \rightarrow \mathcal{S}^c$, where $\mathcal{R}$ is of degree d and $\mathcal{S}$ is of degree $d' = d/c$ for some positive integer c . The bijection can be naturally extended to vectors over $\mathcal{R}$ and $\mathcal{S}$ and the respective quotient rings modulo q .

The ring $\mathcal{R}_q$ inherits its group of automorphisms $\text{Aut}(\mathcal{R}_q)$ from the Galois automorphisms of the $2l$ -th cyclotomic number field [ALS20], i.e., $\text{Aut}(\mathcal{R}_q) = \{\sigma_i \mid i \in \mathbb{Z}_{2l}^\times\} \cong \mathbb{Z}_{2l}^\times$, where σ_i is defined by $X \mapsto X^i$ and $\mathbb{Z}_{2l}^\times$ denotes the multiplicative group of unit of $\mathbb{Z}_{2l}$.

The conjugation automorphism σ_{-1} is of special interest to us in this paper. As observed in [LNP22, Lemma 2.4], for power-of-two cyclotomics, σ_{-1} relates inner products in $\mathcal{R}_q^n$ to the inner products of their coefficient vectors by

$$\langle \tau(\vec{a}), \tau(\vec{b}) \rangle = \text{ct} \left(\langle \sigma_{-1}(\vec{a}), \vec{b} \rangle \right) \text{ for } \vec{a}, \vec{b} \in \mathcal{R}_q^n. \quad (1)$$

In particular, we have that $\|\vec{a}\|_2^2 = \text{ct}(\langle \sigma_{-1}(\vec{a}), \vec{a} \rangle)$. The j -th coefficient of some $\mathbf{a} = \sum_{i=1}^{d-1} a_i X^i \in \mathcal{R}_q$ can be retrieved through $\text{ct}(\sigma_{-1}(X^j) \mathbf{a}) = a_j$, as multiplying by X^{-j} shifts the j -th coefficient to the first position.

Lemma 2.1 ([AL21, Prop. 2]). *The expansion factor of $\mathcal{R}$ is defined as $\gamma_{\mathcal{R}} := \max_{\mathbf{a}, \mathbf{b} \in \mathcal{R}} \frac{\|\mathbf{a}\mathbf{b}\|_\infty}{\|\mathbf{a}\|_\infty \|\mathbf{b}\|_\infty}$. When d is a power of 2, $\gamma_{\mathcal{R}} \leq d$.*

2.3 Module-SIS

Definition 2.1 ([LS15]). *Let $n, m, \beta \in \mathbb{N}$. The Module Short Integer Solution problem M-SIS $_{n,m,\beta}$ over $\mathcal{R}_q$ is defined as follows. Given $\mathbf{A} \xleftarrow{\$} \mathcal{R}_q^{n \times m}$, find an $\vec{x} \in \mathcal{R}_q^m$ such that $\mathbf{A}\vec{x} = \vec{0}$ and $0 < \|\vec{x}\|_2 \leq \beta$.*

The M-SIS assumption states that no PPT adversary can solve this problem with non-negligible advantage. A classical worst-case to average-case reduction was provided in [LS15], showing that the M-SIS problem is at least as hard as finding a short basis in module lattices. The best known attacks for M-SIS $_{n,m,\beta}$ do not significantly depend on m [MR09], so it is often omitted. The number n is called the module rank of the M-SIS instance.

2.4 Aggregate Signatures, Falcon, and SNARKs

In Appendix B, we recall Falcon [PFH⁺22] and standard definitions for aggregate signatures (denoted by AS = (Setup, Gen, Sign, Ver, AggSign, AggVer), of which the first four define a usual signature scheme S), and (succinct) non-interactive arguments (denoted by $\Pi = (\mathcal{G}, \mathcal{P}, \mathcal{V})$).

2.5 LaBRADOR Proof System

In this subsection, we highlight core parts of the LaBRADOR protocol. The complete protocol description is provided in Appendix B.6.

Ring and Challenge Space. Their protocol is presented over a two-splitting $\mathcal{R}_q$. Thus, one can use [LS18, Corollary 1.2] to construct a challenge space $\mathcal{C} \subset \mathcal{R}_q$ satisfying the following properties relevant for soundness and compact proof sizes: (1) It is exponentially large in the security parameter, i.e., $|\mathcal{C}| \geq 2^\lambda$.

(2) Given two distinct challenges $\mathbf{c}, \mathbf{c}' \in \mathcal{C}$, their difference $\bar{\mathbf{c}} = \mathbf{c} - \mathbf{c}'$ is always invertible in $\mathcal{R}_q$. (3) The polynomials $\mathbf{c} \in \mathcal{C}$ have small norm. Let $T_2, T_{\text{op}} \in \mathbb{R}$ be such that $\|\mathbf{c}\|_2^2 \leq T_2$ and $\|\mathbf{c}\|_{\text{op}} \leq T_{\text{op}}$ for all $\mathbf{c} \in \mathcal{C}$. **Relation.** We now describe the relation R for which LaBRADOR has been designed. Informally, the relation R consists of a collection of dot product constraints and a norm bound check over $\mathcal{R}_q$. More formally, Let $\vec{\mathbf{w}}_1, \dots, \vec{\mathbf{w}}_r \in \mathcal{R}_q^n$ be r witness vectors of rank n . Each dot product constraint is defined by a function f of the form

$$f(\vec{\mathbf{w}}_1, \dots, \vec{\mathbf{w}}_r) = \sum_{i,j=1}^r \mathbf{a}_{i,j} \langle \vec{\mathbf{w}}_i, \vec{\mathbf{w}}_j \rangle + \sum_{i=1}^r \langle \vec{\varphi}_i, \vec{\mathbf{w}}_i \rangle - \mathbf{b} \in \mathcal{R}_q,$$

where $\mathbf{a}_{i,j}, \mathbf{b} \in \mathcal{R}_q$ and $\vec{\varphi} \in \mathcal{R}_q^n$, such that $\mathbf{a}_{i,j} = \mathbf{a}_{j,i}$ for all i, j . For some of these functions f' , we are only interested in the constant term $\text{ct}(\cdot)$ of their output. Therefore, we separate the full dot product constraints $\mathcal{F}$ from the constant term constraints $\mathcal{F}'$. Finally, we define the relation R such that $\vec{\mathbf{w}}_1, \dots, \vec{\mathbf{w}}_r$ is a witness for the statement $(\mathcal{F}, \mathcal{F}', \beta)$ if and only if

$$(\forall f \in \mathcal{F}, f(\vec{\mathbf{w}}_1, \dots, \vec{\mathbf{w}}_r) = \mathbf{0}) \wedge (\forall f' \in \mathcal{F}', \text{ct}(f'(\vec{\mathbf{w}}_1, \dots, \vec{\mathbf{w}}_r)) = 0) \wedge \sum_{i=1}^r \|\vec{\mathbf{w}}_i\|_2^2 \leq \beta^2.$$

Modular Johnson-Lindenstrauss Lemma. To prove that the witness vectors have short ℓ_2 -norm, without sending the entire witness $\vec{\mathbf{w}}_1, \dots, \vec{\mathbf{w}}_r$ to the verifier, LaBRADOR uses the projection technique from [GHL22]. It is based on a modular version of the Johnson-Lindenstrauss lemma, which was introduced in [GHL22] and heuristically strengthened in [BS23]. Whereas [BS23] stated the result for the concrete security parameter $\lambda = 128$, we generalize it to arbitrary λ .

Lemma 2.2 (Adapted from [BS23], heuristic). *Let $\mathcal{C}$ be the distribution where 0 has probability $\frac{1}{2}$ and ± 1 both have probability $\frac{1}{4}$. Let $q \in \mathbb{N}$. For every $\lambda \in \mathbb{N}$, there exist constants $C_1 = C_1(\lambda)$ and $C_2 = C_2(\lambda)$ such that the following holds. For every vector $\vec{\mathbf{w}} \in \mathbb{Z}_q^n$ with $\|\vec{\mathbf{w}}\|_2 \geq b$ for some bound $b \leq q/C_1$,*

$$\Pr_{\Pi \leftarrow \mathcal{C}^{2\lambda \times n}} \left[\|\Pi \vec{\mathbf{w}} \bmod q\|_2 < \sqrt{C_2 b} \right] \lesssim 2^{-\lambda},$$

where $\lesssim$ stresses the heuristic nature of the result.

We provide a Python script `jl.py`, accessible in our repository, which computes the constants for different security levels λ . In this work, we are interested in two security levels. For $\lambda = 128$, we obtain $C_1 = 120$ and $C_2 = 30$.¹⁰ For $\lambda = 256$, we obtain $C_1 = 168$ and $C_2 = 60$.

To prove that $\vec{\mathbf{w}} \in \mathcal{R}_q^n$ is short, the verifier sends the random projection matrix $\Pi \leftarrow \mathcal{C}^{2\lambda \times (nd)}$, and the prover responds with the projection $\vec{\mathbf{p}} = \Pi \tau(\vec{\mathbf{w}}) \in \mathbb{Z}_q^{2\lambda}$. If $\|\vec{\mathbf{w}}\|_2 \leq \beta$, then the expected ℓ_2 -norm of $\vec{\mathbf{p}}$ is $\sqrt{\lambda}\beta$. The verifier checks whether $\|\vec{\mathbf{p}}\|_2 \leq \sqrt{\lambda}\beta$, which happens in the honest case with probability $1/2$. If this norm check holds and $\sqrt{\lambda/C_2}\beta < q/C_1$, then by the lemma, $\|\vec{\mathbf{w}}\|_2 \leq (\sqrt{\lambda/C_2})\beta$ with overwhelming probability. Hence, this is an approximate norm proof of constant size with *slack* $\sqrt{\lambda/C_2}$. For both $\lambda \in \{128, 256\}$, the slack equals $\sqrt{128/30} \approx 2$. Notice that in the non-interactive variant, the projection matrix Π can be generated from a short λ -bit seed.

Ajtai commitments and weak openings. LaBRADOR commits to the short witness vectors with Ajtai commitments $\vec{\mathbf{v}} = \mathbf{A}\vec{\mathbf{w}}$ [Ajt96]. These are binding but not hiding. Given openings $\vec{\mathbf{w}}_1^* \neq \vec{\mathbf{w}}_2^*$ to $\vec{\mathbf{v}}$ of norm β , one obtains an M-SIS solution $\vec{\mathbf{w}}_1^* - \vec{\mathbf{w}}_2^*$ for $\mathbf{A}$ of norm 2β . They also use the notion of weak openings from [ALS20]. A weak opening of norm β for $\vec{\mathbf{v}}$ is a vector $\vec{\mathbf{w}}^*$ and a challenge difference $\bar{\mathbf{c}} \in \mathcal{C} - \mathcal{C}$ such that $\vec{\mathbf{v}} = \mathbf{A}\vec{\mathbf{w}}^*$ and $\|\bar{\mathbf{c}}\vec{\mathbf{w}}^*\|_2 \leq \beta$. Given two weak openings of norm β with $\vec{\mathbf{w}}_1^* \neq \vec{\mathbf{w}}_2^*$, we obtain an M-SIS solution for $\mathbf{A}$ of norm $4T_{\text{op}}\beta$. However, if $\bar{\mathbf{c}} = \bar{\mathbf{c}}'$, the M-SIS solution only has norm 2β .

3 Choice of Ring and Challenge Space

Let $\mathcal{R}_q = \prod_{i=1}^l \mathbb{Z}_q[X]/\langle X^d - \zeta_i \rangle$, where $d = l \cdot \delta$, with d the ring degree, l the split factor and δ the split ratio of $\mathcal{R}_q$, as introduced in Section 2.2.

The choice of ring and challenge space is hugely influential on the performance and proof sizes of lattice-based schemes. As recalled in Section 2.5, LaBRADOR [BS23] uses the properties of their two-splitting ring (i.e., $l = 2$) to design a useful challenge space $\mathcal{C} \subset \mathcal{R}_q$ for their protocol. In particular, they

¹⁰ Note that our constant C_1 for $\lambda = 128$ is slightly smaller than the one in [BS23], which was 125, and thus slightly tightens the result.

make use of the facts that in two-splitting rings 1) every non-zero element in $\mathcal{C}$ is invertible and 2) the size of each of the two CRT-slots is exponentially big in the ring degree d . However, direct computations in two-splitting rings are slow, as they do not benefit from the number theoretic transform (NTT).

To improve the performance of the ring, there are two approaches. The first approach is to choose a high-splitting ring. Computations in such rings are much more efficient, as they benefit from the NTT. However, this approach comes at the cost of larger proof sizes.

The second approach is to lift computations in the two-splitting ring $\mathcal{R}_q$ to a fully-splitting ring $\mathcal{R}_{q'}$ with a larger modulus, as in [CHK⁺21]. This approach improves performance while retaining the proof sizes of two-splitting rings. To compute a multiplication $\mathbf{c} = \mathbf{a} \cdot \mathbf{b}$ in $\mathcal{R}_q$, the idea is to compute the product in $\mathcal{R}_{q'}$ as if $\mathbf{a}, \mathbf{b}$ were elements in $\mathcal{R}_{q'}$. By Lemma 2.1, as long as $q' > d((q-1)/2)^2$, there will be no wrap around modulo q' . By reducing the product in $\mathcal{R}_{q'}$ modulo q , we obtain $\mathbf{c} \in \mathcal{R}_q$. For a single multiplication this might not be efficient, as we need to apply the NTT transform $\mathbf{a}, \mathbf{b}$ to compute the product in $\mathcal{R}_{q'}$, and then do the inverse NTT to recover the result. However, for LaBRADOR, we compute dot products $\langle \vec{\mathbf{a}}, \vec{\mathbf{b}} \rangle$ of vectors $\vec{\mathbf{a}}, \vec{\mathbf{b}} \in \mathcal{R}_q$ of large rank n . By setting $q' > nd((q-1)/2)^2$ and lifting the entire dot product computation in one go, the NTT cost is amortized over the cost of the n multiplications, making this approach efficient.

In Section 7, we estimate the performance of both of these approaches. For our use case, we opt for the lifting approach, as it yields both the shortest proofs and the fastest ring computations. Still, there could be other scenarios where one might prefer the high-splitting approach. Hence, we do not want to restrict the results in this paper to a particular choice of ring. For the rest of this section, we therefore introduce a common framework for rings and their challenge spaces. In particular, we introduce the property of well-spreadness, and demonstrate how it implies the properties we need for our challenge space. The proofs in this section are deferred to Appendix D.

3.1 Well-Spread Challenge Spaces

We begin by defining the property of well-spreadness of a challenge space.

Definition 3.1. Let $\mathcal{C} \subset \mathcal{R}_q$ and $B \in [0, 1]$. We say that $\mathcal{C}$ is B -well-spread if for all $i \in [l]$ and for all $\mathbf{y} \in \mathbb{Z}_q[X]/\langle X^\delta - \zeta_i \rangle$

$$\Pr_{\mathbf{c} \xleftarrow{\$} \mathcal{C}} [\mathbf{c} \bmod (X^\delta - \zeta_i) = \mathbf{y}] \leq B.$$

If B is the smallest such value, we say that $\mathcal{C}$ is tightly B -well-spread.

Informally, well-spreadness bounds the probability that any CRT-slot of a random challenge element hits a specific element. Implicitly, well-spreadness has already been shown for different challenge sets for high-splitting rings such as the one in [ALS20, Lem. 3.2], the one in [ESZ22, Lem. 1] and its generalization in [ESLR23, Lem. 1]. In two-splitting rings, using [LS18] we obtain $B = 1/|\mathcal{C}|$ if the challenge space only contains elements of small enough norms. Well-spreadness directly implies a bound on invertibility of randomly sampled challenges (by setting $\mathbf{y} = \mathbf{0}$ in the lemma below) as well as the invertibility of challenge differences (by setting $\mathbf{y} = \mathbf{c}'$ for $\mathbf{c}' \in \mathcal{C}$).

Lemma 3.1. Let $\mathcal{C} \subset \mathcal{R}_q$ be B -well-spread for $B \in [0, 1]$. Let $\mathbf{y} \in \mathcal{R}$ be an arbitrary ring element. It yields

$$\Pr_{\mathbf{c} \xleftarrow{\$} \mathcal{C}} [\mathbf{c} - \mathbf{y} \in \mathcal{R}_q^\times] \geq 1 - l \cdot B.$$

Thus, when B is negligible, challenge differences are invertible except with negligible probability. In Section 4, we consider how many challenges we can collect so that no pair has an invertible challenge difference. The following lemma provides a lower bound for this question. Note that for the challenge spaces of high-splitting rings, $B|\mathcal{C}|$ is superpolynomial in λ .

Lemma 3.2. Let $\mathcal{C} \subset \mathcal{R}_q$ be tightly B -well-spread. Then there exists a set $\mathcal{S} \subseteq \mathcal{C}$ of cardinality $B|\mathcal{C}|$ such that for all $\mathbf{s}_1, \mathbf{s}_2 \in \mathcal{S}, \mathbf{s}_1 \neq \mathbf{s}_2$ we have that $(\mathbf{s}_1 - \mathbf{s}_2) \notin \mathcal{R}_q^\times$.

3.2 Variant of Schwartz-Zippel Lemma

Informally, the Schwartz-Zippel lemma bounds the probability that a polynomial evaluated on random challenge elements yields the zero element. While the original lemma works over fields, we need to apply Schwartz-Zippel over the ring $\mathcal{R}_q$, which is in general not a field. We thus propose the following generalization of Schwartz-Zippel to rings with a well-spread challenge space.

Lemma 3.3. *Let $n \in \mathbb{N}$ and $\mathbf{f} \in \mathcal{R}_q[X_1, \dots, X_n]$ be a non-zero polynomial over $\mathcal{R}_q$ of total degree $\deg(\mathbf{f}) \geq 0$. Further, let $\mathcal{C} \subset \mathcal{R}_q$ be a B -well-spread challenge space for $B \in [0, 1]$. Let $\mathbf{c}_1, \dots, \mathbf{c}_n$ be sampled independently and uniformly at random from $\mathcal{C}$. Then*

$$\Pr[\mathbf{f}(\mathbf{c}_1, \dots, \mathbf{c}_n) = \mathbf{0}] \leq \deg(\mathbf{f}) \cdot B.$$

Remark 3.1. Note that LaBRADOR [BS23] already (implicitly) uses Schwartz-Zippel over the ring $\mathcal{R}_q$. However, they are making use of the specific structure of their two-splitting ring setting and their challenge space. In particular, by [LS18], *any* challenge difference $\mathbf{c} - \mathbf{c}'$ with $\mathbf{c} \neq \mathbf{c}'$ of LaBRADOR is invertible in $\mathcal{R}_q$, hence

$$\mathbf{c} - \mathbf{c}' \in \mathcal{R}_q^\times \Leftrightarrow \mathbf{c} - \mathbf{c}' \neq \mathbf{0} \bmod (X^\delta - \zeta_i) \forall i \in [l] \Leftrightarrow \mathbf{c} \neq \mathbf{c}' \bmod (X^\delta - \zeta_i) \forall i \in [l].$$

In other words, every two distinct challenges have different CRT-slots. On the one hand, this implies that for every $i \in [l]$, sampling uniformly at random from $\mathcal{C}$ and then reducing modulo $X^\delta - \zeta_i$ is the same as directly sampling uniform at random from $\mathcal{C} \bmod X^\delta - \zeta_i$. On the other hand, it implies that the size of $\mathcal{C} \bmod X^\delta - \zeta_i$ is the same as the size of $\mathcal{C}$ itself. One can thus deduce that in the two-splitting case

$$\Pr[\mathbf{f}(\mathbf{c}) = \mathbf{0}] \leq \deg(\mathbf{f})/|\mathcal{C}|.$$

Alternatively, one could use the results of [BCPS18, Thm. 4.2] proving Schwartz-Zippel over rings in which every challenge difference is invertible.

4 Special Soundness and its Limitations

Beullens and Seiler present the LaBRADOR protocol and prove knowledge soundness in the interactive setting, later applying the Fiat-Shamir transform assuming that security is retained heuristically [BS23]. In the following, we build towards a formal analysis of the Fiat-Shamir transformation of LaBRADOR.

Analyzing the knowledge error of multi-round protocols under the Fiat-Shamir transform is highly non-trivial. A naive analysis often blows up the extractor's run-time by a factor Q^μ , where Q is the number of random oracle queries made by the prover and μ the number of rounds of the protocol. Recently, [AFK22, Wik21] proved that multi-round special sound protocols remain knowledge sound under the Fiat-Shamir transform. For this class of protocols the knowledge error only blows up linearly in Q when compared to the interactive variant.

Initially, this approach seems promising for LaBRADOR. Indeed, it is a commit-and-open protocol and has a (coordinate-wise) special sound structure that allows the extractor to recover openings for the commitments. However, in the case of LaBRADOR special soundness does not allow enforcing that the extracted opening is actually a valid witness for the statement, i.e. the opening is short and satisfies the system of dot product constraints.

We proceed as follows. In Section 4.1, we give a recap on special sound protocols and their Fiat-Shamir security, defining some helpful notation along the way. Next, in Section 4.2, we introduce a toy example to illustrate why protocols like LaBRADOR can not be proven knowledge sound using special soundness alone. This motivates our generalization of special soundness to *predicate special soundness* presented in Section 5.

4.1 (Multi-Round) Special Soundness

In this section, we recap the notion of special sound protocols and the results on their knowledge soundness. For a recap on the Fiat-Shamir transform and proofs of knowledge in the ROM, see Appendix B. For the definition of interactive proofs of knowledge, we refer the reader to [ACK21]. For an NP relation R , we let $R(x) = \{w \mid (x, w) \in R\}$.

Definition 4.1 (k -special-soundness). *Let $k \in \mathbb{N}$ and let $\Pi = (\mathcal{P}, \mathcal{V})$ be a 3-message public coin proof/argument of knowledge for a relation R_{pp} . The prover sends the first message a , then the verifier sends a challenge c and the prover responds with z . We say that Π is k -special sound if there exists a polynomial time algorithm which on input a statement x and k accepting transcripts $(a, c_1, z_1), \dots, (a, c_k, z_k)$ for x with the same first prover message a and pairwise distinct verifier challenges $c_1, \dots, c_k \in \mathcal{C}$ outputs a witness w such that $w \in R_{\text{pp}}(x)$.*

This definition can be generalized to the multi-round setting, as in [ACK21]. To do so, it is more convenient to formulate special soundness in terms of trees of transcripts. For a 3-message protocol, k accepting transcripts $(a, c_1, z_1), \dots, (a, c_k, z_k)$ form a k -tree of transcripts. In this tree a is the root, and each challenge c_i is an edge from the root to the node z_i . Each path from the root to a leaf is a transcript.

In the multi-round setting, we have $(k_1, \dots, k_\mu)$ -trees of transcripts. Consider a $2\mu + 1$ -message interactive protocol $\Pi = (\mathcal{P}, \mathcal{V})$. A single transcript $(a_1, c_1, a_2, c_2, \dots, a_\mu, c_\mu, a_{\mu+1})$ is a $(1, \dots, 1)$ -tree of transcripts. Given k_μ transcripts that have the same prefix $(a_1, c_1, a_2, c_2, \dots, a_\mu, c_\mu)$ and pairwise distinct μ -th challenges, we obtain a $(1, \dots, 1, k_\mu)$ -tree of transcripts. We refer to the shared prefix as their trunk. Given $k_{\mu-1}$ $(1, \dots, 1, k_\mu)$ -trees of transcripts with the same trunk $(a_1, c_1, \dots, a_{\mu-1})$ and pairwise distinct $(\mu-1)$ -th challenges, we obtain a $(1, \dots, 1, k_{\mu-1}, k_\mu)$ -tree of transcripts, and so on. Collecting $K = \prod_{i=1}^\mu k_i$ transcripts in this manner, we obtain a $(k_1, \dots, k_\mu)$ -tree of transcripts. We refer the visually inclined reader to [ACK21] for an illustration of these trees.

For convenience, we assume that the first message includes the statement x . Since all the transcripts in a tree have the same first message, we thereby ensure that they are all for the same statement.

Definition 4.2 (K -special-soundness [ACK21]). Let $\mu, k_1, \dots, k_\mu \in \mathbb{N}$ and let Π be a $2\mu + 1$ -message public-coin proof/argument of knowledge for a binary relation R_{pp} . Let $\mathbf{K} = (k_1, \dots, k_\mu)$. We say that Π is $\mathbf{K}$ -special sound if there exists a polynomial time algorithm which on input a statement x and an accepting $\mathbf{K}$ -tree of transcripts for x outputs a witness w such that $w \in R_{\text{pp}}(x)$.

When we later define the predicate special soundness framework in Section 5, we do so in terms of the trees of transcripts of a protocol. It will therefore be convenient to define some additional notation.

Definition 4.3 (Tree of Transcripts). Let $\mu, k_1, \dots, k_\mu \in \mathbb{N}$ and let $\Pi = (\mathcal{P}, \mathcal{V})$ be a $2\mu + 1$ -message public-coin argument of knowledge for a relation R_{pp} . Additionally, let $m \in [\mu]$ and $\ell \in [k_m]$. Let $\mathcal{C}_m$ be the m -th challenge set.

- We define $\mathbb{T}_{\mu+1}$ be the set of possible accepting transcripts for Π .
- We define $\mathbb{T}_{m+1}^{(\ell)}$ be the set of possible accepting $(1, \dots, 1, \ell, k_{m+1}, \dots, k_\mu)$ -trees of transcripts for Π , and denote $\mathbb{T}_m = \mathbb{T}_{m+1}^{(k_m)}$. Each $t \in \mathbb{T}_{m+1}^{(\ell)}$ is a tuple $t = (t_1, \dots, t_\ell) \in \mathbb{T}_{m+1}^\ell$.
- For $t \in \mathbb{T}_{m+1}$, we define $\text{trunk}(t)$ to be the prefix $(a_1, c_1, a_2, c_2, \dots, a_m)$ shared by all the transcripts in t , and $\text{chal}_i(t)$ for $i \in [m]$ to be the i -th challenge $c_i \in \mathcal{C}_i$ shared by the transcripts.
- Let $\mathcal{C}_m^{(\ell)}$ be the set of tuples $(c_1, \dots, c_\ell) \in \mathcal{C}_m$ with $c_{i_1} \neq c_{i_2}$ for all $i_1 \neq i_2$. These are all the combinations of m -th challenges that may occur in $\mathbb{T}_{m+1}^{(\ell)}$.

The interactive knowledge soundness of $\mathbf{K}$ -special sound protocols was formally analyzed in [ACK21]. In subsequent work [AFK22] and [Wik21] analyzed the Fiat-Shamir transformation of $\mathbf{K}$ -special sound protocols.

Theorem 4.1 ([AFK22]). Let Π be any $(k_1, \dots, k_\mu)$ -special sound protocol. For $i \in [\mu]$, let N_i be the cardinality of the i -th challenge space. Then $\text{FS}[\Pi]$ is adaptively knowledge sound with knowledge error $(Q + 1)\kappa$, where Q is an upper bound on the number of random oracle queries made by the prover and κ the knowledge error of Π ,

$$\kappa = \frac{\prod_{i=1}^\mu N_i - \prod_{i=1}^\mu (N_i - k_i + 1)}{\prod_{i=1}^\mu N_i} \leq \sum_{i=1}^\mu \frac{k_i - 1}{N_i}.$$

Their extractor invokes the prover in expectation at most $K + Q(K - 1)$ times, where $K = \prod_{i=1}^\mu k_i$.

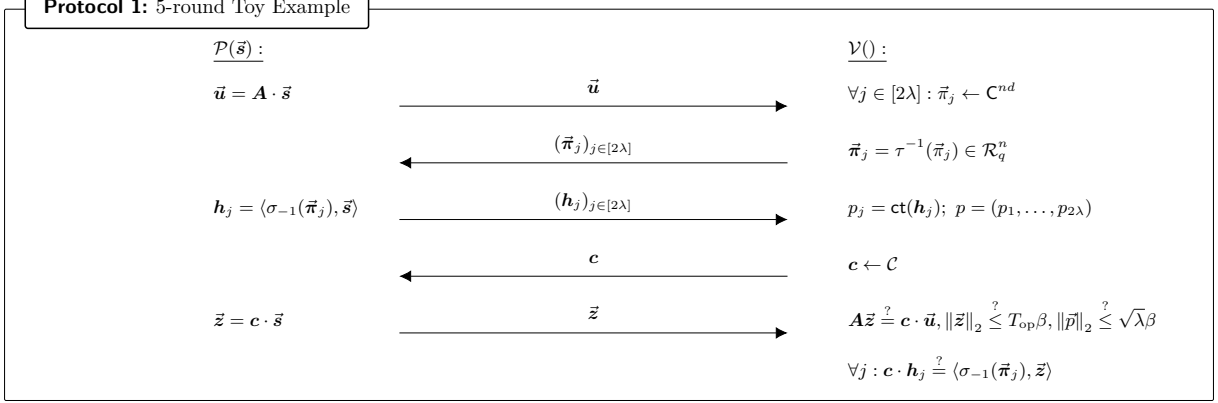
The bound on κ is the knowledge error of the interactive case, as shown in [ACK21].

4.2 Limitations of Special Soundness: A Toy Example

To highlight why protocols like LaBRADOR *cannot* be proven knowledge sound with the special soundness framework, we present a toy example: Protocol 1. It is a commit-and-open protocol, proving knowledge of a short vector. The toy example can be viewed as a bare-bones version of the base LaBRADOR protocol where there is only one witness vector and no dot product constraints.

We begin by presenting the toy protocol. The prover $\mathcal{P}$ is given a short witness $\vec{s} \in \mathcal{R}_q^n$ and sends the commitment $\vec{u} = \mathbf{A} \cdot \vec{s}$ to the verifier, using a public matrix $\mathbf{A} \in \mathcal{R}_q^{\kappa \times n}$. To prove that $\vec{s}$ is short, the verifier $\mathcal{V}$ sends a Johnson-Lindenstrauss projection challenge to $\mathcal{P}$ (as in Section 2.5). Concretely, $\mathcal{V}$

Protocol 1: 5-round Toy Example



samples $\vec{\pi}_j$ as the rows of the projection matrix, interprets them as ring vectors $\vec{\pi}_j$ using the coefficient embedding τ and sends them to $\mathcal{P}$. $\mathcal{P}$ computes the response $\mathbf{h}_j = \langle \sigma_{-1}(\vec{\pi}_j), \vec{s} \rangle$ for each j . Observe that $\text{ct}(\langle \sigma_{-1}(\vec{\pi}_j), \vec{s} \rangle) = \langle \vec{\pi}_j, \tau(\vec{s}) \rangle$ is the j -th coordinate of the projection of $\vec{s}$. Finally, $\mathcal{V}$ sends a challenge $\mathbf{c}$ and the prover responds with the opening $\vec{z} = \mathbf{c}\vec{s}$. This last step is somewhat artificial, as we neither have zero-knowledge nor multiple witnesses, but it allows illustrating the issues encountered with proving knowledge soundness when challenge differences are not always invertible. $\mathcal{V}$ checks that $\vec{z}$ and $\vec{p}$ are short and that the correct relations hold with respect to $\mathbf{c}$.

We want to show that Protocol 1 is an argument of knowledge for the relation

$$R_{\text{pp}} = \{(\beta, \vec{s}) \in \mathbb{N} \times \mathcal{R}_q^n \mid \|\vec{s}\|_2 \leq 2\beta\}.$$

This is the LaBRADOR relation without dot product constraints. Like LaBRADOR, the toy example is not an exact proof of shortness.¹¹ When the honest prover uses a vector of norm β , we are only able to extract a vector roughly twice as large (see Section 2.5 for the precise slack).

Let us consider whether (k_1, k_2) -special soundness may be applied to Protocol 1 for reasonable choices of k_1 and k_2 . In doing so we face two problems.

Challenge invertibility. The usual approach to extracting an opening is to find two accepting transcripts with distinct challenges $\mathbf{c} \neq \mathbf{c}'$ such that $\mathbf{c} - \mathbf{c}'$ is invertible. Then from the prover's last message, we can compute a weak opening (see Section 2.5) to $\vec{u}$, i.e. $\vec{s}^* = \bar{\mathbf{c}}^{-1}(\vec{z} - \vec{z}')$ for $\bar{\mathbf{c}} = \mathbf{c} - \mathbf{c}'$ where the bound $\|\bar{\mathbf{c}}\vec{s}^*\|_2 \leq \|\vec{z}\|_2 + \|\vec{z}'\|_2 = 2T_{\text{op}}\beta$ holds.

We need to set k_2 large enough to guarantee that we obtain two transcripts with an invertible challenge difference. This depends heavily on the choice of ring and challenge space. For a two-splitting ring (with appropriate degree and modulus) there exist exponentially large challenge spaces where the difference of any distinct challenges is invertible [LS18]. Hence, in this case we can set $k_2 = 2$.

However, for high-splitting rings this is not the case. Recall from Section 3 that the challenge spaces for these rings are such that the difference of two independently sampled challenges is invertible with overwhelming probability, but *not* always. The well-spreadness of a challenge space from Definition 3.1 allows us to lower-bound k_2 . If $\mathcal{C}$ is tightly B -well-spread, then by Lemma 3.2 we need to set k_2 to be at least $B|\mathcal{C}|$. While B can be constructed to be negligible in the security parameter, $B|\mathcal{C}|$ will typically be super-polynomial. Thus, the extractors of [ACK21, AFK22] will not run in expected polynomial time.

Shortness of the extracted vector. Assume that we could resolve the previous issue. We have a k_2 of polynomial size such that an accepting $(1, k_2)$ -tree of transcript defines a short weak opening. Still, this does not guarantee that the extracted vector $\vec{s}^*$ is short.

To enforce that the extracted vector is short, we want to use the Johnson-Lindenstrauss lemma (Lemma 2.2). For a fixed witness that is not short, the lemma tells us that there is only a small fraction of the projection challenges that the prover can answer honestly. In our case, the projection of an accepting $(1, k_2)$ -tree must be computed honestly with respect to its vector $\vec{s}^*$.

$$\begin{aligned} \mathbf{c} \cdot \mathbf{h}_j &= \langle \sigma_{-1}(\vec{\pi}_j), \vec{z} \rangle, & \Rightarrow & \quad \mathbf{h}_j = \langle \sigma_{-1}(\vec{\pi}_j), \bar{\mathbf{c}}^{-1}(\vec{z} - \vec{z}') \rangle = \langle \sigma_{-1}(\vec{\pi}_j), \vec{s}^* \rangle. \\ \mathbf{c}' \cdot \mathbf{h}_j &= \langle \sigma_{-1}(\vec{\pi}_j), \vec{z}' \rangle \end{aligned}$$

¹¹ Due to the simplicity of R_{pp} there exists a trivial extractor that simply computes a witness on its own, without invoking the prover. Regardless, we will discuss how to extract from the prover, as Protocol 1 only serves to illustrate the challenges of extraction for more complex protocols and relations.

Thus, if we rewind the prover and provide some projection challenge such that the projection of its witness is not short, then the prover’s only option is to break the binding of the commitment $\bar{u}$, and provide another weak opening. The strategy is therefore to set k_1 larger than the total number of challenges that a fixed large vector can use. Then, an accepting (k_1, k_2) -tree guarantees that we either obtain a short opening $\bar{s}^*$ or a solution to M-SIS. However, we run into the same issue as before. While the fraction of projection challenges a large vector can use is negligible in the security parameter, the number of them is exponential! If we need to set k_1 exponentially large, we can not extract in expected polynomial time. In conclusion, there is no clear way to apply special soundness to Protocol 1, regardless of the choice of ring and challenge space.

5 Predicate Special Soundness

In this section, we prove the Fiat-Shamir transformation of LaBRADOR [BS23] knowledge sound. Recall that LaBRADOR is a public-coin recursive folding protocol reminiscent of Bulletproofs [BCC⁺16, BBB⁺18]. The base protocol has a commit-and-open structure, where the prover initially commits to its witness elements, and concludes by sending an amortized opening to the commitments. The challenges in the intermediate rounds define probabilistic tests that enforce properties on the committed values. If a committed value does not satisfy one of the properties, i.e. it is not short or does not satisfy some dot product constraint, then there is only a negligible fraction of the challenges that the prover can use to make the verifier accept.

In the interactive setting, the soundness of LaBRADOR follows from the fact that only a negligible fraction of the challenges allow the prover to convince the verifier for a fixed invalid witness.

Moving to the non-interactive setting, challenges are now defined by queries to the random oracle, which depend on the commitment to the witness. Unless the prover can break the binding of the commitment scheme, they cannot choose an opening for the commitment after seeing their challenges. At least intuitively, the soundness argument should then, with some loss, transfer to the non-interactive setting; the prover must search for an input to the random oracle which allows them to cheat, with their only hope being for an output of one of the Q random oracle queries to be in negligible fraction of challenges that it can use. If p is an upper bound on the fraction of challenges that a prover can use with an invalid witness and κ_{binding} is the prover’s advantage in breaking the binding of the commitment scheme, then by a union bound $Qp + \kappa_{\text{binding}}$ seems like a natural bound for the knowledge error. If a prover has non-negligible probability of convincing the verifier for some commitment, then it should follow that the extracted opening for said commitment should be a valid witness.

We capture this intuition for the knowledge soundness of commit-and-open protocols like LaBRADOR with a new generalization of special soundness that we call *predicate special soundness* (PSS). A PSS protocol is associated with a special sound structure and a collection of predicates. The special soundness structure (mostly) defines how to extract an opening to the commitments. The predicates enforce additional properties on the extracted openings and the challenges used by the prover. Given a set of transcripts that satisfy both the special soundness requirement and the predicates, a valid witness can be computed. To analyze the knowledge soundness of PSS protocols, we extend the proof of [AFK22] for special sound protocols to handle the predicates. It turns out that the key to analyzing PSS protocols is the upper bound p on the fraction of challenges that the prover can use to satisfy a predicate with an invalid witness. We refer to this fraction as the *failure density* of the predicate. With this notion, we are able to prove the non-interactive LaBRADOR protocol knowledge sound.

In the remainder of this section we formally define predicate special sound protocols and present our result on their security in the non-interactive setting. Since the definitions can be quite heavy at times, we demonstrate how they are applied along the way with our toy example Protocol 1. In Section 5.4, we mention how to generalize PSS to coordinate-wise special sound structures, obtaining coordinate-wise predicate special soundness (CWPSS). This is required to extract from the amortized opening at the last round of the LaBRADOR protocol. Finally, in Section 5.5 we apply the PSS framework to LaBRADOR and show that it is knowledge sound under the Fiat-Shamir transform.

5.1 Challenge predicates

To make requirements of transcripts beyond special soundness we must first introduce a mechanism for describing our desired properties. We will start with properties of challenges in the tree, and then proceed to show how properties of committed values may be derived.

Challenge predicates allow enforcing relationships between challenges beyond them simply being distinct. Level m has k_m challenge predicates.

Definition 5.1 (Challenge Predicates). Let $m \in [\mu]$ and $\ell \in [k_m]$. A challenge predicate on level m for the ℓ th challenge is a polynomial-time computable function $\Phi_{m,\ell}^{\text{chal}} : \mathcal{C}_m^{(\ell)} \rightarrow \{0, 1\}$.

For a tree $t \in \mathbb{T}_m$ to be valid, its m -th challenges $\mathbf{c}_1, \dots, \mathbf{c}_{k_m} \in \mathcal{C}_m$ must satisfy the m -th level challenge predicates. This means that

$$\Phi_{m,i}^{\text{chal}}(\mathbf{c}_1, \dots, \mathbf{c}_i) = 1 \quad \forall i \in [k_m].$$

When checked in sequence, each predicate may be seen as ensuring guarantees for one new challenge at a time with respect to the set of previous challenges.

To see how a challenge predicate may be applied, let us recall Protocol 1. In the final round the prover provides an opening $\vec{z}$. A witness may be computed given two openings for a pair of challenges whose difference is invertible. If the protocol is instantiated with a challenge space where challenge differences are not always invertible, then invertibility must be enforced as an additional property. To do this let $\Phi_{2,2}^{\text{chal}}(\mathbf{c}_1, \mathbf{c}_2) = 1 \Leftrightarrow \bar{\mathbf{c}} = (\mathbf{c}_1 - \mathbf{c}_2) \in \mathcal{R}_q^\times$, where $\mathbf{c}_1, \mathbf{c}_2$ come from the two accepting transcripts t_1, t_2 such that $(t_1, t_2) \in \mathbb{T}_2$. This challenge predicate is all we need to extract our desired weak opening. To disambiguate variables between two transcripts t_1 and t_2 , we refer to them using the same subscript as their transcript. We have that

$$\mathbf{A}(\vec{z}_1 - \vec{z}_2) = (\mathbf{c}_1 - \mathbf{c}_2)\vec{u}.$$

If $\Phi_{2,2}^{\text{chal}}(\mathbf{c}_1, \mathbf{c}_2) = 1$ then $\vec{s}^* = \bar{\mathbf{c}}^{-1}(\vec{z}_1 - \vec{z}_2)$ constitutes a weak opening, where $\mathbf{A}\vec{s}^* = \vec{u}$ and $\|\bar{\mathbf{c}}\vec{s}^*\|_2 \leq 2T_{\text{op}}\beta$. Furthermore, for $j \in [2\lambda]$ we have $\bar{\mathbf{c}} \cdot \mathbf{h}_j = \langle \vec{\pi}_j, \vec{z}_1 - \vec{z}_2 \rangle$. Again, if $\Phi_{2,2}^{\text{chal}}(\mathbf{c}_1, \mathbf{c}_2) = 1$ then $\mathbf{h}_j = \langle \vec{\pi}_j, \vec{s}^* \rangle$ which implies that the projection was correctly computed, $\text{ct}(\langle \vec{\pi}_j, \vec{s}^* \rangle) = p_j$.

When is extraction possible? Not all predicates allow for efficient extraction. We describe one large natural class of predicates that does, characterizing them in terms of their *failure density*. The failure density of our predicates directly contributes to the knowledge error of the protocol.

Given that we found $\ell - 1$ “good” challenges, the failure density of the challenge predicate $\Phi_{m,\ell}^{\text{chal}}$ bounds the fraction of ℓ th challenges that cause it to fail.

Definition 5.2 (Failure Density of Challenge Predicates). Let $m \in [\mu]$ and $\ell \in [k_m]$. Let $(\mathbf{c}_1, \dots, \mathbf{c}_{\ell-1}) \in \mathcal{C}_m^{(\ell-1)}$ be any $\ell - 1$ distinct challenges such that $\Phi_{m,i}^{\text{chal}}(\mathbf{c}_1, \dots, \mathbf{c}_i) = 1$ for all $i \in [\ell - 1]$. Consider the set of possible ℓ -th challenges such that $\Phi_{m,\ell}^{\text{chal}}$ fails,

$$\text{BadChal}_{m,\ell}(\mathbf{c}_1, \dots, \mathbf{c}_{\ell-1}) = \{c \in \mathcal{C}_m \setminus \{\mathbf{c}_1, \dots, \mathbf{c}_{\ell-1}\} \mid \Phi_{m,\ell}^{\text{chal}}(\mathbf{c}_1, \dots, \mathbf{c}_{\ell-1}, c) = 0\}.$$

The challenge predicate $\Phi_{m,\ell}^{\text{chal}}$ has failure density $p_{m,\ell}^{\text{chal}}$ if it always holds that $|\text{BadChal}_{m,\ell}(\mathbf{c}_1, \dots, \mathbf{c}_{\ell-1})| \leq p_{m,\ell}^{\text{chal}}|\mathcal{C}_m|$.

Note, failure density constitutes a clear break with the conventional notion of special soundness as we only require the proportion “bad” challenges to be small, rather than the actual number.

Let us return to our example: Protocol 1 and the challenge predicate $\Phi_{2,2}^{\text{chal}}(\mathbf{c}_1, \mathbf{c}_2)$. In this case if $\mathcal{C}$ is B -well-spread (cf. Definition 3.1) and $\mathcal{R}_q$ has a split factor l , then by Lemma 3.1 the probability that a uniformly sampled $\mathbf{c} \in \mathcal{C}$ is such that $\mathbf{c}_1 - \mathbf{c}$ is not a unit is at most lB . By the uniform distribution, it follows that

$$|\{c \in \mathcal{C} \mid \mathbf{c} \neq \mathbf{c}_1, \Phi_{2,2}^{\text{chal}}(\mathbf{c}_1, c) = 0\}| \leq lB|\mathcal{C}|,$$

implying a failure density $p_{2,2}^{\text{chal}} = lB$. In general, for uniformly sampled challenges, the failure probability exactly matches the failure density.

5.2 Commitment predicates

Commitment predicates allow enforcing properties on committed values. Recall that in commit-and-open protocols, the challenges in the intermediate rounds may define probabilistic tests for enforcing properties on the committed witness. These tests are usually defined such that any fixed witness only allows a prover to cheat with a given probability p for an independently chosen challenge, or alternatively, there are only a bounded fraction p of challenges that a fixed witness can use to cheat. Informally, the fraction p will be the failure density of the associated commitment predicate.

Let us consider a protocol which is $(2, k_2, \dots, k_\mu)$ -special-sound, where each $(1, k_2, \dots, k_\mu)$ -subtree allows computing a witness which was previously committed to. Given just one of these two subtrees the extracted witness may depend on the challenge used for the probabilistic test. However, if the first challenge in the second $(1, k_2, \dots, k_\mu)$ -subtree is sampled independently, then for the same witness there

is a low probability the challenge is in the “bad” set. This allows the guarantees of the test to be obtained with probability close to $1 - p$. In the case where the witness found for the second subtree is distinct we have instead found a violation of binding and no longer need to ensure our desired property.

In our framework we present a slight generalization of this approach, allowing a fork with k subtrees. On an intuitive level the first $k - 1$ subtrees allow extracting an opening and the final subtree must be consistent with that opening, breaking binding if it is inconsistent. Note, while we do not obtain two complete openings with this approach a single subtree may provide a partial opening, which for some commitment schemes is sufficient to break binding. The requirements to break binding are described through a binding predicate, while the property we wish to ensure is captured by a property predicate. We call a pair of a binding and property predicate a commitment predicate.

Definition 5.3 (Commitment Predicates). *Let $m \in [\mu]$ and $\ell \in [k_m]$. A commitment predicate on level m is a pair of polynomial-time computable functions $(\Phi_m^{\text{prop}}, \Phi_m^{\text{bind}})$, where*

$$\Phi_m^{\text{prop}} : \mathbb{T}_{m+1}^{(k_m-1)} \rightarrow \{0, 1\} \quad \text{and} \quad \Phi_m^{\text{bind}} : \mathbb{T}_{m+1}^{(k_m-1)} \times \mathbb{T}_{m+1} \rightarrow \{0, 1\}.$$

We return to our example to demonstrate the use of commitment predicates. Thus far, we have shown the necessary properties for a pair of transcripts to extract a weak opening and ensure a correctly computed projection. Specifically we obtain $(1, 2)$ -trees $t_i = (t_{i,1}, t_{i,2})$ where $\Phi_2(\text{chal}_2(t_{i,1}), \text{chal}_2(t_{i,2})) = 1$. At this point we cannot be sure that the opening we have extracted is independent of the projection, meaning that $\vec{s}^*$ might still be large.

If $\vec{s}$ is in fact large, then there is only a small fraction of the challenges that allow the prover to cheat, without breaking binding. We may define a commitment predicate, by defining a property predicate and a binding predicate. The property predicate describes the property of the witness we wish to enforce, in this case that the witness is short, i.e.

$$\Phi_1^{\text{prop}}(t_1) = 1 \Leftrightarrow \|\vec{s}_1^*\|_2 \leq 2\beta. \quad (2)$$

The binding predicate describes which values the prover is committed to between transcripts. If this predicate is violated we must be able to extract a solution to some hard problem. In this case we rely on M-SIS.

$$\Phi_1^{\text{bind}}(t_1, t_2) = 1 \Leftrightarrow \vec{s}_1^* = \vec{s}_2^*. \quad (3)$$

Before defining the failure density of commitment predicates, we need some additional definitions.

Predicate Systems. To express whether a complete tree of transcripts is valid we collect our predicates to form a *predicate system*. In a predicate system the validity of a tree is defined recursively starting from single accepting transcripts, which we always consider to be valid. Predicates are then enforced bottom-up, first grouping transcripts which fork only in the last challenge to form a new $(1, \dots, 1, k_\mu)$ -subtree. Such a subtree is valid if the predicates for level μ are satisfied. Trees may be grouped according to their common trunk as in Section 4.1 for $\mathbf{K}$ -special-soundness.

We describe validity formally in terms of a boolean function over trees below, and give a visualization of the approach in Figure 1. For a visualization of the predicate system of the toy example, see Figure 2.

Definition 5.4 (Predicate System). *A predicate system Φ for a $(k_1, \dots, k_\mu)$ -tree structure is a collection of predicates for each level in the tree. The m -th level has one commitment predicate $(\Phi_m^{\text{prop}}, \Phi_m^{\text{bind}})$, and k_m challenge predicates $\Phi_{m,1}^{\text{chal}}, \dots, \Phi_{m,k_m}^{\text{chal}}$. We recursively define a series of boolean function Φ_m for $m \in [\mu + 1]$, describing whether a partial tree of transcripts satisfies the predicate system. For a single accepting transcript $t \in \mathbb{T}_{\mu+1}$ we let $\Phi_{\mu+1}(t) = 1$. For all larger subtrees $t = (t_1, \dots, t_{k_m}) \in \mathbb{T}_m$ for some $m \in [\mu]$ with $\mathbf{c}_i = \text{chal}_m(t_i)$, then $\Phi_m(t) = 1$ if and only if*

$$\left(\bigwedge_{i \in [k_m]} \Phi_{m+1}(t_i) = 1 \wedge \Phi_{m,i}^{\text{chal}}(\mathbf{c}_1, \dots, \mathbf{c}_i) = 1 \right) \wedge \Phi_m^{\text{prop}}(t_1, \dots, t_{k_m-1}) = 1 \\ \wedge \Phi_m^{\text{bind}}((t_1, \dots, t_{k_m-1}), t_{k_m}) = 1.$$

For notational convenience, we let $\Phi = \Phi_1$.

Binding relations. To enforce properties on openings, commitment predicates rely on the binding of the commitment scheme. Clearly, if the prover can easily change its opening depending on the challenges it gets, it may with good probability be able to produce accepting transcripts without knowing a valid

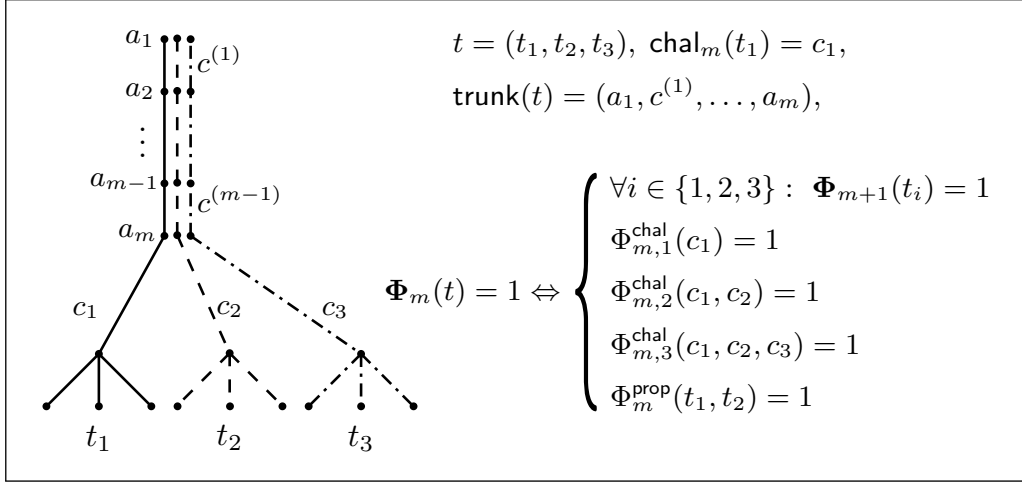


Fig. 1: Constructing $t \in \mathbb{T}_m$ from trees $t_1, t_2, t_3 \in \mathbb{T}_{m+1}$ with a common trunk and $k_m = 3$ distinct level m challenges. Here $(t_1, t_2) \in \mathbb{T}_{m+1}^{(2)}$.

witness. To argue security, we therefore need to reduce to the (computationally) hard problem underlying the binding property. The knowledge extractor should either compute a valid witness for our original relation, or solve the binding problem. Thus, we obtain an extractor for the relation $R_{\text{pp}} \cup R_{\text{pp}}^{\text{bind}}$, where R_{pp} is the original relation and $R_{\text{pp}}^{\text{bind}}$ a relation for the binding problem. Assuming that the binding problem is hard, the protocol must be knowledge sound with respect to the original relation. Formally, we define a binding relation as follows.

Definition 5.5 (Binding Relation). Let $\Pi = (\mathcal{P}, \mathcal{V})$ be a $(\mathbf{K}, \Phi)$ -predicate-special-sound argument of knowledge for a relation R_{pp} , and let $R_{\text{pp}}^{\text{bind}}$ be an additional relation. We say that Φ admits $R_{\text{pp}}^{\text{bind}}$ as a binding relation for public parameters pp if there exists a polynomial time algorithm $\mathcal{B}$, which on input $m \in [\mu]$, a statement x , and trees $(t_1, \dots, t_{k_m}) \in \mathbb{T}_m$ for this statement such that $\Phi_m^{\text{bind}}((t_1, \dots, t_{k_m-1}), t_{k_m}) = 0$ and

$$\forall i \in [k_m] : \Phi_{m+1}(t_i) = 1 \text{ and } \Phi_{m,i}^{\text{chal}}(\text{chal}_m(t_1), \dots, \text{chal}_m(t_i)) = 1,$$

always satisfies $\mathcal{B}(t_1, \dots, t_{k_m}) \in R_{\text{pp}}^{\text{bind}}(x)$.

The central requirement on the input $(t_1, \dots, t_{k_m})$ is that it does not satisfy the binding predicate. However, if the subtrees are not valid, they might not contain openings for the binding predicate. In our toy example, if the challenge difference was not invertible, we could not compute a weak opening from a $(1, 2)$ -tree. Hence, we only require $\mathcal{B}$ to compute a witness for the binding relation when the subtrees are valid and satisfy the challenge predicates.

Let us define a binding relation for our toy example. We may show that for the binding predicate (3), Φ admits the binding relation

$$R_{\text{pp}}^{\text{M-SIS}} = \{ (\beta, \vec{v}) \mid \vec{v} \in \mathcal{R}_q^n, \mathbf{A} \cdot \vec{v} = \vec{0}, 0 < \|\vec{v}\|_2 \leq 8T_{\text{op}}^2\beta \}.$$

We must show that a suitable witness for $R_{\text{pp}}^{\text{M-SIS}}$ may be extracted whenever $\Phi_1^{\text{bind}}(t_1, t_2) = 0$, which happens exactly when $\vec{s}_1^* \neq \vec{s}_2^*$. We claim that $\vec{v} = \bar{c}_1 \bar{c}_2 (\vec{s}_1^* - \vec{s}_2^*)$ is a suitable witness. Both $\vec{s}_1^*$ and $\vec{s}_2^*$ are weak openings to the same commitment. Therefore, $\mathbf{A}(\vec{s}_1^* - \vec{s}_2^*) = \vec{0}$, which in turn implies $\mathbf{A}\vec{v} = \mathbf{A}\bar{c}_1 \bar{c}_2 (\vec{s}_1^* - \vec{s}_2^*) = \vec{0}$. Since $\vec{s}_1^* \neq \vec{s}_2^*$ and $\bar{c}_1, \bar{c}_2$ are units, $\vec{v} \neq \vec{0}$. All that remains is to argue that $\vec{v}$ is short. Using the triangle inequality and the definition of the operator norm, we obtain $\|\vec{v}\| \leq \|\bar{c}_2\|_{\text{op}} \|\bar{c}_1 \vec{s}_1^*\|_2 + \|\bar{c}_1\|_{\text{op}} \|\bar{c}_2 \vec{s}_2^*\|_2$. Recall, we are working with weak openings where $\|\bar{c}_i \vec{s}_i^*\|_2 \leq 2T_{\text{op}}\beta$. As $\bar{c}_i$ is the difference of two challenges in $\mathcal{C}$ we also have $\|\bar{c}_i\|_{\text{op}} \leq 2T_{\text{op}}$. Hence, $\vec{v} \in R_{\text{pp}}^{\text{M-SIS}}(\beta)$.

The failure density of a commitment predicate. We finally have all the pieces needed to define the failure density of a commitment predicate. The failure density captures the intuition of why it should be hard for the prover to succeed without satisfying the property predicate. Namely, it should be hard because of the binding of the commitment scheme and the probabilistic tests in the protocol. Given a fixed opening that does not satisfy the property predicate, it should be hard for the prover to respond to other challenges without breaking binding. We present the formal definition, and then expand on it below.

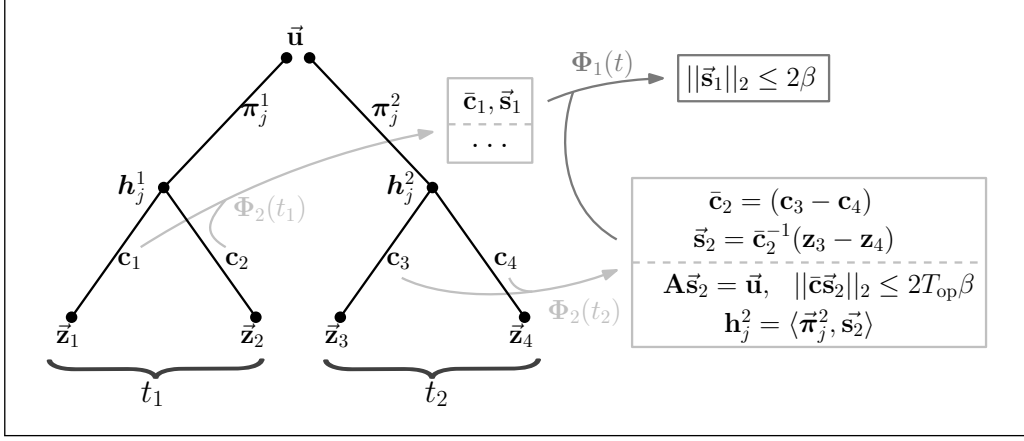


Fig. 2: Our extraction strategy for Protocol 1.

Definition 5.6 (Failure Density of Commitment Predicates). Let $m \in [\mu]$. Define a set of bad subtrees

$$\text{PropUnsat}_m = \left\{ (t_1, \dots, t_{k_m-1}) \in \mathbb{T}_{m+1}^{(k_m-1)} \mid \begin{array}{l} \forall i \in [k_m-1] : \Phi_{m+1}(t_i) = 1, \\ \Phi_{m,i}^{\text{chal}}(c_1, \dots, c_i) = 1, \\ \Phi_m^{\text{prop}}(t_1, \dots, t_{k_m-1}) = 0 \end{array} \right\}$$

using the shorthand $c_i = \text{chal}_m(t_i)$. That is, subtrees in PropUnsat_m fail to satisfy the property predicate but otherwise satisfy the constraints. For each $(t_1, \dots, t_{k_m-1}) \in \mathbb{T}_{m+1}^{(k_m-1)}$, let $\text{BindTree}_m(t_1, \dots, t_{k_m-1})$ be the set of possible k_m -th subtrees that satisfy the binding predicate and the other constraints.

$$\text{BindTree}_m(t_1, \dots, t_{k_m-1}) = \left\{ t \in \mathbb{T}_{m+1} \mid \begin{array}{l} (t_1, \dots, t_{k_m-1}, t) \in \mathbb{T}_m, \Phi_{m+1}(t) = 1, \\ \Phi_{m,k_m}^{\text{chal}}(c_1, \dots, c_{k_m-1}, \text{chal}_m(t)) = 1, \\ \Phi_m^{\text{bind}}((t_1, \dots, t_{k_m-1}), t) = 1 \end{array} \right\}$$

Consider the m -th level challenges occurring for some tree in this set,

$$\text{BindChal}_m(t_1, \dots, t_{k_m-1}) = \{ \text{chal}_m(t) \mid t \in \text{BindTree}_m(t_1, \dots, t_{k_m-1}) \}.$$

The commitment predicate $(\Phi_m^{\text{prop}}, \Phi_m^{\text{bind}})$ has failure density p_m^{com} if $|\text{BindChal}_m(t_1, \dots, t_{k_m-1})| \leq p_m^{\text{com}} |\mathcal{C}_m|$ for all $(t_1, \dots, t_{k_m-1}) \in \text{PropUnsat}_m$.

If the first $k_m - 1$ subtrees $(t_1, \dots, t_{k_m-1})$ are individually valid and they satisfy the challenge predicates for the m th level, they define an opening to the commitment. We consider the case where the opening does not satisfy the property predicate, so that $(t_1, \dots, t_{k_m-1}) \in \text{PropUnsat}_m$. Then, the failure density is a bound on the proportion of m th challenges that the prover can use for the last subtree t_{k_m} , while satisfying the binding predicate. These are the only challenges that the prover can use to make our extraction fail. With these challenges, we neither get a valid witness nor a solution to the binding problem. This is why the failure density is the key measure for the success of our extractor.

We return to the toy example, to find the failure density of the commitment predicate (2, 3). By the Johnson-Lindenstrauss lemma (Lemma 2.2), for a fixed large witness the probability that the projection of the witness is short is only $2^{-\lambda}$. The projection challenges in the lemma are not sampled uniformly. To address this we observe that there exists an algorithm sampling the distribution perfectly given a uniform bit string. To argue that $p_1^{\text{com}} = 2^{-\lambda}$, we use the following remark.

Remark 5.1. To match the uniform sampling required by the extraction framework one might instead consider a challenge set containing all bit strings of some fixed length. These strings may then be given as input to an algorithm sampling the desired distribution. In the case of case projection the distribution for each coefficient C may be sampled perfectly by sampling two uniform bits. Over uniformly sampled challenges, Lemma 2.2 implies that the fraction of bad challenges is at most $2^{-\lambda}$. Note, in general, distinct bit strings may not necessarily give distinct challenges.

5.3 Predicate Special Sound Protocols and their Security

We extend the notion of special-soundness to additionally require a predicate system to be satisfied for successful extraction to be guaranteed. Note, predicate special soundness is equivalent to special-soundness if the predicate system only contains predicates which are trivially satisfied.

Definition 5.7 (Predicate Special Soundness). Let $\Pi = (\mathcal{P}, \mathcal{V})$ be a $2\mu + 1$ -message public-coin argument of knowledge for a relation R_{pp} . We say that Π is $(\mathbf{K}, \Phi)$ -predicate-special-sound for $\mathbf{K} = (k_1, \dots, k_\mu)$ and a predicate system Φ if there exists a polynomial time algorithm which given a statement x and a $\mathbf{K}$ -tree of transcripts t for this statement with $\Phi(t) = 1$ always outputs a witness w such that $w \in R_{\text{pp}}(x)$.

Extending the extractor from [AFK22] for special sound protocols, we may obtain the following result on the knowledge soundness of a predicate special sound protocol.

Theorem 5.1. Let $\Pi = (\mathcal{P}, \mathcal{V})$ be a $(\mathbf{K}, \Phi)$ -predicate-special-sound argument of knowledge for a relation R_{pp} . In addition, let $R_{\text{pp}}^{\text{bind}}$ be a binding relation for Φ with statement x . Then the adaptive Fiat-Shamir transformation $\text{FS}[\Pi]$ is adaptively knowledge sound for the relation $R_{\text{pp}} \cup R_{\text{pp}}^{\text{bind}}$ with knowledge error

$$2(Q + 1) \sum_{i=1}^{\mu} \max \left(\frac{k_i - 1}{|\mathcal{C}_i|}, p_i^{\text{com}} + \sum_{\ell=1}^{k_i} p_{i,\ell}^{\text{chal}} \right),$$

where Q is the number of random oracle queries made by the prover. The number of times that the knowledge extractor invokes the prover is in expectation at most $K + Q(K - 1)$, where $K = \prod_{i=1}^{\mu} k_i$.

We prove this theorem in Section H. The achieved result is actually slightly stronger, in many cases allowing a smaller constant than 2. By applying Theorem 5.1 we may conclude that the Fiat-Shamir transformation of Protocol 1 is knowledge sound for the relation $R_{\text{pp}} \cup R_{\text{pp}}^{\text{M-SIS}}$ with knowledge error

$$2(Q + 1)(\max\{1/|\mathcal{C}_1|, p_1^{\text{com}}\} + \max\{1/|\mathcal{C}_2|, l \cdot B\}),$$

where the extractor invokes the prover at most $4 + 3Q$ times in expectation.

5.4 Extending to Coordinate-Wise PSS

To achieve sublinear proofs, modern lattice-based proof systems often favour a single amortized opening for a series of r witnesses over a separate opening for each witness element. In recent work Fenzi *et al.* [FMN23] showed how these amortization techniques may be reconciled with extraction techniques designed for special-sound protocols. They achieved this by viewing protocols as being special-sound in r coordinates, allowing targeted extraction for each individual witness, resulting in the new notion of coordinate-wise special-soundness. For more on this, see Appendix B.5.

We show how a similar approach may be taken, extending PSS to coordinate-wise PSS. This is the final tool needed to prove non-interactive LaBRADOR secure. For further details, see Appendix G.

5.5 Knowledge Soundness of LaBRADOR under the Fiat-Shamir Transform

We apply our PSS framework to prove knowledge soundness of the Fiat-Shamir transformation of LaBRADOR. For a full analysis we refer the reader to Appendix I. Here $R_{\sigma, \text{pp}}$ is the LaBRADOR relation modified to allow a factor σ norm slack, formally (19).

Theorem 5.2. Let Π be the base LaBRADOR protocol as described in Protocol 2 and 3. We consider the case with a ring $\mathcal{R}_q$ of degree d with splitting factor l and a B -well-spread challenge set, where each challenge has operator norm at most T_{op} . Restrict statements in the relation to have ℓ_2 norm bound β at most q/C_1 , where C_1, C_2 are the parameters of Lemma 2.2. Let $\text{adv}_{\text{M-SIS}, n, \beta}$ be an upper bound on the reduction's advantage in solving the M-SIS problem with module rank n and norm β . Then the Fiat-Shamir transformation $\text{FS}[\Pi]$ is adaptively knowledge sound for the relation $R_{\sigma, \text{pp}}$ where $\sigma = \sqrt{\lambda/C_2}$ with knowledge error

$$2(Q + 1) \left(2^{-\lambda} + (5 + 2l)Br + q^{-d/l} + q^{-\lceil \lambda / \log_2 q \rceil} \right) \quad (4)$$

$$+ \text{adv}_{\text{M-SIS}, \kappa, 8T_{\text{op}}(b+1)\beta'} + \text{adv}_{\text{M-SIS}, \kappa_1, 2\beta'}, \quad (5)$$

where Q is the number of queries made to the random oracle by the prover, and r is the number of witness vectors of rank n in the LaBRADOR statement output by the prover, respectively. The number of times that the knowledge extractor $\mathcal{E}$ invokes the prover is in expectation at most $16r + 8 + Q(16r + 7)$.

Theorem 5.2 follows by application of the extractor from Theorem I.1, and a simple hybrid argument. For a discussion of the knowledge error of LaBRADOR when recursed, see Appendix I.5.

6 Adapting LaBRADOR for Aggregation

6.1 Changing the Modulus

We reformulate our verification constraints to be over a different modulus. From this point forward, we denote $\mathcal{R}_q$ with ring degree $d \in \{512, 1024\}$ and modulus $q = 12289$ as the *Falcon ring*, and $\mathcal{R}_{q'}$ with the same ring degree d but a larger modulus $q' > q$ as the *LaBRADOR ring*. To express the Falcon verification over the LaBRADOR ring, we first use the standard trick of lifting the equation to $\mathcal{R}$, by remembering the modular wrap-around. For each signature, we add an element $\mathbf{v}_i \in \mathcal{R}_{q'}$ to our witness, which should satisfy the equation

$$\mathbf{s}_{i,1} + \mathbf{h}_i \mathbf{s}_{i,2} + q \mathbf{v}_i - \mathbf{t}_i = \mathbf{0} \in \mathcal{R}, \quad (6)$$

without any modular reduction in the coefficients. If an equation holds over $\mathcal{R}$, then clearly the equation also holds modulo any modulus. However, with LaBRADOR, we can only prove that the equation holds modulo q' , and then there are no guarantees that it also holds over $\mathcal{R}$. To prove that the equation holds over $\mathcal{R}$, we use an infinity norm check. Clearly, if the coefficients of our witness vectors are so small that they could not have caused a wrap around modulo q' in the equation, then the equation also holds over $\mathcal{R}$. The specifics of how we perform this norm check are described in the next subsection.

6.2 Norm Checks in the First Iteration

To prove that the aggregated signatures have ℓ_2 -norm at most β , we follow the approach of [GHL22]. They show how to obtain an exact proof of smallness by combining the approximate proof of smallness from [LNS21] with a sum-of-squares proof. The approximate proof of smallness is specified so that it also ensures that there is no wraparound in the Falcon verification equation (6).

A) Four-Square Constraints. Proving that $\|\mathbf{s}_{i,1}\|_2^2 + \|\mathbf{s}_{i,2}\|_2^2 \leq \beta^2$ is equivalent to proving that $\beta^2 - \|\mathbf{s}_{i,1}\|_2^2 - \|\mathbf{s}_{i,2}\|_2^2$ is non-negative. Lagrange's four-square theorem states that any non-negative integer can be written as the sum of four squares. Thus, we can find four integers $\varepsilon_{i,0}, \varepsilon_{i,1}, \varepsilon_{i,2}, \varepsilon_{i,3} \in \mathbb{Z}$ such that

$$\beta^2 - \|\mathbf{s}_{i,1}\|_2^2 - \|\mathbf{s}_{i,2}\|_2^2 = \varepsilon_{i,0}^2 + \varepsilon_{i,1}^2 + \varepsilon_{i,2}^2 + \varepsilon_{i,3}^2. \quad (7)$$

We add the four-square coefficients of the i -th signature to the witness as the coefficients of the polynomial $\varepsilon_i = \varepsilon_{i,0} + \varepsilon_{i,1}X + \varepsilon_{i,2}X^2 + \varepsilon_{i,3}X^3 \in \mathcal{R}_{q'}$.

To formulate the four-square constraints in LaBRADOR, we make use of the relation between the conjugation automorphism σ_{-1} and the ℓ_2 -norm in $\mathcal{R}_{q'}$. Recall that for any vector $\vec{\mathbf{a}} \in \mathcal{R}_{q'}^n$, $\|\vec{\mathbf{a}}\|_2^2 = \text{ct}(\langle \sigma_{-1}(\vec{\mathbf{a}}), \vec{\mathbf{a}} \rangle) \bmod q'$. The idea is to provide $\mathbf{s}'_{i,1} = \sigma_{-1}(\mathbf{s}_{i,1})$, $\mathbf{s}'_{i,2} = \sigma_{-1}(\mathbf{s}_{i,2})$ and $\varepsilon'_i = \sigma_{-1}(\varepsilon_i)$ as part of the witness, so that we can compute the ℓ_2 -norms directly in the dot product constraint. Then the four-square constraints (modulo q') can be expressed in LaBRADOR as

$$\text{ct}(\mathbf{s}'_{i,1} \mathbf{s}_{i,1} + \mathbf{s}'_{i,2} \mathbf{s}_{i,2} + \varepsilon'_i \varepsilon_i - \beta^2) = 0 \bmod q'. \quad (8)$$

To prove that (8) implies (7), we need to prove two things. First, we need to prove that the new elements in the witness are of the correct form (B). Second, we need to prove that the coefficients of the witness elements involved in (8) are so small that they could not have caused a wrap-around modulo q' (C).

B) Constraints for the New Witness Elements. To prove that the new witness elements have been computed correctly, we add new dot product constraints, which check one coefficient at a time. The j -th coefficient of some $\mathbf{a} = \sum_{i=1}^{d-1} a_i X^i \in \mathcal{R}_{q'}$ can be directly verified through a dot product constraint as $\text{ct}(\sigma_{-1}(X^j) \mathbf{a}) = a_j$. Each time we want to check that a_j is equal to some constant $b \in \mathbb{Z}_{q'}$, or we want to check that it is equal to the k -th coefficient of some other witness element $\mathbf{c} \in \mathcal{R}_{q'}$, we add a constraint of the form

$$\text{ct}(\sigma_{-1}(X^j) \mathbf{a} - b) = 0 \bmod q' \quad \text{or} \quad \text{ct}(\sigma_{-1}(X^j) \mathbf{a} - \sigma_{-1}(X^k) \mathbf{c}) = 0 \bmod q'. \quad (9)$$

For each ε_i , we prove that it is a polynomial in $\mathcal{R}_{q'}$ of degree at most 4, by checking that the coefficients $\varepsilon_{i,4}, \dots, \varepsilon_{i,d-1}$ are 0. For the $\mathbf{s}'_{i,1}, \mathbf{s}'_{i,2}, \varepsilon'_i$, we observe that the conjugation automorphism induces a permutation of the coefficients of the input polynomial (up to a sign). For any $\mathbf{a} = a_0 + a_1X + \dots + a_{d-1}X^{d-1} \in \mathcal{R}_{q'}$, it holds $\sigma_{-1}(\mathbf{a}) = a_0 - a_{d-1}X - a_{d-2}X^2 - \dots - a_1X^{d-1}$. Therefore, we just check that each coefficient in $\mathbf{s}'_{i,j}$ matches the corresponding coefficient of $\mathbf{s}_{i,j}$, multiplied by the correct sign. The same goes for the ε'_i . In total, we add roughly $4dN$ new constraints with these checks.

C) Approximate Proof of ℓ_∞ -Smallness. To prove that there is no wrap-around in the constraints (6) and (8) we use approximate proofs of ℓ_∞ -smallness. For this, we use the built in projection step in LaBRADOR for approximate ℓ_2 -smallness. This works because the ℓ_2 -norm of the witness is an upper bound on its ℓ_∞ -norm. In [GHL22] a protocol for approximate proofs of ℓ_∞ -smallness is presented, which can be adapted to our setting, but it concretely ended up giving us slightly larger proofs.

The task at hand is therefore to configure the projection protocol to be our approximate proof of ℓ_∞ -smallness. First, we need to find a ℓ_∞ -norm bound β_∞ for the witness that would guarantee that there could be no wrap-around. Next, we need to find a ℓ_2 -norm upper bound β_2 for the witness of the honest prover. For both security levels $\lambda \in \{128, 256\}$, the projection is a proof that the ℓ_2 -norm of the witness of the prover is at most $\sqrt{128/30}\beta_2$. For completeness we want $\sqrt{128/30}\beta_2 \leq \beta_\infty$, implying that the witness indeed has ℓ_∞ -norm at most β_∞ . Lastly, we need to derive a lower bound for how small the LaBRADOR modulus q' can be while still ensuring that the projection proof is complete and sound. To reduce the size of q' , we actually do two projections, one for the $\mathbf{v}_i$ and one for the rest. Hence, we need to find a β_∞ and β_2 for each projection, and set q' such that both are sound proofs of approximate ℓ_∞ -smallness.

We begin by deriving suitable ℓ_∞ -norm bounds for the witness elements. For the constraints of interest, we want to argue that before reducing modulo q' , the infinity norm of the constraint evaluated on the witness is strictly less than $q'/2$. If the constraint is satisfied modulo q' , it must also be satisfied over the integers.

Let us first consider the i -th four-square constraint (8). It states that the sum of the square of each coefficient of $\mathbf{s}_{i,1}, \mathbf{s}_{i,2}, \boldsymbol{\varepsilon}_i$ should be equal to β^2 modulo q' . If we require that each of the $2d+4$ coefficients has ℓ_∞ -norm strictly less than $\sqrt{q'/(2(2d+4))}$, this sum of squares is strictly less than $q'/2$. Then the sum is also equal to β^2 over the integers. Hence, we need

$$\left\| \sum_{i=1}^N (\mathbf{s}_{i,1} \|\mathbf{s}_{i,2} \|\mathbf{s}'_{i,1} \|\mathbf{s}'_{i,2} \|\boldsymbol{\varepsilon}_i \|\boldsymbol{\varepsilon}'_i) \right\|_\infty < \sqrt{\frac{q'}{2(2d+4)}}.$$

Next we consider the i -th Falcon verification constraint (6). It states that the sum of the three terms $\mathbf{s}_{i,1}, \mathbf{h}_i \mathbf{s}_{i,2}$ and $q \mathbf{v}_i$ should be equal to $\mathbf{t}_i$ modulo q' . By requiring that the ℓ_∞ -norm of each term is strictly less than $q'/6$, their sum has infinity norm strictly less than $q'/2$. By Lemma 2.1 we get that $\|\mathbf{h}_i \mathbf{s}_{i,2}\|_\infty \leq d \|\mathbf{h}_i\|_\infty \|\mathbf{s}_{i,2}\|_\infty \leq dq \|\mathbf{s}_{i,2}\|_\infty$. Hence, we need

$$\|\mathbf{s}_{i,1}\|_\infty < q'/6, \quad \|\mathbf{s}_{i,2}\|_\infty < q'/6dq, \quad \|\mathbf{v}_i\|_\infty < q'/6q.$$

Notice that we have two ℓ_∞ -norm bound requirements for $\mathbf{s}_{i,1}$ and $\mathbf{s}_{i,2}$. However, we see later that q' must be so large that the bound from the four-square constraint is the most restrictive. Thus, we arrive at the final β_∞ bound for each of our projections:

$$\left\| \sum_{i=1}^N (\mathbf{s}_{i,1} \|\mathbf{s}_{i,2} \|\mathbf{s}'_{i,1} \|\mathbf{s}'_{i,2} \|\boldsymbol{\varepsilon}_i \|\boldsymbol{\varepsilon}'_i) \right\|_\infty < \beta_\infty^{(1)} = \sqrt{\frac{q'}{2(2d+4)}},$$

$$\|\mathbf{v}_1\|_\infty \mid \dots \mid \|\mathbf{v}_N\|_\infty < \beta_\infty^{(2)} = q'/6q.$$

The second step in specifying our approximate proof of ℓ_∞ -smallness is to find the β_2 for each projection. For the first projection, we have $\beta_2^{(1)} = 2\beta\sqrt{N}$. Here, the factor 2 comes from the fact that for all $\mathbf{s}_{i,1}, \mathbf{s}_{i,2}$ and $\boldsymbol{\varepsilon}_i$ we also include the respective conjugate in the witness. For the projection of the $\mathbf{v}_i$, we have by Lemma 2.1 that

$$\|\mathbf{v}_i\|_2 = \frac{1}{q} \|\mathbf{t}_i - \mathbf{s}_{i,1} - \mathbf{h}_i \mathbf{s}_{i,2}\|_2 \leq \frac{1}{q} (\sqrt{d}q + \beta + dq\beta).$$

In Falcon $\beta < q$, so that $\|\mathbf{v}_i\|_2 < 1 + \sqrt{d} + d\beta$. Hence, the second projection has $\beta_2^{(2)} = (1 + \sqrt{d} + d\beta)\sqrt{N}$.

Finally, we analyze the requirements on q' for the approximate ℓ_∞ -smallness protocol to be complete and sound. For each projection $k \in \{1, 2\}$ and security level λ , there are two requirements on q' :

1. $\sqrt{\lambda}\beta_2^{(k)} \leq q'/C_1$. This is the condition for using the Johnson-Lindenstrauss Lemma 2.2. When this lemma is applicable, the projection step is a proof that the ℓ_2 -norm is at most $\sqrt{\lambda/C_2}\beta_2^{(k)}$.
2. $\sqrt{\lambda/C_2}\beta_2^{(k)} \leq \beta_\infty^{(k)}$. This means that the projection step is a proof that the witness has infinity norm at most β_∞ .

Concretely comparing the constraints on q' when using Falcon-512 or Falcon-1024, we get that the second condition for the first projection is the most restrictive. With our Johnson-Lindenstrauss parameters, we need

$$q' > (1024/15)(d+2)\beta^2 N. \quad (10)$$

In practice, this bound keeps q' at a reasonable size for implementations. For fast 64-bit lattice implementations, we want q' to be at most a couple of bits less than 64-bits. For Falcon-512, the constraint (10) translates to $q' > 2^{40.12}N$ and for Falcon-1024 to $q' > 2^{42.16}N$. In practice, it seems reasonable to assume that $N \leq 2^{20}$. At $N = 2^{20}$ we need a 61-bit modulus for Falcon-512 and a 63-bit modulus for Falcon-1024, which is at the top end of the allowable range. At the more realistic $N = 2^{10}$, q' only needs to be 51-bits (respectively, 53-bits) long.

6.3 Reformulating the Constraints for a Better Recursion

Until now we have paid little attention to the relationship between the number of witnesses and their rank. This relationship has an impact on prover runtime and how fast LaBRADOR iterations converge towards the base case. In Appendix F.1, we show how to reshape our witness such that we get $r = O(\sqrt{N})$ witness vectors of rank $n = N$, giving us better runtimes and slightly smaller proofs. The tricky part of reshaping the witness is that we must still be able to formulate the four-square constraints. All other constraints are linear in the witness elements, so can be easily reformulated to fit the new witness. However, the four-square constraints are *quadratic* constraints. To formulate them, one needs to compute $\|(\mathbf{s}_{i,1}, \mathbf{s}_{i,2})\|_2^2$ in LaBRADOR as the inner product of witness vectors. We solve this problem with a new padding scheme. See Appendix F.2 for the final constraints.

6.4 Working over Subring

Finally, using the technique from [LNPS21, Sec. 2.8], we reformulate our constraints to be over a subring. In particular, we move from the ring $\mathcal{R}$ of degree d to a subring $\mathcal{S}$ of smaller degree $d' = d/c$, which improves proof sizes. To this end, we make use of the bijection $\phi: \mathcal{R}_q^n \rightarrow \mathcal{S}_q^{c,n}$ defined in Section 2.2. It suffices to show that the bijection is compatible with the LaBRADOR constraint system. Let $\mathcal{F}$ denote the full dot product constraints and $\mathcal{F}'$ the constant term constraints resulting from the discussion above on aggregating Falcon signatures, as summarized in Appendix F.2. As it is a bijection, we know that only the zero element is mapped to zero. More importantly, the map is norm-preserving. Then, any tuple $(\mathcal{F}, \mathcal{F}', \beta)$ over $\mathcal{R}_q$ defines a tuple $(\phi(\mathcal{F}), \phi(\mathcal{F}'), \beta)$ over $\mathcal{S}_q^c$ such that

$$\begin{aligned} \phi(f)(\phi(\vec{w}_1), \dots, \phi(\vec{w}_r)) &= \mathbf{0} \in \mathcal{S}_q^c & \forall \phi(f) \in \phi(\mathcal{F}), \\ \text{ct}(\phi(f'))(\phi(\vec{w}_1), \dots, \phi(\vec{w}_r)) &= \mathbf{0} \in \mathbb{Z}_q^c & \forall \phi(f') \in \phi(\mathcal{F}'), \\ \sum_{i=1}^r \|\phi(\vec{w}_i)\|_2^2 &\leq \beta^2. \end{aligned}$$

This last technique reduces the proof sizes by at least a multiplicative factor 2.

7 Estimates and Comparison

The code to compute the numbers and plots in this section can be found in our online repository at <https://github.com/dfaranha/aggregate-falcon>.

7.1 Estimates and Comparison of Proof Sizes

We tailored all choices of parameters to allow for maximal $N = 10\,000$ signatures to be aggregated. We highlight that if setting up LaBRADOR for aggregating significantly smaller or larger number of Falcon signatures, we recommend re-running our scripts to derive different parameter sets. Setting everything up for a much smaller N would lead to significantly smaller AS. Overall, our aggregate signature scheme for Falcon-512 signatures guarantees a security level of 121 bits, whereas the aggregate signature scheme for Falcon-1024 signatures guarantees 249 bits of security. We give a detailed explanation how this is derived in Appendix E. There, we also provide figures comparing our aggregate signature with the trivial concatenation, depicting the effect of the salts and the effect of moving from two-splitting to almost-fully-splitting.

Aggr. Signature Scheme	# Sign.	N	Sec. Param. λ	$ \sigma_{\text{agg}} $
[JRS23, Tab.C.1]	500		128	3616 kB
Ours for Falcon-512 with salts	500		121	93 kB
Ours for Falcon-512 without salts	500		121	73 kB
[JRS23, Tab. C.1]	1000		128	7444 kB
Ours for Falcon-512 with salts	1000		121	120 kB
Ours for Falcon-512 without salts	1000		121	80 kB
[JRS23, Tab. C.1]	2000		128	15319 kB
Ours for Falcon-512 with salts	2000		121	165 kB
Ours for Falcon-512 without salts	2000		121	85 kB

Table 1: Comparison of our Falcon-512 aggregate signature sizes (with and without salts) with aggregate signature [JRS23] for different parameters.

7.1.1 Comparison with Phoenix [JRS23] To the best of our knowledge, there is only one lattice-based non-interactive aggregate signature scheme, proven in the same non-interactive security model as ours, providing non-trivial compression as well as concrete numbers [JRS23]. We emphasize that their aggregation only works for GPV-style signatures that use the MP-trapdoors [MP12] with LW-sampler [LW15]. Hence, their construction does not work for Falcon. They provide benchmarks [JRS23, Table 4] for the number of signatures N going up to 500, 1000 and 2000 and target a security level of 128, whereas we target 121 bits of security in our aggregation of Falcon-512. In Table 1, one can see that our aggregate signatures are of multiple orders of magnitude smaller than the ones from [JRS23]. Moreover, their aggregation does not lead to smaller aggregate signature sizes for large numbers of signatures, in particular beyond $N \geq 4000$. Note that their starting signature is also significantly larger than a Falcon-512 signature, to begin with. We highlight once more that our goal was to use the native language of LaBRADOR to aggregate original Falcon signatures, which leads to the AS with salts. For illustration purposes, we sometimes also provide the sizes of our aggregate signature if no salts would be included, e.g., for deterministic Falcon or for aggregation in a synchronized model.

7.1.2 Comparison with Squirrel [FSZ22] and Chipmunk [FHSZ23] Squirrel [FSZ22] and its optimization Chipmunk [FHSZ23] are multi-signature schemes in the synchronized setting whose security also relies on M-SIS. Both can aggregate signatures for possibly distinct parties and messages but which were issued for the *same time period*. Due to their tree-based construction, their multi signature schemes can only be used for a certain amount of time (which [FSZ22] calls the *life cycle* of their scheme). Both are significant restrictions which our aggregate signature does not incur. Squirrel provides benchmarks [FSZ22, Table 3] for the number of signatures N being 1024, 4096 and 8192 and life cycles of 8 months, 5 years and 12 years. Chipmunk provides benchmarks [FHSZ23, Table 5] for the number of signatures N being 1024 and 8192 and the same life cycles. Both target a security level of 112, whereas we target a security level of 121 for Falcon-512. Table 2 shows that our aggregate scheme (with salts) produces smaller aggregate signatures than Squirrel but larger ones than Chipmunk for every set of parameters. We stress that we can avoid sending the salts by moving to the same synchronized model as Squirrel and Chipmunk. To do so, one simply replaces the fresh salt by a common time period as input to the random oracle. When comparing our aggregate signature without salts with Chipmunk, we see that the estimated sizes of our scheme are always smaller than the one of Chipmunk. Moreover, we note that single Chipmunk signatures are significantly larger than a Falcon-512 signature to begin with.

7.2 Estimates of Running Times for Polynomial Arithmetic

Section 3 allows for LaBRADOR challenge spaces that either rely on an almost-fully-splitting ring $\mathcal{R}_{q'} = \prod_{i=1}^l \mathbb{Z}_{q'}[X]/\langle X^\delta - \zeta_i \rangle$ with small δ , or on the simpler case of a two-splitting ring. Since LaBRADOR extensively uses polynomial multiplications both in proof generation and verification, the choice of ring and challenge space determines the overall performance. A major factor is supporting efficient polynomial multiplication, for example through the NTT [LS19]. There are two approaches for the two-splitting case: the naive using a CRT representation such that $\delta = d'/2$ [LN17], or lifting to a larger composite-modulus ring such that the NTT can be realized by combining the results of each smaller NTT modulo each factor [CHK⁺21]. Using the bound from Section 6.2, a double-word modulus is sufficient for the lifting with $N = 10000$.

We estimated the performance with a proof-of-concept implementation of the NTT evaluations and polynomial multiplication for various choices of $\mathcal{R}_{q'}$. The implementation for the naive two-splitting case

Aggr. Signature Scheme	Model	# Sign.	N	Sec. Param.	λ	Life Cycle	$ \sigma_{\text{agg}} $
Squirrel [FSZ22]	sync	1024	112			8 months	572 kB
	sync	1024	112			5 years	635 kB
	sync	1024	112			21 years	677 kB
Chipmunk [FHSZ23]	sync	1024	112			8 months	118 kB
	sync	1024	112			5 years	133 kB
	sync	1024	112			21 years	143 kB
Ours for Falcon-512 with salts	std	1024	121			∞	122 kB
Ours for Falcon-512 without salts	sync	1024	121			∞	81 kB
Squirrel [FSZ22]	sync	4096	112			8 months	693 kB
	sync	4096	112			5 years	711 kB
	sync	4096	112			21 years	823 kB
Chipmunk [FHSZ23]	sync	/	/			/	/
Ours for Falcon-512 with salts	std	4096	121			∞	252 kB
Ours for Falcon-512 without salts	sync	4096	121			∞	88 kB
Squirrel [FSZ22]	sync	8192	112			8 months	762 kB
	sync	8192	112			5 years	820 kB
	sync	8192	112			21 years	908 kB
Chipmunk [FHSZ23]	sync	8192	112			8 months	160 kB
	sync	8192	112			5 years	180 kB
	sync	8192	112			21 years	194 kB
Ours for Falcon-512 with salts	std	8192	121			∞	417 kB
Ours for Falcon-512 without salts	sync	8192	121			∞	89 kB

Table 2: Comparison of our Falcon-512 aggregate signature sizes with synchronized multi-signature Squirrel [FSZ22] and its optimization Chipmunk [FHSZ23] for different parameters. By ‘sync’ we denote the synchronized aggregation model, whereas ‘std’ refers to the more general standard model. The ∞ symbol indicates that there is no a-priori bound on the life cycle of our aggregate signature scheme. The sizes of our scheme without salts correspond to a scheme with the same security model as Squirrel/Chipmunk.

was performed using the FLINT v2.9 library, and code adapted from [FWK23] for the other NTT-friendly parameters. Performance figures were collected using GCC 13.2.1 in a 64-bit Intel Core i7-7700 Kaby Lake CPU running at 3.60GHz, with TurboBoost turned off to reduce randomness in the runtime. We first experimentally observed that a field multiplication modulo a double-word q' using Montgomery modular multiplication was approximately 7 times the latency of a multiplication modulo a single-word q' using the signed Montgomery variant. With this ratio at hand, we were able to scale the latency from single-word to double-word, since multiplication in $\mathbb{Z}_{q'}$ is the performance-critical operation for both the NTT transforms and polynomial multiplication in the NTT domain.

Table 3 presents the clock cycles for polynomial arithmetic with different δ and subring degree d' . Observe that the naive two-splitting is rather inefficient, and the lifting two-splitting has the fastest polynomial multiplication. For Falcon-512, we considered the additional cases $\delta = 4$ and fully-splitting with 128-bit q' . Both ensure that $(q')^\delta$ would be at least 2^{128} . For Falcon-1024, we considered the additional cases $\delta = 4$, and $\delta = 8$ with 128-bit q' . From these empirical results, we conclude that the two-splitting lifting approach is most suitable both with respect to execution time and proof size.

Acknowledgment

We are grateful to Gregor Seiler for answering our technical questions about LaBRADOR and to Ngoc Khanh Nguyen for helpful feedback on earlier versions of this work. We further thank our anonymous reviewers, in particular for bringing the lifting approach from [CHK⁺21] to our attention. The work was supported by the Danish Independent Research Council under projects 0165-00107B (C3PO), 1026-00350B (RENAIS) and 2064-00016B (YOSO), as well as by the Protocol Labs Research Grant Program RFP-013 and PL-RGP1-2021-064, and the European Research Council (ERC) under the European Unions’s Horizon 2020 research and innovation programme under grant agreement No 803096 (SPEC).

This paper was prepared in part for information purposes by the Artificial Intelligence Research group of JPMorgan Chase & Co and its affiliates (“JP Morgan”), and is not a product of the Research Department of JP Morgan. JP Morgan makes no representation and warranty whatsoever and disclaims all liability, for the completeness, accuracy or reliability of the information contained herein. This document

Splitting regime / Operation	Falcon-512		Falcon-1024	
	Mult.	(inv) NTT	Mult.	(inv) NTT
Two-naive ($d' = d/8$)	24,700	–	67,435	–
Two-lifting ($d' = d/8$)	<i>1,090</i>	<i>9,890</i>	<i>1,850</i>	<i>20,694</i>
Almost-fully ($\delta = 8, d' = d/4$)	–	–	29,412	13,148
Almost-fully ($\delta = 4, d' = d/4$)	6,284	8,089	91,343	114,408
Fully ($d' = 64$)	3,815	37,443	–	–

Table 3: Timings in clock cycles for polynomial arithmetic (multiplication and NTT when applicable) for efficient choices of subring with degree d' . Numbers in italic correspond to the best trade-off between execution time and proof size.

is not intended as investment research or investment advice, or a recommendation, offer or solicitation for the purchase or sale of any security, financial instrument, financial product or service, or to be used in any way for evaluating the merits of participating in any transaction, and shall not constitute a solicitation under any jurisdiction or to any person, if such solicitation under such jurisdiction or to such person would be unlawful.

References

- ACK21. T. Attema, R. Cramer, and L. Kohl. A compressed Σ -protocol theory for lattices. In *CRYPTO 2021, Part II*, vol. 12826 of *LNCS*, pp. 549–579, Virtual Event, 2021. Springer, Cham.
- ACL⁺22. M. R. Albrecht, V. Cini, R. W. F. Lai, G. Malavolta, and S. A. K. Thyagarajan. Lattice-based SNARKs: Publicly verifiable, preprocessing, and recursively composable - (extended abstract). In *CRYPTO 2022, Part II*, vol. 13508 of *LNCS*, pp. 102–132. Springer, Cham, 2022.
- AFK22. T. Attema, S. Fehr, and M. Klooß. Fiat-shamir transformation of multi-round interactive proofs. In *TCC 2022, Part I*, vol. 13747 of *LNCS*, pp. 113–142. Springer, Cham, 2022.
- AGH10. J. H. Ahn, M. Green, and S. Hohenberger. Synchronized aggregate signatures: new definitions, constructions and applications. In *ACM CCS 2010*, pp. 473–484. ACM Press, 2010.
- Ajt96. M. Ajtai. Generating hard instances of lattice problems (extended abstract). In *28th ACM STOC*, pp. 99–108. ACM Press, 1996.
- AL21. M. R. Albrecht and R. W. F. Lai. Subtractive sets over cyclotomic rings - limits of Schnorr-like arguments over lattices. In *CRYPTO 2021, Part II*, vol. 12826 of *LNCS*, pp. 519–548, Virtual Event, 2021. Springer, Cham.
- ALS20. T. Attema, V. Lyubashevsky, and G. Seiler. Practical product proofs for lattice commitments. In *CRYPTO 2020, Part II*, vol. 12171 of *LNCS*, pp. 470–499. Springer, Cham, 2020.
- AMAB⁺23. C. Aguilar-Melchor, M. R. Albrecht, T. Bailleux, N. Bindel, J. Howe, A. Hülsing, D. Joseph, and M. Manzano. Batch signatures, revisited. *Cryptology ePrint Archive, Report 2023/492*, 2023. <https://eprint.iacr.org/2023/492>.
- BBB⁺18. B. Bünz, J. Bootle, D. Boneh, A. Poelstra, P. Wuille, and G. Maxwell. Bulletproofs: Short proofs for confidential transactions and more. In *2018 IEEE Symposium on Security and Privacy*, pp. 315–334. IEEE Computer Society Press, 2018.
- BCC⁺16. J. Bootle, A. Cerulli, P. Chaidos, J. Groth, and C. Petit. Efficient zero-knowledge arguments for arithmetic circuits in the discrete log setting. In *EUROCRYPT 2016, Part II*, vol. 9666 of *LNCS*, pp. 327–357. Springer, Berlin, Heidelberg, 2016.
- BCMS20. B. Bünz, A. Chiesa, P. Mishra, and N. Spooner. Recursive proof composition from accumulation schemes. In *TCC 2020, Part II*, vol. 12551 of *LNCS*, pp. 1–18. Springer, Cham, 2020.
- BCPS18. A. Bishnoi, P. L. Clark, A. Potukuchi, and J. R. Schmitt. On zeros of a polynomial in a finite grid. *Comb. Probab. Comput.*, 27(3):310–333, 2018.
- BDF⁺11. D. Boneh, Ö. Dagdelen, M. Fischlin, A. Lehmann, C. Schaffner, and M. Zhandry. Random oracles in a quantum world. In *ASIACRYPT 2011*, vol. 7073 of *LNCS*, pp. 41–69. Springer, Berlin, Heidelberg, 2011.
- BDN18. D. Boneh, M. Drijvers, and G. Neven. Compact multi-signatures for smaller blockchains. In *ASIACRYPT 2018, Part II*, vol. 11273 of *LNCS*, pp. 435–464. Springer, Cham, 2018.
- BF23. B. Bünz and B. Fisch. Multilinear schwartz-zippel mod N and lattice-based succinct arguments. In *TCC 2023, Part III*, vol. 14371 of *LNCS*, pp. 394–423. Springer, Cham, 2023.
- BGLS03. D. Boneh, C. Gentry, B. Lynn, and H. Shacham. Aggregate and verifiably encrypted signatures from bilinear maps. In *EUROCRYPT 2003*, vol. 2656 of *LNCS*, pp. 416–432. Springer, Berlin, Heidelberg, 2003.

- BK20. D. Boneh and S. Kim. One-time and interactive aggregate signatures from lattices. *preprint*, 2020.
- BN06. M. Bellare and G. Neven. Multi-signatures in the plain public-key model and a general forking lemma. In *ACM CCS 2006*, pp. 390–399. ACM Press, 2006.
- BNN07. M. Bellare, C. Namprempe, and G. Neven. Unrestricted aggregate signatures. In *ICALP 2007*, vol. 4596 of *LNCS*, pp. 411–422. Springer, Berlin, Heidelberg, 2007.
- BR21. K. Boudgoust and A. Roux-Langlois. Compressed linear aggregate signatures based on module lattices. Cryptology ePrint Archive, Report 2021/263, 2021.
- BRL23. K. Boudgoust and A. Roux-Langlois. Overfull: Too Large Aggregate Signatures Based on Lattices. *The Computer Journal*, p. bxad013, 2023.
- BS23. W. Beullens and G. Seiler. LaBRADOR: Compact proofs for R1CS from module-SIS. In *CRYPTO 2023, Part V*, vol. 14085 of *LNCS*, pp. 518–548. Springer, Cham, 2023.
- BT23. K. Boudgoust and A. Takahashi. Sequential half-aggregation of lattice-based signatures. In *ESORICS 2023, Part I*, vol. 14344 of *LNCS*, pp. 270–289. Springer, Cham, 2023.
- BTT22. C. Boschini, A. Takahashi, and M. Tibouchi. MuSig-L: Lattice-based multi-signature with single-round online phase. In *CRYPTO 2022, Part II*, vol. 13508 of *LNCS*, pp. 276–305. Springer, Cham, 2022.
- Che23. Y. Chen. DualMS: Efficient lattice-based two-round multi-signature with trapdoor-free simulation. In *CRYPTO 2023, Part V*, vol. 14085 of *LNCS*, pp. 716–747. Springer, Cham, 2023.
- CHK⁺21. C.-M. M. Chung, V. Hwang, M. J. Kannwischer, G. Seiler, C.-J. Shih, and B.-Y. Yang. NTT multiplication for NTT-unfriendly rings. *IACR TCHES*, 2021(2):159–188, 2021.
- CJJ22. A. R. Choudhuri, A. Jain, and Z. Jin. SNARGs for $\mathcal{P}$ from LWE. In *62nd FOCS*, pp. 68–79. IEEE Computer Society Press, 2022.
- CLMQ21. Y. Chen, A. Lombardi, F. Ma, and W. Quach. Does fiat-shamir require a cryptographic hash function? In *CRYPTO 2021, Part IV*, vol. 12828 of *LNCS*, pp. 334–363, Virtual Event, 2021. Springer, Cham.
- Cor02. J.-S. Coron. Optimal security proofs for PSS and other signature schemes. In *EUROCRYPT 2002*, vol. 2332 of *LNCS*, pp. 272–287. Springer, Berlin, Heidelberg, 2002.
- DGKV22. L. Devadas, R. Goyal, Y. Kalai, and V. Vaikuntanathan. Rate-1 non-interactive arguments for batch-NP and applications. In *63rd FOCS*, pp. 1057–1068. IEEE Computer Society Press, 2022.
- DHSS20. Y. Doröz, J. Hoffstein, J. H. Silverman, and B. Sunar. MMSAT: A scheme for multimesage multiuser signature aggregation. Cryptology ePrint Archive, Report 2020/520, 2020.
- DOTT22. I. Damgård, C. Orlandi, A. Takahashi, and M. Tibouchi. Two-round n-out-of-n and multi-signatures and trapdoor commitment from lattices. *Journal of Cryptology*, 35(2):14, 2022.
- EB14. R. El Bansarkhani and J. Buchmann. Towards lattice based aggregate signatures. In *AFRICACRYPT 14*, vol. 8469 of *LNCS*, pp. 336–355. Springer, Cham, 2014.
- ESLR23. M. F. Esgin, R. Steinfeld, D. Liu, and S. Ruj. Efficient hybrid exact/relaxed lattice proofs and applications to rounding and VRFs. In *CRYPTO 2023, Part V*, vol. 14085 of *LNCS*, pp. 484–517. Springer, Cham, 2023.
- ESZ22. M. F. Esgin, R. Steinfeld, and R. K. Zhao. MatRiCT⁺: More efficient post-quantum private blockchain payments. In *2022 IEEE Symposium on Security and Privacy*, pp. 1281–1298. IEEE Computer Society Press, 2022.
- FHSZ23. N. Fleischhacker, G. Herold, M. Simkin, and Z. Zhang. Chipmunk: Better synchronized multi-signatures from lattices. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, pp. 386–400, 2023.
- FMN23. G. Fenzi, H. Moghaddas, and N. K. Nguyen. Lattice-based polynomial commitments: Towards asymptotic and concrete efficiency. Cryptology ePrint Archive, Paper 2023/846, 2023. <https://eprint.iacr.org/2023/846>.
- FN16. D. Fiore and A. Nitulescu. On the (in)security of SNARKs in the presence of oracles. In *TCC 2016-B, Part I*, vol. 9985 of *LNCS*, pp. 108–138. Springer, Berlin, Heidelberg, 2016.
- FS87. A. Fiat and A. Shamir. How to prove yourself: Practical solutions to identification and signature problems. In *CRYPTO’86*, vol. 263 of *LNCS*, pp. 186–194. Springer, Berlin, Heidelberg, 1987.
- FSZ22. N. Fleischhacker, M. Simkin, and Z. Zhang. Squirrel: Efficient synchronized multi-signatures from lattices. In *ACM CCS 2022*, pp. 1109–1123. ACM Press, 2022.
- FWK23. V. Farzaliyev, J. Willemsen, and J. K. Kaasik. Improved lattice-based mix-nets for electronic voting. *IET Inf. Secur.*, 17(1):18–34, 2023.
- GHL22. C. Gentry, S. Halevi, and V. Lyubashevsky. Practical non-interactive publicly verifiable secret sharing with thousands of parties. In *EUROCRYPT 2022, Part I*, vol. 13275 of *LNCS*, pp. 458–487. Springer, Cham, 2022.
- GPV08. C. Gentry, C. Peikert, and V. Vaikuntanathan. Trapdoors for hard lattices and new cryptographic constructions. In *40th ACM STOC*, pp. 197–206. ACM Press, 2008.
- GR06. C. Gentry and Z. Ramzan. Identity-based aggregate signatures. In *PKC 2006*, vol. 3958 of *LNCS*, pp. 257–273. Springer, Berlin, Heidelberg, 2006.
- HKW15. S. Hohenberger, V. Koppula, and B. Waters. Universal signature aggregators. In *EUROCRYPT 2015, Part II*, vol. 9057 of *LNCS*, pp. 3–34. Springer, Berlin, Heidelberg, 2015.
- HPS98. J. Hoffstein, J. Pipher, and J. H. Silverman. NTRU: A ring-based public key cryptosystem. In *Third Algorithmic Number Theory Symposium (ANTS)*, vol. 1423 of *LNCS*, pp. 267–288. Springer, 1998.

- JRLS23. C. Jeudy, A. Roux-Langlois, and O. Sanders. Revisiting preimage sampling for lattices. Cryptology ePrint Archive, Paper 2023/446, 2023. Version 23.05.2023 <https://eprint.iacr.org/archive/2023/446/20230516:150953>.
- JRS23. C. Jeudy, A. Roux-Langlois, and O. Sanders. Phoenix: Hash-and-sign with aborts from lattice gadgets. Cryptology ePrint Archive, Report 2023/446, 2023. <https://eprint.iacr.org/2023/446>.
- LMRS04. A. Lysyanskaya, S. Micali, L. Reyzin, and H. Shacham. Sequential aggregate signatures from trapdoor permutations. In *EUROCRYPT 2004*, vol. 3027 of *LNCS*, pp. 74–90. Springer, Berlin, Heidelberg, 2004.
- LN17. V. Lyubashevsky and G. Neven. One-shot verifiable encryption from lattices. In *EUROCRYPT 2017, Part I*, vol. 10210 of *LNCS*, pp. 293–323. Springer, Cham, 2017.
- LNP22. V. Lyubashevsky, N. K. Nguyen, and M. Plançon. Lattice-based zero-knowledge proofs and applications: Shorter, simpler, and more general. In *CRYPTO 2022, Part II*, vol. 13508 of *LNCS*, pp. 71–101. Springer, Cham, 2022.
- LNPS21. V. Lyubashevsky, N. K. Nguyen, M. Plançon, and G. Seiler. Shorter lattice-based group signatures via “almost free” encryption and other optimizations. In *ASIACRYPT 2021, Part IV*, vol. 13093 of *LNCS*, pp. 218–248. Springer, Cham, 2021.
- LNS21. V. Lyubashevsky, N. K. Nguyen, and G. Seiler. Shorter lattice-based zero-knowledge proofs via one-time commitments. In *PKC 2021, Part I*, vol. 12710 of *LNCS*, pp. 215–241. Springer, Cham, 2021.
- LS15. A. Langlois and D. Stehlé. Worst-case to average-case reductions for module lattices. *Designs, Codes and Cryptography*, 75:565–599, 2015.
- LS18. V. Lyubashevsky and G. Seiler. Short, invertible elements in partially splitting cyclotomic rings and applications to lattice-based zero-knowledge proofs. In *EUROCRYPT 2018, Part I*, vol. 10820 of *LNCS*, pp. 204–224. Springer, Cham, 2018.
- LS19. V. Lyubashevsky and G. Seiler. NTTTRU: Truly fast NTRU using NTT. *IACR TCHES*, 2019(3):180–201, 2019.
- LW15. V. Lyubashevsky and D. Wichs. Simple lattice trapdoor sampling from a broad class of distributions. In *PKC 2015*, vol. 9020 of *LNCS*, pp. 716–730. Springer, Berlin, Heidelberg, 2015.
- Lyu09. V. Lyubashevsky. Fiat-Shamir with aborts: Applications to lattice and factoring-based signatures. In *ASIACRYPT 2009*, vol. 5912 of *LNCS*, pp. 598–616. Springer, Berlin, Heidelberg, 2009.
- Lyu12. V. Lyubashevsky. Lattice signatures without trapdoors. In *EUROCRYPT 2012*, vol. 7237 of *LNCS*, pp. 738–755. Springer, Berlin, Heidelberg, 2012.
- MP12. D. Micciancio and C. Peikert. Trapdoors for lattices: Simpler, tighter, faster, smaller. In *EUROCRYPT 2012*, vol. 7237 of *LNCS*, pp. 700–718. Springer, Berlin, Heidelberg, 2012.
- MR09. D. Micciancio and O. Regev. Lattice-based cryptography. *Post-quantum cryptography*, pp. 147–191, 2009.
- Ngu22. N. K. Nguyen. *Lattice-Based Zero-Knowledge Proofs Under a Few Dozen Kilobytes*. PhD thesis, ETH Zurich, 2022. <https://www.research-collection.ethz.ch/handle/20.500.11850/574844>.
- NS22. N. K. Nguyen and G. Seiler. Practical sublinear proofs for R1CS from lattices. In *CRYPTO 2022, Part II*, vol. 13508 of *LNCS*, pp. 133–162. Springer, Cham, 2022.
- PFH⁺22. T. Prest, P.-A. Fouque, J. Hoffstein, P. Kirchner, V. Lyubashevsky, T. Pornin, T. Ricosset, G. Seiler, W. Whyte, and Z. Zhang. FALCON. Technical report, National Institute of Standards and Technology, 2022. available at <https://csrc.nist.gov/Projects/post-quantum-cryptography/selected-algorithms-2022>.
- TS23a. T. Tomita and J. Shikata. Compact aggregate signature from module-lattices. Cryptology ePrint Archive, Report 2023/471, 2023. Version 2023-11-26 <https://eprint.iacr.org/archive/2023/471/20231126:134757>.
- TS23b. T. Tomita and J. Shikata. Compact signature aggregation from module-lattices. Cryptology ePrint Archive, Report 2023/471, 2023. Version 2023-05-08 <https://eprint.iacr.org/archive/2023/471/20230508:220915>.
- Val08. P. Valiant. Incrementally verifiable computation or proofs of knowledge imply time/space efficiency. In *TCC 2008*, vol. 4948 of *LNCS*, pp. 1–18. Springer, Berlin, Heidelberg, 2008.
- Wik21. D. Wikström. Special soundness in the random oracle model. Cryptology ePrint Archive, Report 2021/1265, 2021.
- WW19. Z. Wang and Q. Wu. A practical lattice-based sequential aggregate signature. In *ProvSec 2019*, vol. 11821 of *LNCS*, pp. 94–109. Springer, Cham, 2019.
- WW22. B. Waters and D. J. Wu. Batch arguments for NP and more from standard bilinear group assumptions. In *CRYPTO 2022, Part II*, vol. 13508 of *LNCS*, pp. 433–463. Springer, Cham, 2022.
- WW23. H. Wee and D. J. Wu. Lattice-based functional commitments: Fast verification and cryptanalysis. In *ASIACRYPT 2023, Part V*, vol. 14442 of *LNCS*, pp. 201–235. Springer, Singapore, 2023.

Appendix

A Related Work

A.1 Aggregate Signatures from Lattices

Aggregate Signatures and BARG/SNARK-based Solutions. Boneh et al. [BGLS03] introduced the notion of aggregate signatures for the first time, followed by a number of efficient constructions based on pre-quantum assumptions e.g. [GR06, BNN07]. Several generic constructions of AS based on non-interactive arguments exist in the literature, although none of them evaluate concrete efficiency if instantiated with the NIST finalists. Hohenberger, Koppula, and Waters [HKW15] construct AS from iO and the RSA assumption. Waters and Wu [WW22] propose BARGs for NP from standard pairing-based assumptions and use BARGs to construct compact AS in the standard model. To prove the security of AS, they show *somewhere* knowledge soundness of BARG is sufficient, meaning that an extractor only needs to obtain a single witness for one of the statements $x_1, \dots, x_N$ chosen by the adversary. Their generic AS construction could be instantiated with LWE-based BARG of [CJJ22]. Devadas et al. [DGKV22] generalize BARG to *multi-hop* BARG, which allows aggregating multiple batch proofs for NP statements. Their construction of multi-hop BARGs from LWE can be applied to instantiate multi-hop AS in the standard model, where an aggregator can combine possibly aggregated signatures. Outside the standard model, Tomita and Shikata [TS23b] observe that LaBRADOR can be used in a straightforward manner to instantiate lattice-based AS in the random oracle model, by translating the verification condition of the base signature scheme to R1CS. In an updated version [TS23a], they bypass the R1CS translation, while instantiating LaBRADOR for a specific signature scheme derived from [CLMQ21].

Our work takes a different angle from these generic feasibility results in that (1) we strive to optimize the LaBRADOR relation and challenge space in order to natively support the verification equations of the standardized Falcon scheme, instead of naively converting them into a boolean circuit or R1CS, (2) we provide concrete size estimates of our AS construction, (3) we present a general framework for analyzing Fiat-Shamir AoK from multi-round protocols including LaBRADOR, and (4) we explicitly prove a signing oracle for hash-then-sign signatures does not interfere with witness extraction to formally conclude security of SNARK-based AS (in the random oracle model).

Aggregate Signatures tailored to GPV. To the best of our knowledge, only a few GPV-based AS exist in the literature. Jeudy, Roux-Langlois, and Sanders [JRLS23] present AS constructed from the Micciancio-Peikert trapdoor [MP12] and Lyubashevsky-Wichs Gaussian sampler [LW15]. Since their aggregation strategy highly exploits particulars of [MP12]-based GPV, it is currently unclear how a similar approach extends to an NTRU-based instantiation of GPV including Falcon. By allowing signers interact with each other in a round-robin fashion, one can obtain a *sequential* aggregate signature (SAS) [LMRS04]. As mentioned earlier, two existing SAS based on GPV [WW19, EB14] only offer very limited compression rates.

Aggregate Signatures tailored to Fiat-Shamir with Aborts. Within the Fiat-Shamir with Aborts paradigm [Lyu09, Lyu12], MMSAT [DHSS20] was proposed as a *half*-aggregate signature, compressing only half of the two signature components. Boudgoust and Roux-Langlois [BRL23, BR21] point out a flaw in MMSAT and present a secure variant of MMSAT assuming Module-SIS, Module-LWE and ROM, but they also observe that the size of aggregate signature is larger than the trivial concatenation. Very recently, Boudgoust and Takahashi [BT23] presented a FS-based SAS that outputs an aggregate signature smaller than the naive concatenation. Similar to GPV-based SAS, their concrete compression rate is still quite limited ($\approx 99\%$) if instantiated with Dilithium parameter sets.

Allowing interaction between signers, one can construct Fiat-Shamir-based AS by applying a generic conversion method to an interactive multi-signature (where all the signers sign the same message) [BN06, BK20]. Thus, a line of work on lattice-based multi-signatures e.g. [DOTT22, BTT22, Che23] could also be turned into interactive AS. This generic method does not fit in the typical use cases of AS such as a certificate chain, because it has to ask the signers to synchronize with each other in advance to agree on all N messages to be signed.

Synchronized Aggregate Signatures. Squirrel [FSZ22] and its recent optimization Chipmunk [FHSZ23] can be viewed as lattice-based AS but follow an entirely different paradigm than GPV or FS. Although they provide concretely efficient constructions, Squirrel and Chipmunk come with limitations: (1) the constructions are only proven secure in a “synchronized” model where signers are only allowed to produce

a single signature in each time step, and (2) they only allow a signer to produce a bounded number of signatures for a fixed public key. Our approach to AS does not suffer from these drawbacks.

Related Notions. Batch Signature (BS) studied by Aguilar-Melchor et al. [AMAB⁺23] is a different primitive than AS. While AS asks an aggregator to compress N individually generated signatures, BS requests a single signer to generate a compact signature on N messages with the same signing key. AS and BS share somewhat similar goals in that BS also reduces bandwidth and verification complexity, but it requires modification in the signing operations. In contrast, AS typically introduces a customized verification algorithm while the sign algorithm is unchanged.

A.2 Concrete Analysis of Fiat-Shamir

Attema, Cramer, and Kohl [ACK21] generalized special soundness for Σ -protocol to $\mathbf{K} = (k_1, \dots, k_\mu)$ -special soundness for $(2\mu + 1)$ -round public-coin protocols and presented an improved analysis of knowledge extractor in the interactive setting. Attema, Fehr, and Klooß [AFK22] and Wikström [Wik21] concurrently proved concrete knowledge error of Fiat-Shamir AoK constructed from $\mathbf{K}$ -special sound protocols. Fenzi and Nguyen [FMN23] generalized the result of [AFK22] to account for coordinate-wise special sound (CWSS) protocols. Nguyen [Ngu22, 5.1.3, 8.5.1.3] analyzes knowledge soundness of their multi-round interactive protocols using a similar extraction strategy to ours (i.e., extract a candidate witness w , and test the validity of w with freshly sampled challenges). Although our analysis of “extract and test” extraction in the Fiat-Shamir setting may partially resemble [LNP22] and [BF23], we highlight the advantages of our approach in detail.

The Extractor of [LNP22]. Lyubashevsky, Nguyen and Plançon [LNP22, Appendix B] take a similar approach to ours to analyze Fiat-Shamir AoK derived from a 9-round protocol. Although they extend the abstract sampling game of [AFK22] to accommodate probabilistic tests post-extraction, their analysis focuses on the special case where only two challenge values are needed for every round, assuming *always* invertible challenge differences when sampling without replacement in the last round [LS18]. In our framework, we handle a more general case with arbitrary k_i distinct challenges for every round i , coordinate-wise challenges, and imperfect challenge space (i.e. not every challenge difference is invertible in $\mathcal{R}_{q'}$). The last feature is in particular enabled by introducing challenge predicates. We also believe our PSS abstraction has better reusability in that it allows future protocol designers to specify arbitrary predicates for every round and then immediately derive a concrete knowledge error by invoking Theorem 5.1 without going into the details of [AFK22]. In contrast, the approach of [LNP22] is closely coupled to the extraction techniques of [AFK22] requiring a strong understanding of these previous works to apply elsewhere.

Almost Special Soundness. In [BF23] Bünz and Fisch present an extension of special-soundness (dubbed *almost special soundness* (AMSS)) which allows them to enforce additional properties. For protocols to be compatible with their approach, they require that the used commitments be deterministic and therefore non-hiding. This is part of a broader requirement that if the protocol is run again for the same challenges, a resulting accepting transcript must either be the same, or allow breaking binding. Limiting the prover responses is core to their technique, if the prover has the freedom to choose intermediate messages without breaking binding it may significantly bias the distribution of the subsequent challenges through “grinding”. While it does use deterministic commitments, LaBRADOR allows multiple prover responses when projecting the witness, to reduce the completeness error the prover is given a choice of λ different projections.

B Additional Preliminaries

B.1 Signatures

We recall the standard syntax for digital signature schemes.

Definition B.1 (S). A signature scheme (S) for a message space M consists of a tuple of PPT algorithm $S = (\text{Setup}, \text{Gen}, \text{Sign}, \text{Ver})$ defined as follows:

$\text{Setup}(1^\lambda) \rightarrow \text{pp}$: On input the security parameter λ , the setup algorithm outputs the public parameters pp .

$\text{Gen}(\text{pp}) \rightarrow (\text{sk}, \text{pk})$: On input the public parameters pp , the key generation algorithm outputs a pair of secret key sk and public key pk .

Game 1: EU-CMA_S($\mathcal{A}, \lambda$)

<pre> 1: $\text{pp} \leftarrow \text{Setup}(1^\lambda)$ 2: $(\text{pk}, \text{sk}) \leftarrow \text{Gen}(\text{pp})$ 3: $\mathcal{Q} := \emptyset$ 4: $(m, \sigma) \leftarrow \mathcal{A}^{\text{OSign}}(\text{pp}, \text{pk})$ 5: if $\text{Ver}(\text{pk}, m, \sigma) \wedge m \notin \mathcal{Q}$ then 6: return 1 7: else 8: return 0 </pre>	<pre> OSign(m) 1: $\sigma \leftarrow \text{Sign}(\text{sk}, m)$ 2: $\mathcal{Q} := \mathcal{Q} \cup \{m\}$ 3: return σ </pre>
--	---

$\text{Sign}(\text{sk}, m) \rightarrow \sigma$: On input a secret key sk and a message $m \in M$, the signing algorithm outputs a signature σ .

$\text{Ver}(\text{pk}, m, \sigma) \rightarrow b$: On input a public key pk , a message $m \in M$ and a signature σ , the verification algorithm outputs either 1 (accept) or 0 (reject).

Definition B.2 (Correctness). Let $S = (\text{Setup}, \text{Gen}, \text{Sign}, \text{Ver})$ be a signature scheme for a message space M . It is called correct if for all $\lambda \in \mathbb{N}$ and all $m \in M$ it yields

$$\Pr [\text{Ver}(\text{pk}, m, \sigma) = 1] = 1 - \text{negl}(\lambda),$$

where $\text{pp} \leftarrow \text{Setup}(1^\lambda)$, $(\text{sk}, \text{pk}) \leftarrow \text{Gen}(\text{pp})$ and $\sigma \leftarrow \text{Sign}(\text{sk}, m)$.

Definition B.3 (Unforgeability). Let $S = (\text{Setup}, \text{Gen}, \text{Sign}, \text{Ver})$ be a signature scheme for a message space M . It satisfies existential unforgeability under adaptive chosen-message attacks (EU-CMA) if for all PPT adversaries $\mathcal{A}$

$$\text{Adv}_S^{\text{EU-CMA}}(\mathcal{A}) := \Pr [\text{EU-CMA}_S(\mathcal{A}, \lambda) = 1] = \text{negl}(\lambda),$$

where the EU-CMA_S game is described in [Game 1](#).

A slightly stronger notion of EU-CMA security, needed later in [Lemma C.2](#), allows the adversary to have a runtime that is *expected* to be polynomial time.

Definition B.4 (Unforgeability⁺). Let $S = (\text{Setup}, \text{Gen}, \text{Sign}, \text{Ver})$ be a signature scheme for a message space M . It satisfies special existential unforgeability under adaptive chosen-message attacks (EU-CMA⁺) if for all probabilistic adversaries $\mathcal{A}$ who run in expected polynomial time it yields

$$\text{Adv}_S^{\text{EU-CMA}^+}(\mathcal{A}) := \Pr [\text{EU-CMA}_S(\mathcal{A}, \lambda) = 1] = \text{negl}(\lambda),$$

where the EU-CMA_S game is described in [Game 1](#).

There is a (non-tight) generic transformation from EU-CMA to EU-CMA⁺ as we sketch in the following: Let $\mathcal{A}_E$ be an adversary against EU-CMA⁺ security running in expected polynomial time t_E with advantage ε_E . We assume that both values are known. We can generically construct an adversary $\mathcal{A}_W$ against EU-CMA running in worst-case polynomial time as follows: The adversary $\mathcal{A}_W$ runs $\mathcal{A}_E$ as a subroutine for time $2t_E/\varepsilon_E$. After this time, they stop. If $\mathcal{A}_E$ has output something, $\mathcal{A}_W$ forwards this output. Else, they output $\perp$. The runtime of $\mathcal{A}_W$ is $t_W = 2t_E/\varepsilon_E$, which is polynomial. By the Markov inequality, the advantage of $\mathcal{A}_W$ is $\varepsilon_W \geq \varepsilon_E - t_E/(2t_E/\varepsilon_E) = \varepsilon_E/2$.

B.2 Aggregate Signatures

Aggregate signatures (AS) were first introduced by Boneh et al. [[BGLS03](#)]. We recall now the syntax of an AS scheme, together with the definitions of correctness and security.

Definition B.5 (AS). An aggregate signature scheme (AS) for a message space M consists of a tuple of PPT algorithms $\text{AS} = (\text{Setup}, \text{Gen}, \text{Sign}, \text{Ver}, \text{AggSign}, \text{AggVer})$ defined as follows:

$S = (\text{Setup}, \text{Gen}, \text{Sign}, \text{Ver})$ is a signature scheme as in [Definition B.1](#).

Game 2: EU-ACK_{AS}($\mathcal{A}, \lambda$)

1: $\text{pp} \leftarrow \text{Setup}(1^\lambda)$ 2: $(\text{pk}, \text{sk}) \leftarrow \text{Gen}(\text{pp})$ 3: $\mathcal{Q} := \emptyset$ 4: $(\{\text{pk}_i, m_i\}_{i \in [N]}, \sigma_{\text{agg}}) \leftarrow \mathcal{A}^{\text{OSign}}(\text{pp}, \text{pk})$ 5: if $\text{AggVer}(\{\text{pk}_i, m_i\}_{i \in [N]}, \sigma_{\text{agg}}) \wedge \exists i^* \in [N]: (\text{pk}_{i^*} = \text{pk} \wedge m_{i^*} \notin \mathcal{Q})$ then 6: return 1 7: else 8: return 0	OSign(m) 1: $\sigma \leftarrow \text{Sign}(\text{sk}, m)$ 2: $\mathcal{Q} := \mathcal{Q} \cup \{m\}$ 3: return σ
---	---

$\text{AggSign}(\text{pp}, \{\text{pk}_i, m_i, \sigma_i\}_{i \in [N]}) \rightarrow \sigma_{\text{agg}}$: On input a list of N message, public key and signature tuples, the aggregate signing algorithm outputs an aggregate signature σ_{agg} .

$\text{AggVer}(\text{pp}, \{\text{pk}_i, m_i\}_{i \in [N]}, \sigma_{\text{agg}}) \rightarrow b$: On input a list of N message, public-key tuples and an aggregate signature σ_{agg} , the aggregate verification algorithm either outputs 1 (accept) or 0 (reject).

Definition B.6 (Aggregate Correctness). Let $\text{AS} = (\text{Setup}, \text{Gen}, \text{Sign}, \text{Ver}, \text{AggSign}, \text{AggVer})$ be an aggregate signature scheme for a message space M . It is called correct if for all $\lambda, N \in \mathbb{N}$ it yields

$$\Pr [\text{AggVer}(\{\text{pk}_i, m_i\}_{i \in [N]}, \sigma_{\text{agg}}) = 1] = 1 - \text{negl}(\lambda),$$

where $\text{pp} \leftarrow \text{Setup}(1^\lambda)$, $m_i \in M$, $(\text{sk}_i, \text{pk}_i) \leftarrow \text{Gen}(\text{pp})$, $\sigma_i \leftarrow \text{Sign}(\text{sk}_i, m_i)$ for all $i \in [N]$ and $\sigma_{\text{agg}} \leftarrow \text{AggSign}(\{\text{pk}_i, m_i, \sigma_i\}_{i \in [N]})$.

Definition B.7 (Aggregate Unforgeability). An AS scheme satisfies existential unforgeability in the aggregate chosen key model (EU-ACK), if for all PPT adversaries $\mathcal{A}$,

$$\text{Adv}_{\text{AS}}^{\text{EU-ACK}}(\mathcal{A}) := \Pr [\text{EU-ACK}_{\text{AS}}(\mathcal{A}, \lambda) = 1] = \text{negl}(\lambda),$$

where the EU-ACK_{AS} game is described in [Game 2](#).

We sometimes also talk about aggregation in the synchronized setting, as introduced in [\[GR06, AGH10\]](#). Informally, one can think of appending to every message and signature a time period t , and restricting aggregation to signatures of the same time period. During the security game, the adversary is only allowed to get one signature of the honest signing key per time period and per message.

B.3 Falcon Signature Scheme

In the following, we describe the Falcon signature scheme from a high level perspective, focusing on the aspects that are relevant to our work. Falcon [\[PFH⁺22\]](#) is an instantiation of the GPV framework [\[GPV08\]](#) for lattice-based hash-then-sign signature schemes over the NTRU [\[HPS98\]](#) class of structured lattices.¹² It is provably secure in both the classical and quantum random oracle models [\[GPV08, BDF⁺11\]](#).

Falcon works over a power-of-two cyclotomic ring $\mathcal{R}$ modulo q . To sign some target $\mathbf{t} \in \mathcal{R}_q$, the signer uses their secret key together with a trapdoor preimage sampler to sample a pair $(\mathbf{s}_1, \mathbf{s}_2)$ from a discrete Gaussian distribution such that

$$\mathbf{s}_1 + \mathbf{h}\mathbf{s}_2 = \mathbf{t} \text{ and } \|(\mathbf{s}_1, \mathbf{s}_2)\|_2 \leq \beta,$$

where $\mathbf{h} \in \mathcal{R}_q$ is the public key sampled from the NTRU distribution. To compress the signature, one observes that $\mathbf{s}_1$ can be recomputed from $\mathbf{s}_2$ and $\mathbf{t}$, so it does not need to be stored.

To sign a message $m \in \{0, 1\}^*$, the target is chosen according to some hash function $\text{H} : \{0, 1\}^* \rightarrow \mathcal{R}_q$. However, the security of any GPV signature scheme critically relies on the signer not providing two different signatures for the same target. To prevent such collisions, three countermeasures are proposed in [\[GPV08\]](#); making the signature scheme stateful, using a deterministic preimage sampler or adding a random salt. Falcon opted for the latter, sampling some salt $r \xleftarrow{\$} \{0, 1\}^k$ and prepending it to the message before hashing, i.e., $\mathbf{t} = \text{H}(r, m)$. The salt always has to be included in the signature. We call $(r, \mathbf{s}_1, \mathbf{s}_2)$ an *uncompressed* Falcon signature and $(r, \mathbf{s}_2)$ a *compressed* Falcon signature.

¹² We formally recall the definition of a hash-then-sign signature scheme in [Section C.1](#).

Table 4: Relevant Falcon parameters from [PFH⁺22], where λ is the security level, d the ring degree, q the modulus and k the salt bitlength. The signature size is for compressed signatures (r, s_2) . The two security levels are referred to as Falcon-512 and Falcon-1024.

λ	d	q	k	Signature bytelength	Norm bound $\lfloor \beta^2 \rfloor$
128	512	12 289	320	666	34 034 726
256	1024			1280	70 265 242

B.4 SNARKs

Definition B.8 (Binary Relation). A binary relation R_{pp} , parameterized pp , is a set of tuples (x, w) where x is called the statement and w is the witness.

Definition B.9 (Non-Interactive Argument in the ROM). Let R_{pp} be a binary relation and $\mathbf{H}$ be a random oracle. A non-interactive argument system in the random oracle model for R_{pp} is a tuple of PPT algorithms $\Pi = (\mathcal{G}, \mathcal{P}, \mathcal{V})$ defined as follows:

$\mathcal{G}(1^\lambda) \rightarrow \text{pp}$: On input the security parameter λ , the setup algorithm outputs the public parameters pp .

$\mathcal{P}^{\mathbf{H}}(\text{pp}, x, w) \rightarrow \pi$: On input the public parameter pp , a statement x with a witness w , the prove algorithm outputs a proof π .

$\mathcal{V}^{\mathbf{H}}(\text{pp}, x, \pi) \rightarrow b$: On input the public parameter pp , a statement x and a proof π , the verification algorithm outputs either 1 (accept) or 0 (reject).

We recall standard properties of a non-interactive argument system.

Definition B.10 (Completeness). A non-interactive argument Π is called complete if for all $\lambda, N \in \mathbb{N}$, for all $\text{pp} \in \mathcal{G}(1^\lambda)$, and for all $(x, w) \in R_{\text{pp}}$, it yields

$$\Pr[\mathcal{V}^{\mathbf{H}}(\text{pp}, x, \pi) = 1 : \pi \leftarrow \mathcal{P}^{\mathbf{H}}(\text{pp}, x, w)] = 1 - \text{negl}(\lambda).$$

When analyzing knowledge soundness of Fiat-Shamir-transformed predicate-special-sound protocols, we rely on the following formulation.

Definition B.11 (Knowledge soundness [AFK22, Definition 9]). A non-interactive random oracle proof $(\mathcal{P}, \mathcal{V})$ for a relation R is adaptively knowledge sound with knowledge error $\kappa : \mathbb{N} \times \mathbb{N} \rightarrow [0, 1]$ if there exists a positive polynomial p and an algorithm $\mathcal{E}$, called a knowledge extractor, with the following properties: The extractor, given input $n \in \mathbb{N}$ and oracle access to any adaptive Q -query random oracle prover $\mathcal{P}^*$ that outputs statements x with $|x| = n$, runs in an expected number of steps that is polynomial in n and Q (where invocations of $\mathcal{P}^*$ have unit cost) and outputs a tuple $(x, \pi, \text{aux}, v, w)$ such that $\{(x, \pi, \text{aux}, v) : (x, \pi, \text{aux}) \leftarrow \mathcal{P}^{*\text{RO}} \wedge v \leftarrow \mathcal{V}^{\text{RO}}(x, \pi)\}$ and $\{(x, \pi, \text{aux}, v) : (x, \pi, \text{aux}, v, w) \leftarrow \mathcal{E}^{\mathcal{P}^*}(n)\}$ are identically distributed and

$$\Pr[v = 1 \wedge (x, w) \in R \mid (x, \pi, \text{aux}, v, w) \leftarrow \mathcal{E}^{\mathcal{P}^*}(n)] \geq \frac{\varepsilon(\mathcal{P}^*) - \kappa(n, Q)}{p(n)},$$

where $\varepsilon(\mathcal{P}^*) = \Pr[\mathcal{V}^{\text{RO}}(x, \pi) = 1 \mid (x, \pi) \leftarrow \mathcal{P}^{*\text{RO}}]$. Here, $\mathcal{E}$ implements RO for $\mathcal{P}^*$. In particular, $\mathcal{E}$ can arbitrarily program RO. Moreover, the randomness is over the randomness of $\mathcal{E}$, $\mathcal{V}$, $\mathcal{P}^*$ and RO.

We simply say it has *knowledge soundness* if $\kappa(n, Q) = \text{negl}(\lambda)$.

Remark B.1. Note that to show this property, by the linearity of expectation, it is enough to consider deterministic provers $\mathcal{P}^*$ [AFK22, Remark 2].

To prove security of our aggregate signature scheme, we require that our argument of knowledge is still sound for provers receiving auxiliary input. Following the approach of [FN16], we provide a slightly stronger version of knowledge soundness taking this auxiliary input into account. Our formulation is tailored to random oracle-based arguments unlike the CRS-based definition of [FN16, Definition 4]. Accordingly, we make sure that auxiliary input does not depend on the public parameters and the random oracle $\mathbf{H}$ such that it does not interfere with rewinding-based knowledge extraction. We use the definition below to show security of SNARK-based generic construction of aggregate signatures. Additionally, we make the generation of public parameters explicit, to show that the auxiliary is produced independently of the public parameters.

Definition B.12 ($\mathcal{Z}$ -auxiliary Knowledge Soundness). We extend the knowledge soundness definition to allow the prover to receive auxiliary information. Recall $\mathcal{G}$ produces public parameters pp . Let $\mathcal{Z}$ be a PPT algorithm taking the security parameter as input and outputting some auxiliary information aux-in . We make the following changes from Definition B.11, the prover is given additional inputs $\text{pp} \leftarrow \mathcal{G}(1^\lambda)$ and $\text{aux-in} \leftarrow \mathcal{Z}(1^\lambda)$, the extractor also receives pp and aux-in . Knowledge soundness is now quantified over executions of $\mathcal{G}$ and $\mathcal{Z}$,

$$\Pr \left[v = 1 \wedge (x, w) \in R_{\text{pp}} \mid \begin{array}{c} \text{pp} \leftarrow \mathcal{G}(1^\lambda), \text{aux-in} \leftarrow \mathcal{Z}(1^\lambda), \\ (x, \pi, \text{aux-out}, v, w) \leftarrow \mathcal{E}^{\mathcal{P}^*}(\text{pp}, \text{aux-in})(n, \text{pp}, \text{aux-in}) \end{array} \right] \geq \varepsilon(\mathcal{P}^*) - \kappa(n, Q).$$

It is relatively straightforward to check that LaBRADOR also satisfies Definition B.12 as we remark after Theorem I.1.

We say that Π is a *Proof of Knowledge* if it satisfies completeness and knowledge soundness. If knowledge soundness only holds for PPT adversaries, we say that is an *Argument* of Knowledge. Moreover, if $|\pi| \in O(\text{poly}(\lambda) \cdot \text{polylog}(|x| + |w|))$ then we say Π is *succinct*. A succinct non-interactive argument of knowledge is called a *SNARK*. A common approach for designing non-interactive protocols is transforming interactive protocols to a corresponding non-interactive protocol in the random oracle model.

Definition B.13 (Fiat-Shamir Transform). The adaptive Fiat-Shamir transformation $\text{FS}[\Pi]$ of a protocol $\Pi = (\mathcal{P}, \mathcal{V})$ is a non-interactive argument in the ROM (Definition B.9). For a statement x and witness w , the prover $\mathcal{P}^{\text{RO}_1, \dots, \text{RO}_\mu}$ runs $\mathcal{P}_\Pi$, but rather than interacting with the verifier to obtain the challenge it instead queries the random oracle, more precisely in round i with prover message a_i

$$c_i \leftarrow \text{RO}_i(x, i, c_{i-1}, a_i).$$

The prover outputs proof $\pi = (a_1, \dots, a_{\mu+1})$. Given a proof π , the verifier $\mathcal{V}^{\text{RO}_1, \dots, \text{RO}_\mu}$ recomputes the challenges as $c_i \leftarrow \text{RO}_i(x, i, c_{i-1}, a_i)$ and accepts iff $\mathcal{V}_\Pi$ accepts the transcript $(a_1, c_1, \dots, c_\mu, a_{\mu+1})$.

B.5 Coordinate-Wise Special Soundness

In this section, we recap the notion of coordinate-wise special soundness introduced in [FMN23]. Assume that the challenge sent by the verifier is a vector $\vec{c} \in S^r$ for some set S . For each $i \in [r]$, we define the relation $\equiv_i$ for challenge vectors that only differ in the i -th coordinate. For $\vec{x} = (x_1, \dots, x_r) \in S^r$ and $\vec{y} = (y_1, \dots, y_r) \in S^r$,

$$\vec{x} \equiv_i \vec{y} \iff x_i \neq y_i, \forall j \in [r] \setminus \{i\} : x_j = y_j.$$

Using this relation, the set of permissible sets of challenge vectors is defined as

$$\text{SS}(S, r, k) = \left\{ \{\vec{c}_1, \dots, \vec{c}_K\} \subseteq 2^{(S^r)} \mid \begin{array}{l} \exists e \in [K], \forall i \in [r], \\ \exists J = \{j_1, \dots, j_{k-1}\} \subseteq [K] \setminus \{e\}, \\ \forall j, j' \in J \cup \{e\}, j \neq j' : \vec{c}_j \equiv_i \vec{c}_{j'} \end{array} \right\},$$

where $K = r(k-1) + 1$. The vector $\vec{c}_e$ is the unique element such that any other vector in the set differs to it in exactly one coordinate. We say that $\vec{c}_e$ is the central vector. For each coordinate i , there is exactly $k-1$ distinct vectors that differ with $\vec{c}_e$ in only the i -th coordinate. Hence, for each coordinate we have k distinct challenges.

Definition B.14 (r -coordinate-wise k -special-soundness). Let $k, r \in \mathbb{N}$ and let Π be a 3-message public-coin proof/argument of knowledge for a relation R . Assume the challenge set of Π is S^r for some set S . We say that Π is r -coordinate-wise k -special-sound if there exists a polynomial time algorithm which on input a statement x and $r(k-1) + 1$ transcripts with the same first message and challenges in $\text{SS}(S, r, k)$ outputs a witness w such that $(x, w) \in R$.

Setting $r = 1$ yields regular k -special-soundness. The notion generalizes to $2\mu + 1$ -message $(r_1, \dots, r_\mu)$ -coordinate-wise $(k_1, \dots, k_\mu)$ -special sound in the same way as before. Assume the m -th challenge set is $S_m^{r_m}$. Given $r_\mu(k_\mu - 1) + 1$ transcripts with the same prefix $(a_1, c_1, a_2, c_2, \dots, a_\mu)$ and μ -th challenge vectors in $\text{SS}(S_\mu, r_\mu, k_\mu)$, we obtain a $(r_1, \dots, r_\mu)$ -coordinate-wise $(1, \dots, 1, k_\mu)$ -special-sound tree of transcripts. Finding $r_{\mu-1}(k_{\mu-1} - 1) + 1$ such trees with the same prefix $(a_1, c_1, \dots, a_{\mu-1})$ and $(\mu-1)$ -th challenges in $\text{SS}(S_{\mu-1}, r_{\mu-1}, k_{\mu-1})$, we obtain a $(r_1, \dots, r_\mu)$ -coordinate-wise $(1, \dots, 1, k_{\mu-1}, k_\mu)$ -tree of transcripts, etc.

Definition B.15 (R -coordinate-wise K -special-soundness). Let $\mu, k_1, \dots, k_\mu, r_1, \dots, r_\mu \in \mathbb{N}$ and let Π be a $2\mu + 1$ -message public-coin proof/argument of knowledge for a relation R . Let $\mathbf{K} = (k_1, \dots, k_\mu)$ and let $\mathbf{R} = (r_1, \dots, r_\mu)$. Assume the m -th challenge set is $S_m^{r_m}$ for some set S_m . We say that Π is R -coordinate-wise K -special-sound if there exists a polynomial time algorithm which on input a statement x and a R -coordinate-wise K -tree of transcripts outputs a witness w such that $(x, w) \in R$.

B.6 Details of LaBRADOR

The LaBRADOR protocol is an iterative multi-round public-coin interactive proof, which can be made non-interactive in the random oracle model by applying the Fiat-Shamir transform. The security of LaBRADOR relies on the hardness of the Module Shortest Integer Solution problem. Informally, each iteration takes as input a statement x and witness w and produces a transcript that is a proof of knowledge for $(x, w) \in R$. All of the messages in this transcript except the last message define a part of small size included in the final proof. The last message w' is sent by the prover and defines a new statement x' , such that $(x', w') \in R$. The new witness w' is shorter than w , but might still not be very short. Instead of including the last message w' in the final proof, we prove that $(x', w') \in R$. Again, all but the last message are inserted in the final proof. This iterative process is repeated until we no longer make progress on the length of the final message w' , at which point we output w' as the last part of the final proof.

Each iteration consists of 5 steps. Like the original paper, we present the interactive version of the protocol. Later in this paper, we concretely analyze the Fiat-Shamir transform of this protocol. For completeness, we describe the complete procedures for the first iteration of LaBRADOR prover and verifier in [Protocol 2](#) and [Protocol 3](#), respectively. In what follows, we explain every step in detail.

Step 1. Commit: The prover commits to each witness vector with an Ajtai commitment $\vec{v}_i = \mathbf{A}\vec{w}_i \in \mathcal{R}_q^\kappa$ [Ajt96]. Note that the norm check on the sum of the witnesses implies that $\|\vec{w}_i\|_2 \leq \beta$. Thus, the parameter κ is set such that $\text{M-SIS}_{\kappa, 2\beta}$ is hard, so that the commitments are binding. Notice that for the purpose of succinctness, we do not need that the commitments are hiding.

Sending each commitment to the verifier would be costly, so instead the prover produces a single outer commitment to all the $\vec{v}_i$. We refer to the $\vec{v}_i$ as the inner commitments. Because the inner commitments are not short, they first have to be decomposed into $t_1 \geq 2$ parts with respect to a small base b_1 , i.e., $\vec{v}_i = \vec{v}_i^{(0)} + b_1 \vec{v}_i^{(1)} + \dots + b_1^{t_1-1} \vec{v}_i^{(t_1-1)}$. All of these parts are concatenated to $\vec{v} \in \mathcal{R}_q^{t_1 \kappa r}$.

To efficiently open a large number of commitments at once, it will be helpful to also commit to garbage polynomials $\mathbf{g}_{i,j} = \langle \vec{w}_i, \vec{w}_j \rangle$ for $i, j \in [r]$. As for $\vec{v}$, we decompose $\mathbf{g}_{i,j}$ into t_2 parts w.r.t. the basis b_2 , and concatenating them into $\vec{g} \in \mathcal{R}_q^{t_2(r^2+r)/2}$. See [Step 4](#) for how $\vec{g}$ is used. Finally, the prover produces the outer commitment to $\vec{v}$ and $\vec{g}$ with $\vec{u}_1 = \mathbf{B}\vec{v} + \mathbf{C}\vec{g} \in \mathcal{R}_q^{\kappa_1}$ and sends it to the verifier. Note the matrices used for Ajtai commitments are public parameters.

Step 2. Project: To reach a security level λ , the verifier samples $\Pi_i \leftarrow \mathbb{C}^{2\lambda \times (nd)}$ for $i = 1, \dots, r$ and sends them to the prover. The prover responds with the combined projection $\vec{p} = \sum_{i=1}^r \Pi_i \tau(\vec{w}_i) \in \mathbb{Z}_q^{2\lambda}$. The verifier checks whether $\|\vec{p}\|_2 \leq \sqrt{\lambda}\beta$. If not, the procedure is repeated until the prover can come up with a short enough $\vec{p}$.

To enforce that the final projection was computed correctly, new constraints are added. Let $\vec{\pi}_i^{(j)}$ be the (unique) ring element corresponding to the j -th row of Π_i . For $j = 1, \dots, 2\lambda$, applying Equation 1 and using the linearity of $\text{ct}(\cdot)$, we observe that

$$\begin{aligned} 0 &= \sum_{i=1}^r \langle \tau(\vec{\pi}_i^{(j)}), \tau(\vec{w}_i) \rangle - p_j = \sum_{i=1}^r \text{ct} \left(\langle \sigma_{-1}(\vec{\pi}_i^{(j)}), \vec{w}_i \rangle \right) - p_j \\ &= \text{ct} \left(\sum_{i=1}^r \langle \sigma_{-1}(\vec{\pi}_i^{(j)}), \vec{w}_i \rangle - p_j \right), \end{aligned}$$

where p_j is the j -th coordinate of $\vec{p}$. This defines a LaBRADOR compatible constant-term constraint.

Step 3. Aggregate constraints: The goal in this step is to aggregate all the dot product constraints in $\mathcal{F}$ and $\mathcal{F}'$ to a single dot product constraint F . First, for security level λ , $\mathcal{F}'$ is aggregated to $\lceil \lambda / \log_2(q) \rceil$ constant term constraints by taking random linear combinations. For $k = 1, \dots, \lceil \lambda / \log_2(q) \rceil$, the verifier

sends $\vec{\psi}^{(k)} \xleftarrow{\$} \mathbb{Z}_q^{|\mathcal{F}'|}$, $\vec{\omega}^{(k)} \xleftarrow{\$} \mathbb{Z}_q^{2\lambda}$, which define the constraint

$$\begin{aligned} f''^{(k)}(\vec{w}_1, \dots, \vec{w}_r) &= \sum_{j=1}^{|\mathcal{F}'|} \psi_j^{(k)} f'^{(j)}(\vec{w}_1, \dots, \vec{w}_r) \\ &\quad + \sum_{j=1}^{2\lambda} \omega_j^{(k)} \left(\sum_{i=1}^r \langle \sigma_{-1}(\vec{\pi}_i^{(j)}), \vec{w}_i \rangle - p_j \right) \\ &= \sum_{i,j=1}^r \mathbf{a}_{i,j}''^{(k)} \langle \vec{w}_i, \vec{w}_j \rangle + \sum_{i=1}^r \langle \vec{\varphi}_i''^{(k)}, \vec{w}_i \rangle - b_0''^{(k)}, \end{aligned}$$

where $\mathbf{a}_{i,j}''^{(k)}$, $\vec{\varphi}_i''^{(k)}$ and $b_0''^{(k)}$ are set accordingly. Next, the $f''^{(k)}$ are extended to full constraints of the type in $\mathcal{F}$. The prover extends the integers $b_0''^{(k)}$ to full polynomials $\mathbf{b}''^{(k)}$ such that $f''^{(k)}(\vec{w}_1, \dots, \vec{w}_r) = \mathbf{0}$. The prover then sends the $\mathbf{b}''^{(k)}$ to the verifier, which checks that the constant term of each has the correct value.

Finally, the functions in $\mathcal{F}$ and the $f''^{(k)}$ are aggregated to a single dot product constraint F by again taking a random linear combination. The verifier sends $\vec{\alpha} \xleftarrow{\$} \mathcal{R}_q^{|\mathcal{F}|}$ and $\vec{\beta} \xleftarrow{\$} \mathcal{R}_q^{\lceil \lambda / \log_2(q) \rceil}$, which define

$$\begin{aligned} F(\vec{w}_1, \dots, \vec{w}_r) &= \sum_{k=1}^{|\mathcal{F}|} \alpha_k f^{(k)}(\vec{w}_1, \dots, \vec{w}_r) + \sum_{k=1}^{\lceil \lambda / \log_2(q) \rceil} \beta_k f''^{(k)}(\vec{w}_1, \dots, \vec{w}_r) \\ &= \sum_{i,j=1}^r \mathbf{a}_{i,j} \langle \vec{w}_i, \vec{w}_j \rangle + \sum_{i=1}^r \langle \vec{\varphi}_i, \vec{w}_i \rangle - \mathbf{b}, \end{aligned}$$

where again $\mathbf{a}_{i,j}$, $\vec{\varphi}_i$ and $\mathbf{b}$ are set accordingly.

Having previously committed to the garbage polynomials $\mathbf{g}_{i,j}$, we will now also need to commit to the linear terms $\mathbf{h}_{i,j} = \frac{1}{2} (\langle \vec{\varphi}_i, \vec{w}_j \rangle + \langle \vec{\varphi}_j, \vec{w}_i \rangle)$ for $i, j \in [r]$. Once again, the $\mathbf{h}_{i,j}$ are each decomposed into t_1 parts with respect to the basis b_1 , and concatenated to the vector $\vec{\mathbf{h}} \in \mathcal{R}_q^{t_1(r^2+r)/2}$. The verifier commits to $\vec{\mathbf{h}}$ with $\vec{\mathbf{u}}_2 = D\vec{\mathbf{h}}$.

Step 4. Amortize witness: In order to convince the verifier that $F(\vec{w}_1, \dots, \vec{w}_r) = \mathbf{0}$ the prover provides a random linear combination of the witness vectors. The verifier sends challenges $\mathbf{c}_i \leftarrow \mathcal{C}$ for $i = 1, \dots, r$ and the prover computes $\vec{\mathbf{z}} = \sum_{i=1}^r \mathbf{c}_i \vec{w}_i$. Instead of checking that F is satisfied by the witness, the verifier will check that the following 3 dot product constraints hold:

$$\begin{aligned} (1) \quad \langle \vec{\mathbf{z}}, \vec{\mathbf{z}} \rangle &= \sum_{i,j=1}^r \mathbf{g}_{i,j} \mathbf{c}_i \mathbf{c}_j, & (2) \quad \sum_{i=1}^r \langle \vec{\varphi}_i, \vec{\mathbf{z}} \rangle \mathbf{c}_i &= \sum_{i,j=1}^r \mathbf{h}_{i,j} \mathbf{c}_i \mathbf{c}_j, \\ (3) \quad \sum_{i,j=1}^r \mathbf{a}_{i,j} \mathbf{g}_{i,j} + \sum_{i=1}^r \mathbf{h}_{i,i} &- \mathbf{b} = \mathbf{0} \end{aligned}$$

Here the $\mathbf{g}_{i,j}$ and the $\mathbf{h}_{i,j}$ are the committed garbage polynomials. Due to the symmetry $\mathbf{g}_{i,j} = \mathbf{g}_{j,i} = \langle \vec{w}_i, \vec{w}_j \rangle$, only $r(r+1)/2$ terms are needed. The same holds for $\mathbf{h}_{i,j} = \mathbf{h}_{j,i} = \frac{1}{2} (\langle \vec{\varphi}_i, \vec{w}_j \rangle + \langle \vec{\varphi}_j, \vec{w}_i \rangle)$.

For the final message, the prover sends $\vec{\mathbf{z}}, \vec{\mathbf{v}}, \vec{\mathbf{g}}, \vec{\mathbf{h}}$. The verifier checks the following:

1. That the above constraints (1), (2) and (3) are satisfied.
2. That $\vec{\mathbf{z}}$ is a somewhat short amortized opening to the $\vec{w}_i$'s, so that $\mathbf{A}\vec{\mathbf{z}} = \sum_{i=1}^r \mathbf{c}_i \vec{w}_i$.
3. That $(\vec{\mathbf{v}}, \vec{\mathbf{g}})$ and $\vec{\mathbf{h}}$ are somewhat short openings to respectively $\vec{\mathbf{u}}_1$ and $\vec{\mathbf{u}}_2$.

Step 5. Recurse: Instead of including the last message in the transcript, we recursively run the protocol with it as the new witness. The three verification checks from above that the last message should satisfy are translated to a new statement in R .

First, define $\vec{\mathbf{e}} = \vec{\mathbf{v}} \|\vec{\mathbf{g}}\| \vec{\mathbf{h}} \in \mathcal{R}_q^m$ with $m = rt_1\kappa + (t_1 + t_2)(r^2 + r)/2$. To avoid a blowup in the witness norm, $\vec{\mathbf{z}}$ is decomposed into 2 parts $\vec{\mathbf{z}}^{(0)}, \vec{\mathbf{z}}^{(1)}$ w.r.t. the basis b . The new set of witness vectors are now $\vec{\mathbf{z}}^{(0)}, \vec{\mathbf{z}}^{(1)}, \vec{\mathbf{e}}$. Note that they can all be made to have the rank $n' = \max(n, m)$ by padding with 0's when

necessary. The verification equations depend linearly on $\vec{v}, \vec{g}$ and $\vec{h}$. Hence, it is easy to see how to reformulate them to dot product constraints $\tilde{\mathcal{F}}$ in $\vec{z}^{(0)}, \vec{z}^{(1)}, \vec{e}$. The norm checks for the commitment openings are combined into one global check $\|\vec{z}^{(0)}\|_2^2 + \|\vec{z}^{(1)}\|_2^2 + \|\vec{e}\|_2^2 \leq \beta'^2$. The purpose of the original norm checks was to ensure that the openings were binding. For appropriate parameters, this is still implied by the combined ℓ_2 -norm bound β' . Thus, we arrive at a statement in R .

Next, the witness is folded to give better control of the parameters in the next iteration. Given folding parameters ν and μ , $\vec{z}^{(0)}, \vec{z}^{(1)} \in \mathcal{R}_q^n$ are folded into ν pieces in $\mathcal{R}_q^{\lceil n/\nu \rceil}$ and $\vec{e}$ is folded into μ pieces in $\mathcal{R}_q^{\lceil m/\mu \rceil}$. By this, we mean that we obtain $r' = 2\nu + \mu$ witness vectors $\vec{w}'_1, \dots, \vec{w}'_{2\nu+\mu}$ such that $\vec{z}^{(0)} = \vec{w}'_1 \parallel \dots \parallel \vec{w}'_\nu$, $\vec{z}^{(1)} = \vec{w}'_{\nu+1} \parallel \dots \parallel \vec{w}'_{2\nu}$ and $\vec{e} = \vec{w}'_{2\nu+1} \parallel \dots \parallel \vec{w}'_{2\nu+\mu}$. Padding with 0's when necessary, the new witness vectors have rank $n' = \max(\frac{n}{\nu}, \frac{m}{\mu})$. The folding parameters are chosen such that $\frac{n}{\nu} \approx \frac{m}{\mu}$, to minimize the padding required. Reformulating the constraints in terms of these new witness vectors is easy, since in general $\langle \vec{x}_1 \parallel \vec{x}_2, \vec{y}_1 \parallel \vec{y}_2 \rangle = \langle \vec{x}_1, \vec{y}_1 \rangle + \langle \vec{x}_2, \vec{y}_2 \rangle$. The same combined ℓ_2 -norm bound β' still applies. Hence, $(w'_1, \dots, w'_r)$ define the witness for the statement $(\tilde{\mathcal{F}}, \beta')$ in R , which is to be used in the next iteration.

The goal of the recursion is to reduce the witness rank n . The strategy is to carefully pick ν, μ at each iteration such that enough progress is made in reducing n' while not blowing up r' . Since $m' = O(r'^2)$, too large ν, μ will be counterproductive to further reducing n' . For the recursion, Beullens and Seiler set the decomposition parameters heuristically such that $b \approx b_1 \approx b_2$, which ensures that all the witness vectors have roughly the same width (i.e., the same ℓ_∞ -norm). They also want to be in the balanced state where $2n \approx m$, where the ℓ_2 -norm contribution of $\vec{z}^{(0)} \parallel \vec{z}^{(1)}$ and $\vec{e}$ is about the same. The strategy will be to reduce $\frac{n}{\nu}$ as much as possible while maintaining that $\frac{n}{\nu} \approx \frac{m}{\mu}$ and that $2n' \approx m'$. Asymptotically, the optimal choice is $r' = O(r^{1/3})$, yielding $n' = O(r^{2/3})$. Hence, only $O(\log \log n)$ iterations of the base protocol are needed.

Protocol 2: The LaBRADOR Protocol

Relation R Consists of tuples of statement $x = (((\mathbf{a}_{i,j}^{(k)})_{i,j \in [r]}, (\vec{\varphi}_i^{(k)})_{i \in [r]}, \mathbf{b}^{(k)})_{k \in [K]}, ((\mathbf{a}_{i,j}'^{(l)})_{i,j \in [r]}, (\vec{\varphi}_i'^{(l)})_{i \in [r]}, b_0'^{(l)})_{l \in [L]}, \beta)$ and witness $w = (\vec{\mathbf{w}}_1, \dots, \vec{\mathbf{w}}_r)$ where $\vec{\mathbf{w}}_i \in \mathcal{R}_{q'}^n$ for $i \in [r]$ such that

$$\begin{aligned} f^{(k)}(\vec{\mathbf{w}}_1, \dots, \vec{\mathbf{w}}_r) &= \sum_{i,j=1}^r \mathbf{a}_{i,j}^{(k)} \langle \vec{\mathbf{w}}_i, \vec{\mathbf{w}}_j \rangle + \sum_{i=1}^r \langle \vec{\varphi}_i^{(k)}, \vec{\mathbf{w}}_i \rangle - \mathbf{b}^{(k)} = \mathbf{0}, \forall k \in [K] \\ \text{ct} \left(f'^{(l)}(\vec{\mathbf{w}}_1, \dots, \vec{\mathbf{w}}_r) \right) &= \text{ct} \left(\sum_{i,j=1}^r \mathbf{a}_{i,j}'^{(l)} \langle \vec{\mathbf{w}}_i, \vec{\mathbf{w}}_j \rangle + \sum_{i=1}^r \langle \vec{\varphi}_i'^{(l)}, \vec{\mathbf{w}}_i \rangle - b_0'^{(l)} \right) \bmod q', \forall l \in [L] \\ \sum_{i=1}^r \|\vec{\mathbf{w}}_i\|_2^2 &\leq \beta^2. \end{aligned}$$

$\mathcal{G}(1^\lambda)$ Uniformly sample $\mathbf{A} \in \mathcal{R}_{q'}^{\kappa \times n}$, $\mathbf{B}_{ik} \in \mathcal{R}_{q'}^{\kappa_1 \times \kappa}$ for $i \in [1, r]$, $k \in [0, t_1 - 1]$, $\mathbf{C}_{ijk} \in \mathcal{R}_{q'}^{\kappa_2 \times 1}$ for $i \in [1, r]$, $j \in [i, r]$, $k \in [0, t_2 - 1]$, and $\mathbf{D}_{i,j,k} \in \mathcal{R}_{q'}^{\kappa_2 \times 1}$ for $i \in [1, r]$, $j \in [i, r]$, $k \in [0, t_1 - 1]$. Output $\text{pp} = (\mathbf{A}, \mathbf{B}_{ik}, \mathbf{C}_{ijk}, \mathbf{D}_{ijk})$

$\mathcal{P}(\text{pp}, x, w)$

1. Commit:

- 1: For $i \in [r]$: $\vec{\mathbf{v}}_i = \mathbf{A}\vec{\mathbf{w}}_i = \vec{\mathbf{v}}_i^{(0)} + \dots + \vec{\mathbf{v}}_i^{(t_1-1)} b_1^{t_1-1}$ // Generate inner commitments
- 2: For $i \in [r], j \in [i, r]$: $\mathbf{g}_{ij} = \langle \vec{\mathbf{w}}_i, \vec{\mathbf{w}}_j \rangle = \mathbf{g}_{ij}^{(0)} + \dots + \mathbf{g}_{ij}^{(t_2-1)} b_2^{t_2-1}$
- 3: $\vec{\mathbf{u}}_1 = \sum_{i=1}^r \sum_{k=0}^{t_1-1} \mathbf{B}_{ik} \vec{\mathbf{v}}_i^{(k)} + \sum_{i \leq j} \sum_{k=0}^{t_2-1} \mathbf{C}_{ijk} \mathbf{g}_{ij}^{(k)}$ // Generate outer commitment
- 4: Send $\vec{\mathbf{u}}_1$ to $\mathcal{V}$

2. Projection Upon receiving $\Pi_i = (\vec{\pi}_i^{(j)})_{j \in [2\lambda]} \xleftarrow{\$} \mathcal{C}^{2\lambda \times (nd)}$ for $i \in [r]$ from $\mathcal{V}$:

- 1: For $j \in [2\lambda]$: $p_j = \sum_{i=1}^r \langle \vec{\pi}_i^{(j)}, \vec{\mathbf{w}}_i \rangle$
- 2: Send $\vec{p} = (p_j)_{j \in [2\lambda]}$ to $\mathcal{V}$

3. Aggregating $f'^{(l)}$ Let $K'' = \lceil \lambda / \log q \rceil$. Upon receiving $\psi_l^{(k)}, \omega_j^{(k)} \in \mathbb{Z}_{q'}$ for $k \in [K''], l \in [L], j \in [2\lambda]$ from $\mathcal{V}$:

- 1: For $k \in [K'']$: $\mathbf{a}_{ij}''^{(k)} = \sum_{l=1}^L \psi_l^{(k)} \mathbf{a}_{ij}^{(l)}$, $\vec{\varphi}_i''^{(k)} = \sum_{l=1}^L \psi_l^{(k)} \vec{\varphi}_i^{(l)} + \sum_{j=1}^{2\lambda} \omega_j^{(k)} \sigma_{-1}(\vec{\pi}_i^{(j)})$, $b_0''^{(k)} = \sum_{l=1}^L \psi_l^{(k)} b_0'^{(l)} + \langle \vec{\omega}^{(k)}, \vec{p} \rangle$,

$$\begin{aligned} f''^{(k)}(\vec{\mathbf{w}}_1, \dots, \vec{\mathbf{w}}_r) &:= \sum_{l=1}^L \psi_l^{(k)} f'^{(l)}(\vec{\mathbf{w}}_1, \dots, \vec{\mathbf{w}}_r) + \sum_{j=1}^{2\lambda} \omega_j^{(k)} (\langle \sigma_{-1}(\vec{\pi}_i^{(j)}), \vec{\mathbf{w}}_i \rangle - p_j) \\ &= \sum_{i,j=1}^r \mathbf{a}_{ij}''^{(k)} \langle \vec{\mathbf{w}}_i, \vec{\mathbf{w}}_j \rangle + \sum_{i=1}^r \langle \vec{\varphi}_i''^{(k)}, \vec{\mathbf{w}}_i \rangle - b_0''^{(k)} \\ \mathbf{b}''^{(k)} &:= \sum_{i,j=1}^r \mathbf{a}_{ij}''^{(k)} \langle \vec{\mathbf{w}}_i, \vec{\mathbf{w}}_j \rangle + \sum_{i=1}^r \langle \vec{\varphi}_i''^{(k)}, \vec{\mathbf{w}}_i \rangle \end{aligned}$$

- 2: Send $(\mathbf{b}''^{(k)})_{k \in [K'']}$ to $\mathcal{V}$.

4. Aggregating linear constraints Upon receiving $\vec{\alpha} \in \mathcal{R}_{q'}^K$ and $\vec{\beta} \in \mathcal{R}_{q'}^{K''}$ from $\mathcal{V}$:

- 1: For $i, j \in [r]$: $\vec{\varphi}_i = \sum_{k=1}^K \alpha_k \vec{\varphi}_i^{(k)} + \sum_{k=1}^{K''} \beta_k \vec{\varphi}_i''^{(k)}$.
- 2: For $i \in [r], j \in [i, r]$: $(\langle \vec{\varphi}_i, \vec{\mathbf{w}}_j \rangle + \langle \vec{\varphi}_j, \vec{\mathbf{w}}_i \rangle) / 2 = \mathbf{h}_{ij}^{(0)} + \dots + \mathbf{h}_{ij}^{(t_1-1)} b_1^{t_1-1}$.
- 3: Send $\vec{\mathbf{u}}_2 = \sum_{i \leq j} \sum_{k=0}^{t_1-1} \mathbf{D}_{ijk} \mathbf{h}_{ij}^{(k)}$ to $\mathcal{V}$.

5. Amortizing opening proof Upon receiving $\mathbf{c}_i \in \mathcal{C}$ for $i \in [r]$:

- 1: $\vec{\mathbf{z}} = \sum_{i=1}^r \mathbf{c}_i \vec{\mathbf{w}}_i = \vec{\mathbf{z}}^{(0)} + b \vec{\mathbf{z}}^{(1)}$.
- 2: Send $\vec{\mathbf{z}}^{(0)}, \vec{\mathbf{z}}^{(1)}$ and $\vec{\mathbf{v}}_i, \mathbf{g}_{i,j}, \mathbf{h}_{ij}$ for $i \in [r], j \in [i, r]$ to $\mathcal{V}$.

Protocol 3: The LaBRADOR Protocol (cont.)

$\mathcal{V}(\text{pp}, x, \tau)$

1. Parse $\tau = (\vec{\mathbf{u}}_1, (\vec{\pi}_i^{(j)})_{i \in [r], j \in [2\lambda]}, \vec{p}, (\psi_l^{(k)}, \omega_j^{(k)})_{k \in [K''], l \in [L], j \in [2\lambda]}, (\mathbf{b}''^{(k)})_{k \in [K'']}, \vec{\alpha}, \vec{\beta}, \vec{\mathbf{u}}_2, (\mathbf{c}_i)_{i \in [r]}, \vec{\mathbf{z}}, (\vec{\mathbf{v}}_i, \mathbf{g}_{i,j}, \mathbf{h}_{ij})_{i \in [r], j \in [i, r]})$
2. **Check constant term of $\mathbf{b}''^{(k)}$** Check $b_0''^{(k)} = \sum_{l=1}^L \psi_l^{(k)} b_0^{(l)} + \langle \vec{\omega}^{(k)}, \vec{p} \rangle$
3. **Compute aggregated relation** Define aggregated statement F such that

$$\begin{aligned} F(\vec{\mathbf{w}}_1, \dots, \vec{\mathbf{w}}_r) &= \sum_{k=1}^K \alpha_k f^{(k)}(\vec{\mathbf{w}}_1, \dots, \vec{\mathbf{w}}_r) + \sum_{k=1}^{K''} \beta_k f''^{(k)}(\vec{\mathbf{w}}_1, \dots, \vec{\mathbf{w}}_r) \\ &= \sum_{i,j=1}^r \mathbf{a}_{i,j} \langle \vec{\mathbf{w}}_i, \vec{\mathbf{w}}_j \rangle + \sum_{i=1}^r \langle \vec{\varphi}_i, \vec{\mathbf{w}}_i \rangle - \mathbf{b} = 0 \end{aligned}$$

- 1: For $k \in [K'']$: $\mathbf{a}_{ij}''^{(k)} = \sum_{l=1}^L \psi_l^{(k)} \mathbf{a}_{ij}'^{(l)}$, $\vec{\varphi}_i''^{(k)} = \sum_{l=1}^L \psi_l^{(k)} \vec{\varphi}_i^{(l)} + \sum_{j=1}^{2\lambda} \omega_j^{(k)} \sigma_{-1}(\vec{\pi}_i^{(j)})$
- 2: For $i \in [r], j \in [i, r]$: $\mathbf{a}_{ij} = \sum_{k=1}^K \alpha_k \mathbf{a}_{ij}^{(k)} + \sum_{k=1}^{K''} \beta_k \mathbf{a}_{ij}''^{(k)}$
- 3: For $i \in [r]$: $\vec{\varphi}_i = \sum_{k=1}^K \alpha_k \vec{\varphi}_i^{(k)} + \sum_{k=1}^{K''} \beta_k \vec{\varphi}_i''^{(k)}$
- 4: $\mathbf{b} = \sum_{k=1}^K \alpha_k \mathbf{b}^{(k)} + \sum_{k=1}^{K''} \beta_k \mathbf{b}''^{(k)}$
4. **Check amortized opening of inner commitments** Check $\mathbf{A}\vec{\mathbf{z}} = \sum_{i=1}^r \mathbf{c}_i \vec{\mathbf{v}}_i$
5. **Check aggregated innerproduct constraints** Check $\langle \vec{\mathbf{z}}, \vec{\mathbf{z}} \rangle = \sum_{i,j=1}^r \mathbf{g}_{ij} \mathbf{c}_i \mathbf{c}_j$
6. **Check aggregated linear constraints** Check $\sum_{i=1}^r \langle \vec{\varphi}_i, \vec{\mathbf{z}} \rangle \mathbf{c}_i = \sum_{i,j=1}^r \mathbf{h}_{ij} \mathbf{c}_i \mathbf{c}_j$
7. **Check aggregated relation** Check $\sum_{i,j=1}^r \mathbf{a}_{ij} \mathbf{g}_{ij} + \sum_{i=1}^r \mathbf{h}_{ii} - \mathbf{b} = 0$
8. **Check norms of decomposed inner commitments**
 - 1: $\vec{\mathbf{z}} = \mathbf{b}\vec{\mathbf{z}}^{(1)} + \vec{\mathbf{z}}^{(0)}$
 - 2: For $i \in [r]$: $\vec{\mathbf{v}}_i = \vec{\mathbf{v}}_i^{(0)} + \dots + \vec{\mathbf{v}}_i^{(t_1-1)} b_1^{t_1-1}$
 - 3: For $i \in [r], j \in [i, r]$: $\mathbf{g}_{ij} = \mathbf{g}_{ij}^{(0)} + \dots + \mathbf{g}_{ij}^{(t_2-1)} b_2^{t_2-1}$
 - 4: For $i \in [r], j \in [i, r]$: $\mathbf{h}_{ij} = \mathbf{h}_{ij}^{(0)} + \dots + \mathbf{h}_{ij}^{(t_1-1)} b_1^{t_1-1}$
 - 5: Check $\sum_{k=0}^1 \|\vec{\mathbf{z}}^{(k)}\|^2 + \sum_{i=1}^r \sum_{k=0}^{t_1-1} \|\vec{\mathbf{v}}_i^{(k)}\|^2 + \sum_{i=1}^r \sum_{k=0}^{t_2-1} \|\mathbf{g}_{ij}^{(k)}\|^2 + \sum_{i=1}^r \sum_{k=0}^{t_1-1} \|\mathbf{h}_{ij}^{(k)}\|^2 \leq \beta'$
9. **Check opening of outer commitments** Check $\vec{\mathbf{u}}_1 = \sum_{i=1}^r \sum_{k=0}^{t_1-1} \mathbf{B}_{ik} \vec{\mathbf{v}}_i^{(k)} + \sum_{i \leq j} \sum_{k=0}^{t_2-1} \mathbf{C}_{ijk} \mathbf{g}_{ij}^{(k)}$ and $\vec{\mathbf{u}}_2 = \sum_{i \leq j} \sum_{k=0}^{t_1-1} \mathbf{D}_{ijk} \mathbf{h}_{ij}^{(k)}$

C From SNARKs to Aggregate Signatures

In this section, we show how to generically construct an aggregate signature from a hash-then-sign signature and a non-interactive argument system.

C.1 Hash-then-Sign Signatures

We define the class of hash-then-sign signature schemes. It is easy to see that Falcon [PFH⁺22] falls under the category.

Definition C.1 (HtS). A signature scheme $S = (\text{Setup}, \text{Gen}, \text{Sign}, \text{Ver})$ for a message space M is said to be in the class of (randomized) hash-then-sign HtS with the random oracle $G : \{0, 1\}^* \rightarrow \text{Ra}$ if each algorithm proceeds as follows.

$\text{Setup}(1^\lambda)$: Outputs pp that defines a family of preimage sampleable functions (PSF) $\mathcal{F} = \{F_k : \text{Do} \rightarrow \text{Ra}\}_k$.

$\text{Gen}(\text{pp})$: Outputs a key pair (pk, sk) which defines a PSF $F_{\text{pk}} \in \mathcal{F}$. The pk allows computing $y = F_{\text{pk}}(x)$ for $x \in \text{Do}$, while sk allows sampling a preimage x from some distribution $\mathcal{D}$ defined over a set $F_{\text{pk}}^{-1}(y)$ for any $y \in \text{Ra}$. We write $x \leftarrow \text{SampleD}(\text{sk}, y, \mathcal{D})$ to denote sampling x from $\mathcal{D}(F_{\text{pk}}^{-1}(y))$.

$\text{Sign}^G(\text{sk}, m)$:

1. Sample a uniformly random salt $r \in \{0, 1\}^k$
2. $y = G(r, m)$
3. $x \leftarrow \text{SampleD}(\text{sk}, y, \mathcal{D})$
4. Output $\sigma = (r, x)$

$\text{Ver}^G(\text{pk}, m, \sigma)$:

1. $y = G(r, m)$
2. Output 1 iff $x \in \text{Do}$ and $F_{\text{pk}}(x) = y$

The security of HtS signatures usually relies on the assumption that, without the knowledge of sk , it is hard to sample a preimage x of the right distribution $\mathcal{D}$ over $F_{\text{pk}}^{-1}(y)$ for a uniform random y .

There are also other flavors of HtS. For instance, one can make the signature deterministic by setting $k = 0$ and using pseudo-random coins for SampleD , hence $y = G(m)$ and $\sigma = x$. Alternatively, one can replace a freshly sampled salt r by a time period t . That is, $y = G(t, m)$ and $\sigma = (t, x)$. This time period parameter becomes relevant when aggregation is done in a synchronized model.

C.2 Snarky Aggregate Hash-then-Sign Signatures

Let $S = (\text{Setup}, \text{Gen}, \text{Sign}, \text{Ver}) \in \text{HtS}$ be a hash-then-sign signature scheme for a message space M and let $\Pi = (\mathcal{G}, \mathcal{P}, \mathcal{V})$ be a non-interactive argument system in the ROM for the binary relation $R_{\text{pp}} = \{((\text{pk}_i, y_i), x_i)_{i \in [N]} : \forall i \in [N], F_{\text{pk}_i}(x_i) = y_i \wedge x_i \in \text{Do}\}$. Let H be a random oracle for Π and G be a random oracle for S , respectively. We now construct an aggregate signature scheme $\text{AS} = (\text{Setup}, \text{Gen}, \text{Sign}, \text{Ver}, \text{AggSign}, \text{AggVer})$ as follows:

$\text{Setup}(1^\lambda)$: Run $S.\text{Setup}(1^\lambda) \rightarrow \text{pp}_S$ and $\Pi.\mathcal{G}(1^\lambda) \rightarrow \text{pp}_\Pi$, set $\text{pp} := (\text{pp}_S, \text{pp}_\Pi)$ and output pp .

$\text{Gen}(\text{pp})$: Parse $\text{pp} = (\text{pp}_S, \text{pp}_\Pi)$, run $S.\text{Gen}(\text{pp}_S) \rightarrow (\text{sk}, \text{pk})$, and output (sk, pk) .

$\text{Sign}^G(\text{sk}, m)$: Run $S.\text{Sign}^G(\text{sk}, m) \rightarrow \sigma = (r, x)$ and output σ .

$\text{Ver}^G(\text{pk}, m, \sigma)$: Run $S.\text{Ver}^G(\text{pk}, m, \sigma) \rightarrow b$ and output b .

$\text{AggSign}^{G, H}(\text{pp}, \{\text{pk}_i, m_i, \sigma_i\}_{i \in [N]})$: Parse $\text{pp} = (\text{pp}_S, \text{pp}_\Pi)$ and $\sigma_i = (r_i, x_i)$, compute $y_i = G(r_i, m_i)$ for $i \in [N]$, set $x := \{\text{pk}_i, y_i\}_{i \in [N]}$ and $w := \{x_i\}_{i \in [N]}$, run $\Pi.\mathcal{P}^H(\text{pp}_\Pi, x, w) \rightarrow \pi$, set $\sigma_{\text{agg}} := (\pi, (r_i)_{i \in [N]})$ and output σ_{agg} .

$\text{AggVer}^{\text{G,H}}(\text{pp}, \{\text{pk}_i, m_i\}_{i \in [N]}, \sigma_{\text{agg}})$: Parse $\text{pp} = (\text{pp}_S, \text{pp}_\Pi)$ and $\sigma_{\text{agg}} = (\pi, (r_i)_{i \in [N]})$, compute $y_i = \text{G}(r_i, m_i)$ for $i \in [N]$, set $x := \{\text{pk}_i, y_i\}_{i \in [N]}$ and run $\Pi.\mathcal{V}^{\text{H}}(\text{pp}_\Pi, x, \pi) \rightarrow b$ and output b .

Remark C.1. In theory, the above construction does not satisfy the definition of compactness even if the proof system Π is succinct: σ_{agg} contains N salts individually generated by different signers. We show in Appendix E that for our parameter regimes the size of salts does not significantly impact the size of the aggregate signature.

Lemma C.1. *If S is correct and Π is complete, then AS is correct.*

Proof. Let $w := \{x_i\}_{i \in [N]}$ be honestly generated preimages with respect to $x := \{\text{pk}_i, y_i\}_{i \in [N]}$, where $y_i = \text{G}(r_i, m_i)$ and $\sigma_i = (r_i, x_i)$. By the correctness of S and the union bound it holds

$$\Pr[(x, w) \notin R] = \Pr[\exists i \in [N]: \text{Ver}^{\text{G}}(\text{pk}_i, m_i, \sigma_i) = 0] \leq N \cdot \text{negl}(\lambda).$$

Now, by completeness of Π it holds

$$\begin{aligned} & \Pr[\text{AggVer}^{\text{G,H}}(\{\text{pk}_i, m_i\}_{i \in [N]}, \sigma_{\text{agg}}) = 0] \\ &= \Pr[\Pi.\text{Ver}^{\text{H}}(\text{pp}_\Pi, x, \pi) = 0] \\ &= \Pr[\Pi.\text{Ver}^{\text{H}}(\text{pp}_\Pi, x, \pi) = 0 \wedge (x, w) \in R] + \Pr[\Pi.\text{Ver}^{\text{H}}(\text{pp}_\Pi, x, \pi) = 0 \wedge (x, w) \notin R] \\ &\leq \Pr[\Pi.\text{Ver}^{\text{H}}(\text{pp}_\Pi, x, \pi) = 0 \wedge (x, w) \in R] + \Pr[(x, w) \notin R] \\ &\leq \Pr[\Pi.\text{Ver}^{\text{H}}(\text{pp}_\Pi, x, \pi) = 0 | (x, w) \in R] + \Pr[(x, w) \notin R] \\ &\leq \text{negl}(\lambda) + N \cdot \text{negl}(\lambda). \end{aligned}$$

Hence, for all $\lambda, N \in \mathbb{N}$ it yields

$$\Pr[\text{AggVer}^{\text{G,H}}(\{\text{pk}_i, m_i\}_{i \in [N]}, \sigma_{\text{agg}}) = 1] = 1 - \text{negl}(\lambda).$$

□

Lemma C.2. *If S is EU-CMA^+ secure, $k \in \text{poly}(\lambda)$, and Π is $\mathcal{Z}$ -auxiliary input knowledge sound, then AS is EU-ACK secure in the random oracle model, where $\mathcal{Z}(1^\lambda)$ proceeds as follows: run $\text{pp}_S \leftarrow \text{S.Setup}(1^\lambda)$, $(\text{sk}, \text{pk}) \leftarrow \text{S.Gen}(\text{pp}_S)$, $y_i \xleftarrow{\$} \text{Ra}$, $x_i \leftarrow \text{SampleD}(\text{sk}, y_i, \mathcal{D})$ for $i = 1, \dots, Q_s$, $\bar{y}_i \xleftarrow{\$} \text{Ra}$ for $i = 1, \dots, Q_g$ and output $(\text{pp}_S, \text{pk}, x_1, y_1, \dots, x_{Q_s}, y_{Q_s}, \bar{y}_1, \dots, \bar{y}_{Q_g})$, where Q_s is the number of signing queries and Q_g is the number queries to the random oracle G , made by an EU-ACK adversary respectively. Let Q_h be the number of queries to the random oracle H .*

Proof. We show that if there exists an adversary $\mathcal{A}$ breaking the EU-ACK security of AS with non-negligible probability, while assuming that Π has a negligible knowledge error κ , then we can build an adversary $\mathcal{B}$, who uses the extractor of Π , to break the EU-CMA^+ security of S . The proof strategy below closely follows Theorem 4 of [FN16] adapted to the setting of aggregate signatures.

Our reduction $\mathcal{B}$ essentially proceeds in three steps. First, upon receiving the challenge public key pk from the EU-CMA^+ security game, the reduction $\mathcal{B}$ queries OSign with a fixed message m to obtain a sequence of signatures and then queries G with another fixed message $\bar{m} \neq m$ and distinct salts $\bar{r}_i$ as inputs to obtain hashes $\bar{y}_i$, respectively. Second, $\mathcal{B}$ defines a cheating SNARK prover $\mathcal{P}$ which internally runs $\mathcal{A}$ and simulates its view in the EU-ACK game using signatures and random hash outputs as auxiliary information. Finally, $\mathcal{B}$ runs a knowledge extractor $\mathcal{E}$ against $\mathcal{P}$ to get signatures corresponding to the statement output by $\mathcal{P}$. If $\mathcal{A}$ wins the EU-ACK game, at least one of the signatures contains x^* such that it satisfies $F_{\text{pk}}(x^*) = \bar{y}_i = \text{G}(\bar{r}_i, \bar{m})$ for some i and the challenge public key pk . Such x^* indeed qualifies as a forgery breaking the EU-CMA^+ security of S .

In more detail, the reduction $\mathcal{B}$, upon receiving (pp_S, pk) , first obtains $\bar{y}_i$ for $i = 1, \dots, Q_g$ from G by querying G with some $(\bar{r}_i, \bar{m}) \in \{0, 1\}^k \times M$, where $\bar{r}_i$ are distinct. Then $\mathcal{B}$ picks $m \neq \bar{m}$ and queries OSign Q_s times with m to retrieve signatures $(r_i, x_i)_{i \in [Q_s]}$ such that $\text{G}(r_i, m) = y_i$, $F_{\text{pk}}(x_i) = y_i$ and $x_i \in \text{Do}$. $\mathcal{B}$ prepares pp_Π for the proof system by running $\text{pp}_\Pi \leftarrow \Pi.\mathcal{G}(1^\lambda)$.

Next, consider a cheating prover $\mathcal{P}(\text{pp}_\Pi, \text{aux-in})$ given access to the random oracle H (whose responses are simulated by $\mathcal{B}$), where $\text{aux-in} = (\text{pp}_S, \text{pk}, x_1, y_1, \dots, x_{Q_s}, y_{Q_s}, \bar{y}_1, \dots, \bar{y}_{Q_g})$. The prover $\mathcal{P}$ proceeds as follows to simulate the view of $\mathcal{A}$ playing the EU-ACK game, in which the random oracles are denoted by G' and H' , and the signing oracle is denoted by OSign' , respectively.

1. Run $\mathcal{A}$ on input $((\text{pp}_\Pi, \text{pp}_S), \text{pk})$.
2. Upon receiving a query to H' from $\mathcal{A}$, relay that query to H and forward the response from H to $\mathcal{A}$.
3. Upon receiving an i th query with message m'_i to OSign' from $\mathcal{A}$, sample uniform $r'_i \in \{0, 1\}^k$, and abort if the response for $G'(r'_i, m'_i)$ is already defined. Else, consume (x_i, y_i) in aux-in to program $G'(r'_i, m'_i) := y_i$ and return (r'_i, x_i) as a signature.
4. Upon receiving an i th fresh query $(\bar{r}'_i, \bar{m}'_i)$ to G' from $\mathcal{A}$, consume $\bar{y}_i$ in aux-in to program $G'(\bar{r}'_i, \bar{m}'_i) := \bar{y}_i$ and return $\bar{y}_i$ to $\mathcal{A}$.
5. Upon receiving a forgery $(\text{pk}_i^*, m_i^*)_{i \in [N]}$ and $(\pi, (r_i^*)_{i \in [N]})$, let $y_i^* = G'(r_i^*, m_i^*)$ for $i \in [N]$ and output $x = (\text{pk}_i^*, y_i^*)_{i \in [N]}$ and π .

The probability that $\mathcal{P}$ aborts at Step 3 is at most $Q_g(Q_g + Q_s)/2^k$, which is negligible in λ . Unless programming fails, $\mathcal{P}$ perfectly simulates the EU-ACK game for $\mathcal{A}$. The prover $\mathcal{P}$ therefore outputs an accepting proof except with a negligible loss, that is,

$$\text{Adv}_{\text{AS}}^{\text{EU-ACK}}(\mathcal{A}) \leq \Pr[\mathcal{V}^{\text{H}}(\text{pp}_\Pi, x, \pi) = 1 : (x, \pi) \leftarrow \mathcal{P}^{\text{H}}(\text{pp}_\Pi, \text{aux-in})] + \text{negl}(\lambda),$$

where pp_Π and aux-in are as defined by $\mathcal{B}$. Moreover, if $\mathcal{A}$ breaks EU-ACK there exists some i such that $\text{pk}_i^* = \text{pk}$ and m_i^* was never queried to OSign' , implying that $y_i^* = G'(r_i^*, m_i^*) = \bar{y}_j$ for some $j \in [Q_g]$. Recall $\mathcal{B}$ obtained $\bar{y}_j$ by querying $G(\bar{r}_j, \bar{m})$ in the EU-CMA⁺ game, where $\bar{m}$ was never queried to OSign . Our goal is to extract x^* such that $F_{\text{pk}}(x^*) = y_i^*$ and $x^* \in \text{Do}$, which $\mathcal{B}$ can output together with $\bar{r}_j$ and $\bar{m}_j$ to break EU-CMA⁺. By $\mathcal{Z}$ -auxiliary input knowledge soundness, there exists an extractor $\mathcal{E}$ that computes a valid witness for the same distribution of statements as the successful cheating prover $\mathcal{P}$ except with a negligible knowledge error, that is,

$$\begin{aligned} & \Pr \left[\mathcal{V}^{\text{H}}(\text{pp}_\Pi, x, \pi) = 1 \mid \begin{array}{l} \text{pp} \leftarrow \mathcal{G}(1^\lambda), \text{aux-in} \leftarrow \mathcal{Z}(1^\lambda), \\ (x, \pi) \leftarrow \mathcal{P}^{\text{H}}(\text{pp}_\Pi, \text{aux-in}) \end{array} \right] \\ & \leq \Pr \left[(x, w) \in R_{\text{pp}} \mid \begin{array}{l} \text{pp} \leftarrow \mathcal{G}(1^\lambda), \text{aux-in} \leftarrow \mathcal{Z}(1^\lambda), \\ (x, \pi, \text{aux-out}, v, w) \leftarrow \mathcal{E}^{\mathcal{P}^*}(\text{pp}_\Pi, \text{aux-in})(n, \text{pp}_\Pi, \text{aux-in}) \end{array} \right] + \kappa(n, Q_h), \end{aligned}$$

where in particular w contains x^* such that $F_{\text{pk}}(x^*) = y_i^*$ and $x^* \in \text{Do}$ if the relation is satisfied. Here we rely on $\{(x, \pi, \text{aux-out}, v) : (x, \pi, \text{aux-out}) \leftarrow \mathcal{P}^{\text{H}}(\text{pp}_\Pi, \text{aux-in}) \wedge v \leftarrow \mathcal{V}^{\text{H}}(\text{pp}_\Pi, x, \pi)\}$ and $\{(x, \pi, \text{aux}, v) : (x, \pi, \text{aux}, v, w) \leftarrow \mathcal{E}^{\mathcal{P}^*}(\text{pp}_\Pi, \text{aux-in})(n, \text{pp}_\Pi, \text{aux-in})\}$ being identically distributed, ensuring that we are extracting for the same distribution of statements as the $\mathcal{P}^*$ in EU-ACK proves for. Overall, if $\mathcal{A}$ wins the EU-ACK game, then $\mathcal{B}$ also outputs a signature $(\bar{r}_j, x^*)$ on $\bar{m}$ that verifies w.r.t. pk and G except with a negligible loss. Note that $\mathcal{E}$ is only running in *expected* polynomial time, thus $\mathcal{B}$ only breaks EU-CMA⁺ security, not EU-CMA security. $\square$

Deterministic Case. For completeness, we also provide an alternative reduction in case the underlying signature scheme is deterministic i.e. $k = 0$ and $\text{S.Sign}^G(\text{sk}, m)$ outputs a unique signature $\sigma = x$ for fixed input (sk, m) ¹³, which is not covered by the result of [FN16]. Unlike the previous proof, the deterministic case requires a guessing argument of [Cor02] incurring a security loss proportional to the number of signing queries. Our concrete instantiation in later section does not invoke this result, since the standardized Falcon signature does include a sufficiently long random salt. However, the result may be of independent interest if one attempts to construct an aggregate signature scheme from deterministic HtS for the sake of asymptotic compactness.

By slightly modifying the proof below, one can replace the salt by a time period and aggregate the signature in a synchronized setting [GR06].

Lemma C.3. *If the length of salt $k = 0$, S.Sign is deterministic, the message space satisfies $|M| > Q_s + 2Q_g$ and S is EU-CMA⁺ secure against an adversary that queries Q_g queries to the random oracle G and $Q_s + Q_g$ queries to the signing oracle, and Π is $\mathcal{Z}$ -auxiliary input knowledge sound, then AS is EU-ACK secure in the random oracle model against an adversary that makes Q_g queries to the random oracle and Q_s queries to the signing oracle, where $\mathcal{Z}(1^\lambda)$ proceeds as follows: run $\text{pp}_S \leftarrow \text{S.Setup}(1^\lambda)$, $(\text{sk}, \text{pk}) \leftarrow \text{S.Gen}(\text{pp}_S)$, $y_i \xleftarrow{\$} \text{Ra}$, $x_i \leftarrow \mathcal{D}(F_{\text{pk}}^{-1}(y_i))$ for $i = 1, \dots, Q_s + Q_g$, $\bar{y}_i \xleftarrow{\$} \text{Ra}$ for $i = 1, \dots, Q_g$ and output $(\text{pp}_S, \text{pk}, x_1, y_1, \dots, x_{Q_s+Q_g}, y_{Q_s+Q_g}, \bar{y}_1, \dots, \bar{y}_{Q_g})$. The reduction incurs an additional $O(Q_s)$ multiplicative loss in the advantage of breaking EU-CMA⁺ security.*

¹³ In practice, this can be realized by making the Sign algorithm stateful, or by deriving random coins ρ for SampleD deterministically using PRF i.e. $\rho = \text{PRF}_K(m)$ where a secret key K is stored as part of sk .

Proof. The proof is analogous to [Lemma C.2](#), but we change the way queries to G' and OSign' are answered by $\mathcal{P}$. This is because the sign oracle cannot program the random oracle on the fly anymore due to a lack of random salt. To remedy the situation, we adapt the probabilistic random oracle response routine of [\[Cor02\]](#).

The reduction $\mathcal{B}$ playing the EU-CMA^+ game, upon receiving (pp_S, pk) , first obtains $\bar{y}_i$ for $i = 1, \dots, Q_g$ from G by querying G with distinct messages $\bar{m}_i \in M$. Then $\mathcal{B}$ picks distinct m_i for $i = 1, \dots, Q_s + Q_g$ (all of which are also different from $\bar{m}_i$) and queries OSign $Q_s + Q_g$ times with m_i to retrieve signatures $(x_i)_{i \in [Q_s + Q_g]}$ such that $G(m_i) = y_i$, $F_{\text{pk}}(x_i) = y_i$ and $x_i \in \text{Do}$. $\mathcal{B}$ prepares pp_Π for the proof system by running $\text{pp}_\Pi \leftarrow \Pi.\mathcal{G}(1^\lambda)$ and defines the auxiliary input $\text{aux-in} = (\text{pp}_S, \text{pk}, x_1, y_1, \dots, x_{Q_s + Q_g}, y_{Q_s + Q_g}, \bar{y}_1, \dots, \bar{y}_{Q_g})$.

The reduction $\mathcal{B}$ then constructs the following cheating prover $\mathcal{P}$ that simulates the view of $\mathcal{A}$ playing the EU-ACK game.

1. Initialize $j = k = 0$ and an empty table T .
2. Run $\mathcal{A}$ on input $((\text{pp}_\Pi, \text{pp}_S), \text{pk})$.
3. Upon receiving a query to H' from $\mathcal{A}$, relay that query to H and forward the response from H to $\mathcal{A}$.
4. Upon receiving an i th fresh query $\bar{m}'_i$ to G' from $\mathcal{A}$, flip a biased coin that comes out heads with probability p . If the coin comes out head, increment j , consume (x_j, y_j) in aux-in to program the random oracle $G'(\bar{m}'_i) := y_j$, and record $(\bar{m}'_i, x_j, y_j)$ in the table T ; else, increment k and consume $\bar{y}_k$ of aux-in to set $G'(\bar{m}'_i) := \bar{y}_k$.
5. Upon receiving an i th query with message m'_i to OSign' from $\mathcal{A}$, if $G'(m'_i)$ is undefined, increment j , consume (x_j, y_j) in aux-in to program the random oracle $G'(m'_i) := y_j$, record $(\bar{m}'_i, x_j, y_j)$ in the table T , and return x_j as a signature. Else, if $\exists (m'_i, x, y) \in T$ for some (x, y) , return x as a signature, and abort otherwise.
6. Upon receiving a forgery $(\text{pk}_i^*, m_i^*)_{i \in [N]}$ and $\sigma_{\text{agg}} = \pi$, let $y_i^* = G'(m_i^*)$ for $i \in [N]$. Let $i^* \in [N]$ be such that $\text{pk}_{i^*} = \text{pk}$ and $m_{i^*}^*$ was never queried to OSign' . If $\exists (m_{i^*}^*, x, y) \in T$ for some (x, y) , abort; else, output $x = (\text{pk}_{i^*}^*, y_{i^*}^*)_{i \in [N]}$ and π .

Unless it aborts, $\mathcal{P}$ perfectly simulates the EU-ACK game for $\mathcal{A}$. The probability that $\mathcal{P}$ outputs an accepting proof is

$$p^{Q_s} \cdot (1 - p) \cdot \text{Adv}_{\text{AS}}^{\text{EU-ACK}}(\mathcal{A}) \leq \Pr [\mathcal{V}^H(\text{pp}_\Pi, x, \pi) = 1 : (x, \pi) \leftarrow \mathcal{P}^H(\text{pp}_\Pi, \text{aux-in})],$$

because $\mathcal{P}$ does *not* abort if the biased coin comes out heads for all Q_s signing queries *and* the coin comes out tail for a query $G'(m_{i^*}^*)$ used for the forgery. By setting $p = Q_s / (Q_s + 1)$, since $(1 / (1 + 1/Q_s))^{Q_s} \geq 1/e$ for $Q_s \geq 0$, we have that

$$\frac{1}{e \cdot (Q_s + 1)} \cdot \text{Adv}_{\text{AS}}^{\text{EU-ACK}}(\mathcal{A}) \leq \Pr [\mathcal{V}^H(\text{pp}_\Pi, x, \pi) = 1 : (x, \pi) \leftarrow \mathcal{P}^H(\text{pp}_\Pi, \text{aux-in})].$$

The reduction $\mathcal{B}$ then invokes a knowledge extractor $\mathcal{E}$ on $\mathcal{P}$ as in [Lemma C.2](#) and succeeds in outputting a preimage x^* such that $F_{\text{pk}}(x^*) = \bar{y}_k = G(\bar{m}_k)$ for some $k \in [Q_g]$, where $\bar{m}_k \neq m_j$ for any j thanks to the way $\mathcal{B}$ picked messages. Overall, the probability that $\mathcal{B}$ successfully outputs a forgery is at least

$$\frac{1}{e \cdot (Q_s + 1)} \cdot \text{Adv}_{\text{AS}}^{\text{EU-ACK}}(\mathcal{A}) - \kappa(n, Q_h).$$

□

D Omitted Details of Section 3

Proof of [Lemma 3.1](#)

Proof. Recall that a ring element is invertible, if and only if none of its CRT-slots are zero. Thus

$$\begin{aligned}
\Pr_{\mathbf{c} \xleftarrow{\$} \mathcal{C}} [\mathbf{c} - \mathbf{y} \in \mathcal{R}_q^\times] &= \Pr[\mathbf{c} - \mathbf{y} \bmod (X^\delta - \zeta_i) \neq \mathbf{0} \ \forall i \in [l]] \\
&= 1 - \Pr[\exists i \in [l]: \mathbf{c} - \mathbf{y} \bmod (X^\delta - \zeta_i) = \mathbf{0}] \\
&\geq 1 - \sum_{i=1}^l \Pr[\mathbf{c} - \mathbf{y} \bmod (X^\delta - \zeta_i) = \mathbf{0}] \\
&= 1 - \sum_{i=1}^l \Pr[\mathbf{c} \bmod (X^\delta - \zeta_i) = \mathbf{y}] \\
&\geq 1 - l \cdot B,
\end{aligned}$$

where we applied the union bound in the third to fourth line and the property of B -well-spreadness in the last inequality. $\square$

Proof of Lemma 3.2

Proof. Let $\mathcal{C}$ be a tightly B -well-spread challenge space. Then by definition, there exists some $i \in [l]$ and $\mathbf{y} \in \mathbb{Z}_q[X]/\langle X^\delta - \zeta_i \rangle$ such that

$$\Pr_{\mathbf{c} \xleftarrow{\$} \mathcal{C}} [\mathbf{c} \bmod (X^\delta - \zeta_i) = \mathbf{y}] = B.$$

Since $\mathbf{c}$ was sampled uniformly at random, it must be that $B = b/|\mathcal{C}|$ with $b \in \mathbb{Z}_{\geq 0}$. Here b is the number of challenges with i th CRT slot $\mathbf{y}$. The difference of any of these b challenges is $\mathbf{0}$ in the i th CRT slot. An element in $\mathcal{R}_q$ is invertible if and only if all of its CRT slots are non-zero. Thus, we have found our set $\mathcal{S}$ of $b = B|\mathcal{C}|$ challenges. $\square$

Proof of Lemma 3.3

Proof. The proof is by induction over n . We start with the base case, where $n = 1$. Since $\mathbf{f}$ is non-zero, there exists $j \in [l]$, such that $\mathbf{f} \bmod (X^\delta - \zeta_j)$ is non-zero. Further, for any $\mathbf{c} \in \mathcal{R}_q$ it yields

$$\begin{aligned}
\Pr[\mathbf{f}(\mathbf{c}) = \mathbf{0}] &= \Pr[\mathbf{f}(\mathbf{c}) \bmod (X^\delta - \zeta_i) = \mathbf{0} \ \forall i \in [l]] \\
&\leq \Pr[\mathbf{f}(\mathbf{c}) \bmod (X^\delta - \zeta_j) = \mathbf{0}].
\end{aligned}$$

As $\mathbf{f} \bmod (X^\delta - \zeta_j)$ is a polynomial over the field $\mathbb{Z}_q[X]/(X^\delta - \zeta_j)$, it has at most $D := \deg(\mathbf{f} \bmod (X^\delta - \zeta_j))$ distinct roots. Moreover, the degree after reducing modulo $(X^\delta - \zeta_j)$ cannot increase, hence $D \leq \deg(\mathbf{f})$. The probability that $\mathbf{f} \bmod (X^\delta - \zeta_j)$ is zero cannot decrease if we assume that all of its roots are in $\mathcal{C} \bmod X^\delta - \zeta_j$. Moreover, by the well-spreadness of $\mathcal{C}$, the probability that the j -th CRT-slot of a random challenge element $\mathbf{c}$ hits one of those roots is at most B . Thus, by a union bound over the D roots of $\mathbf{f} \bmod (X^\delta - \zeta_j)$, it yields

$$\Pr[\mathbf{f}(\mathbf{c}) = \mathbf{0}] \leq D \cdot B \leq \deg(\mathbf{f}) \cdot B.$$

Now, assume that the lemma holds for all polynomials in $n - 1$ variables and let $\mathbf{f}(X_1, \dots, X_n)$ be a polynomial in n variables. We can interpret $\mathbf{f}$ as a polynomial in X_1 by writing it as

$$\mathbf{f}(X_1, \dots, X_n) = \sum_{i=0}^{\deg(\mathbf{f})} \mathbf{f}_i(X_2, \dots, X_n) \cdot X_1^i,$$

where all the $\mathbf{f}_i$ are polynomials in $n - 1$ variables. As $\mathbf{f}$ is non-zero, there exists an index $j \in \{0, \dots, \deg(\mathbf{f})\}$ such that $\mathbf{f}_j$ is non-zero. Let j be the largest of such indices. Since the degree of $\mathbf{f}_j \cdot X_1^j$ is at most $\deg(\mathbf{f})$, we know that $\mathbf{f}_j$ has degree at most $\deg(\mathbf{f}) - j$. We now sample $\mathbf{c}_2, \dots, \mathbf{c}_n$ independently and uniformly at random from $\mathcal{C}$. By the induction hypothesis, $\Pr[\mathbf{f}_j(\mathbf{c}_2, \dots, \mathbf{c}_n) = \mathbf{0}] \leq (\deg(\mathbf{f}) - j) \cdot B$. Assuming that $\mathbf{f}_j(\mathbf{c}_2, \dots, \mathbf{c}_n) \neq \mathbf{0}$, then $\mathbf{f}(X_1, \mathbf{c}_2, \dots, \mathbf{c}_n)$ is a polynomial in one variable and of degree j . By the induction base case, for $\mathbf{c}_1$ sampled uniformly at random from $\mathcal{C}$, it yields $\Pr[\mathbf{f}(\mathbf{c}_1, \mathbf{c}_2, \dots, \mathbf{c}_n) = \mathbf{0} \mid \mathbf{f}_j(\mathbf{c}_2, \dots, \mathbf{c}_n) \neq \mathbf{0}] \leq j \cdot B$. We denote the event $\mathbf{f}(\mathbf{c}_1, \mathbf{c}_2, \dots, \mathbf{c}_n) = \mathbf{0}$ by E and the event $\mathbf{f}_j(\mathbf{c}_2, \dots, \mathbf{c}_n) = \mathbf{0}$ by F . We observe that

$$\begin{aligned}
\Pr[E] &= \Pr[E \mid F] \cdot \Pr[F] + \Pr[E \mid \neg F] \cdot \Pr[\neg F] \\
&\leq \Pr[F] + \Pr[E \mid \neg F] \\
&\leq (\deg(\mathbf{f}) - j) \cdot B + j \cdot B = \deg(\mathbf{f}) \cdot B,
\end{aligned}$$

concluding the induction. $\square$

E Missing Details on Estimates

Aimed Security Level. All size estimates are targeting a concrete security level λ_{final} by assuming that at most $N \leq 10\,000$ signatures are aggregated, as we believe this is a realistic scenario. By Lemma C.2, assuming λ_{sig} bits of security for the underlying signature scheme and λ_{snark} bits of knowledge error for the used SNARK, we define $\lambda_{\text{final}} = \min\{\lambda_{\text{sig}}, \lambda_{\text{snark}}\}$ bits of security for our aggregate signature scheme. Recall that the signature schemes Falcon-512 and Falcon-1024 provide $\lambda_{\text{sig}} = 128$ and $\lambda_{\text{sig}} = 256$ bits of security, respectively. Equations 4+5 in Theorem 5.2 and Equation 20 in Section I.5 define λ_{snark} . Note that we ignore the security loss caused by the extractor’s runtime in Theorem 5.2 and thus set $Q = 0$. This is a common practice when setting concrete parameters and can be compared to ignoring the security loss caused by the forking lemma, as for instance done in the aggregate signature [FSZ22]. Moreover, for $N \leq 10\,000$, we observe at most $t = 8$ recursions of LaBRADOR. Equations 4+5 are composed of 6 additive terms. Our strategy is to upper bound every single term by $2^{-\lambda}$ for some λ , leading to $2^{-\lambda_{\text{snark}}} \leq 2 \cdot t \cdot 6 \cdot 2^{-\lambda} \leq 2^{-\lambda+7}$. Concretely, we set $\lambda = 128$ for Falcon-512 aggregation and $\lambda = 256$ for Falcon-1024 aggregation. Hence, overall, our aggregate signature scheme for Falcon-512 signatures guarantees a security level of 121 bits, whereas the aggregate signature scheme for Falcon-1024 signatures guarantees 249 bits of security.

The original LaBRADOR protocol aimed at $\lambda = 128$. Increasing λ to 256 (or other values) has an impact on all additive terms of Equations 4+5. First and foremost, the two M-SIS problems need to be parameterized such that their concrete security is λ . Moreover, the challenge space $\mathcal{C}$ needs to be of size at least 2^λ and its well-spreadness bound B should lead to $(5 + 2l)Br \leq 2^{-\lambda}$, where l is the split factor of the subring $\mathcal{S}_{q'}$ and $r = 6\lceil\sqrt{N}\rceil + 1$, an upper bound on the number of witnesses in each iteration. Further, the security level defines the dimensions and constants in the Johnson-Lindenstrauss projections (Lemma 2.2) and defines the dimensions of the aggregation vector $\tilde{\beta}$ in Step 3 of LaBRADOR. If future use cases of LaBRADOR require a different security level than $\lambda \in \{128, 256\}$, all those factors need to be taken into account.

Comparison With Trivial Concatenation. Figure 3 (Left) provides estimates for the proof sizes of our aggregate signature (with salts) for both Falcon parameter regimes and compares it with the trivial solution of concatenating all individual signatures. The parameter regimes Falcon-512 and Falcon-1024 correspond to the parameter sets proposed in the specifications of Falcon, as recalled in Table 4, as well as our choice of challenge sets. We observe that for both parameter regimes starting from ca. 110 signatures, our aggregate signature is shorter than the trivial solution. Whereas both the trivial aggregation and our aggregate signature (due to the salts) grow linearly with the number of signatures, one can clearly see ours provides significantly shorter aggregate signatures.

Effect of the Salt. As the salt of every individual Falcon signature is included in the final aggregate signature (cf. the construction in Section C.2), the size of the aggregate signature is linear in the number N . Thus, our scheme is not succinct from an asymptotic point of view. However, the salt in Falcon consists of 320 bits, which is only a small part of the complete signature. The rather small effect of the salt is depicted in Figure 3 (Right). Only for rather large numbers of signatures N , the effect is significant.

Challenge Sets. As explained in Section 3, there are different choices for the splitting behavior of the ring $\mathcal{S}$ modulo q' . Overall, we distinguish two settings: two-splitting rings and almost-fully-splitting rings.

When instantiating the challenge set $\mathcal{C}$, one has to take multiple properties into account. First, in the knowledge soundness proof of LaBRADOR, we need $|\mathcal{C}| \geq 2^\lambda \cdot \mu$, where λ is the security parameter and $\mu = (5 + 2l)r$ is defined by the split factor l and an upper bound on the number of witnesses r in each round. Second, the size of each of the l CRT-slots is given by q^δ , where δ is the split ratio. In the knowledge soundness proof of LaBRADOR we require $q^\delta \geq 2^\lambda$ for a aimed security level λ . Third, we need a sufficiently good well-spreadness B . Lastly, the norm bounds impact the size of the aggregate signature. By construction, for every $\mathbf{c} \in \mathcal{C}$, it yields $\|\mathbf{c}\|_1 \leq w \cdot \gamma$, $\|\mathbf{c}\|_\infty \leq \gamma$ and $\|\mathbf{c}\|_2 \leq \sqrt{w}\gamma$. Thus, $T_2 \leq w\gamma^2$. We are also interested in a bound T_{op} on the operator norm. Note that in power-of-two cyclotomic rings it holds $\|\mathbf{r} \cdot \mathbf{s}\|_2 \leq \|\mathbf{r}\|_1 \cdot \|\mathbf{s}\|_2$ for every two ring elements $\mathbf{r}, \mathbf{s} \in \mathcal{R}$. Thus, $\|\mathbf{cs}\|_2 / \|\mathbf{s}\|_2 \leq \|\mathbf{c}\|_1 \leq w \cdot \gamma$, implying that $\|\mathbf{c}\|_{\text{op}} \leq w \cdot \gamma$, thus we can set $T_{\text{op}} \leq w \cdot \gamma$. We use the same approach as in LaBRADOR and assume slightly tighter bounds by rejection sampling on challenge elements. More precisely, we set $T_2 := w\gamma/c$ and $T_{\text{op}} := w\gamma^2/c$ for $c = 2$ if $\lambda = 128$ and $c = 2.5$ if $\lambda = 256$.

In the almost-fully-splitting case, the probability that challenges and challenge differences are invertible depends on the well-spreadness of the challenge space (cf. Lemma 3.1). To derive parameters for the challenges (weight and infinity norm bound), we adapted the (heuristic) SageMath code from [ESZ22]. In order to make the probability of non-invertibility as small as $2^{-\lambda}$, we require a larger weight than in the

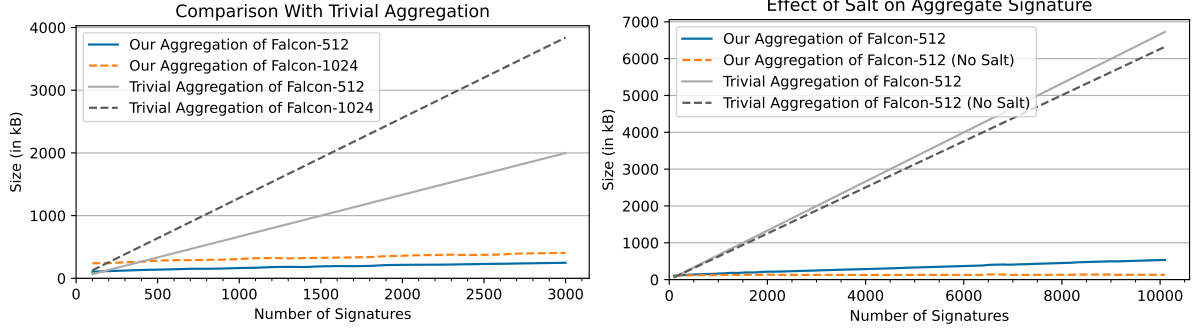


Fig. 3: The size in kB of our aggregate signature compared to the trivial aggregation through concatenation, for both Falcon-512 and Falcon-1024 (Left), and compared to the trivial aggregation through concatenation for Falcon-512, both with and without salt (Right). With salt corresponds to the original Falcon scheme, and without salt corresponds to a deterministic version of Falcon, not proposed for standardization. On the left graph we only show sizes for up to 3000 signatures to allow easier comparisons for small numbers of signatures.

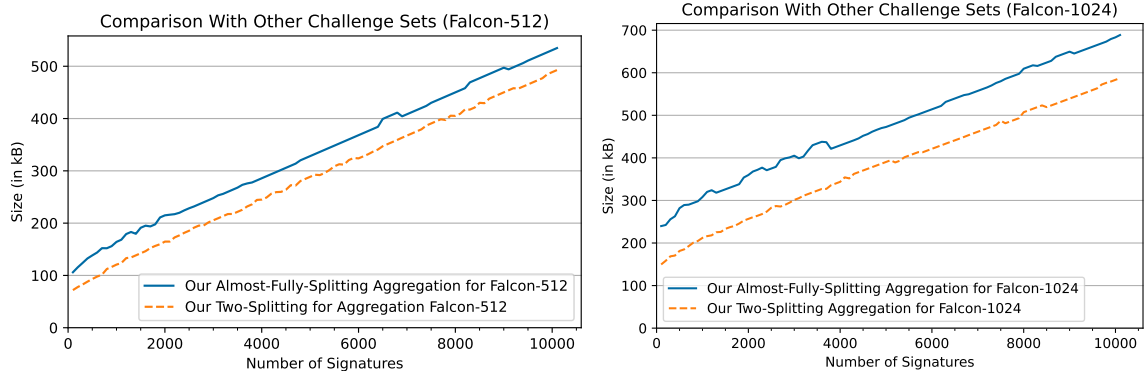


Fig. 4: The size in kB of our aggregate signature scheme compared to the sizes for different choices of challenge sets, for both Falcon-512 and Falcon-1024.

two-splitting case. This explains why the curve of the two-splitting case is slightly below the curve of the almost-fully-splitting case.

Overall, our numbers in Figure 4 show that going from two-splitting to almost-fully-splitting comes at some small costs with respect to estimated proof sizes. If the curious reader is wondering why the lines in Figure 4 are not monotonically increasing, we would like to mention that our global recursion strategy (as explained in Section 6.3) might not be *optimal* for a specific number of signatures N . We think it is possible to obtain a smooth curve if the recursion strategy is locally tailored for a specific N . We leave this up to future work.

F Full Description of Padded Falcon Aggregation Constraints

F.1 Reformulating Constraints for Better Recursion

Padding Scheme. We first present the padding scheme generically. Say that we have the elements $\mathbf{w}_1, \dots, \mathbf{w}_N \in \mathcal{R}_{q'}$ in the old witness, along with the elements $\mathbf{w}'_1, \dots, \mathbf{w}'_N \in \mathcal{R}_{q'}$ such that $\|\mathbf{w}_i\|_2^2 = \text{ct}(\mathbf{w}'_i \mathbf{w}_i)$. Define $\rho = \lfloor \sqrt{N} \rfloor$. In the new witness, we are going to have $\lceil \frac{N}{\rho} \rceil$ vectors $\vec{\mathbf{u}}_1, \dots, \vec{\mathbf{u}}_{\lceil \frac{N}{\rho} \rceil} \in \mathcal{R}_{q'}^N$ containing the $\mathbf{w}_i$, and ρ vectors $\vec{\mathbf{u}}'_1, \dots, \vec{\mathbf{u}}'_\rho \in \mathcal{R}_{q'}^N$ containing the $\mathbf{w}'_i$. We define the vectors such that

$$(\vec{\mathbf{u}}_i)_j = \begin{cases} \mathbf{w}_i & \text{if } (i-1)\rho < j \leq i\rho \\ 0 & \text{else} \end{cases} \quad \text{and} \quad (\vec{\mathbf{u}}'_i)_j = \begin{cases} \mathbf{w}'_j & \text{if } j \equiv i \pmod{\rho} \\ 0 & \text{else} \end{cases}.$$

The padding scheme is best described visually, see Figure 5. To keep track of where the $\mathbf{w}_i, \mathbf{w}'_i$ are stored, we define the index functions

$$\text{index}(i) = \left\lceil \frac{i}{\rho} \right\rceil \quad \text{and} \quad \text{index}'(i) = (i-1 \bmod \rho) + 1.$$

Then $\mathbf{w}_i$ is stored at $(\vec{\mathbf{u}}_{\text{index}(i)})_i$ and $\mathbf{w}'_i$ is stored at $(\vec{\mathbf{u}}'_{\text{index}'(i)})_i$. Importantly, except for $\vec{\mathbf{u}}_{\text{index}(i)}$ and $\vec{\mathbf{u}}'_{\text{index}'(i)}$, the i -th entry of each vector is always $\mathbf{0}$. From this, it follows that

$$\text{ct} \left(\langle \vec{\mathbf{u}}_{\text{index}(i)}, \vec{\mathbf{u}}'_{\text{index}'(i)} \rangle \right) = \text{ct}(\mathbf{w}_i \mathbf{w}'_i) = \|\mathbf{w}_i\|_2^2.$$

Final Witness and Constraints. After applying the padding scheme transformation, we end up with $r = 3\lceil \frac{N}{\rho} \rceil + 3\rho + 1$ witness vectors of rank $n = N$:

- $\vec{\mathbf{y}}_{1,j}, \dots, \vec{\mathbf{y}}_{\lceil \frac{N}{\rho} \rceil, j}$ and $\vec{\mathbf{y}}'_{1,j}, \dots, \vec{\mathbf{y}}'_{\rho, j}$ for $j = 1, 2$: The padding of $\mathbf{s}_{1,j}, \dots, \mathbf{s}_{N,j}$ and $\mathbf{s}'_{1,j}, \dots, \mathbf{s}'_{N,j}$.
- $\vec{\mathbf{e}}_1, \dots, \vec{\mathbf{e}}_{\lceil \frac{N}{\rho} \rceil}$ and $\vec{\mathbf{e}}'_1, \dots, \vec{\mathbf{e}}'_\rho$: The padding of $\boldsymbol{\varepsilon}_1, \dots, \boldsymbol{\varepsilon}_N$ and $\boldsymbol{\varepsilon}'_1, \dots, \boldsymbol{\varepsilon}'_N$.
- $\vec{\mathbf{v}}$: A single vector collecting the $\mathbf{v}_1, \dots, \mathbf{v}_N$.

The old constraints need to be transformed to fit the new witness. Let $\vec{\boldsymbol{\delta}}_i \in \mathcal{R}_{q'}^N$ be the vector that has i -th entry $\mathbf{1}$ and all other entries $\mathbf{0}$. Then we can formulate the i -th Falcon verification equation as

$$\langle \vec{\boldsymbol{\delta}}_i, \vec{\mathbf{y}}_{\text{index}(i),1} \rangle + \langle \mathbf{h}_i \vec{\boldsymbol{\delta}}_i, \vec{\mathbf{y}}_{\text{index}(i),2} \rangle + \langle q \vec{\boldsymbol{\delta}}_i, \vec{\mathbf{v}} \rangle - \mathbf{t}_i = \mathbf{0} \bmod q'.$$

The new i -th four-square constraint is

$$\text{ct} \left(\langle \vec{\mathbf{y}}_{\text{index}(i),1}, \vec{\mathbf{y}}'_{\text{index}'(i),1} \rangle + \langle \vec{\mathbf{y}}_{\text{index}(i),2}, \vec{\mathbf{y}}'_{\text{index}'(i),2} \rangle + \langle \vec{\mathbf{e}}_{\text{index}(i)}, \vec{\mathbf{e}}'_{\text{index}'(i)} \rangle - \beta^2 \right) = 0 \bmod q'.$$

The dot product constraints for the $\mathbf{s}'_{i,1}, \mathbf{s}'_{i,2}, \boldsymbol{\varepsilon}_i, \boldsymbol{\varepsilon}'_i$ are all of the form (9), checking that a single $\mathbb{Z}_{q'}$ -coefficient of a witness element is equal to a constant or that it is equal to another coefficient in the witness. To check that j -th $\mathbb{Z}_{q'}$ -coefficient of the i -th entry of some witness vector $\vec{\mathbf{a}} \in \mathcal{R}_{q'}^N$ is equal to some constant $b \in \mathbb{Z}_{q'}$, or to check that it is equal to the j' -th coefficient of the i' -th entry of some other witness vector $\vec{\mathbf{c}} \in \mathcal{R}_{q'}^N$, we add a constraint of the form

$$\text{ct} \left(\langle \sigma_{-1}(X^j) \vec{\boldsymbol{\delta}}_i, \vec{\mathbf{a}} \rangle - b \right) = 0 \bmod q' \quad \text{or} \quad \text{ct} \left(\langle \sigma_{-1}(X^j) \vec{\boldsymbol{\delta}}_i, \vec{\mathbf{a}} \rangle - \langle \sigma_{-1}(X^{j'}) \vec{\boldsymbol{\delta}}_{i'}, \vec{\mathbf{c}} \rangle \right) = 0 \bmod q'.$$

We also need to add constraints to enforce that the other entries of the witness are 0 in accordance with the padding scheme. To check that the i -th entry of some witness vector $\vec{\mathbf{a}} \in \mathcal{R}_{q'}^N$ is $\mathbf{0}$, we add the constraint

$$\langle \vec{\boldsymbol{\delta}}_i, \vec{\mathbf{a}} \rangle = \mathbf{0} \bmod q'.$$

Finally, for the projections in the first iteration, nothing really changes. We only need to project the entries that are not 0 in the padding scheme, because we check that the other entries are indeed 0 and contribute nothing to the norm of the witness vectors. Hence, $\vec{p}_1$ and $\vec{p}_2$ are random linear projections of the same elements as before, meaning that we can keep the the bounds from the previous section.

Impact on Runtime and Proof Size. To see the benefits of the new formulation, let us first analyze the formulation we had at end of Section 6.2. With $r = O(N)$ witness elements, there are $O(r^2) = O(N^2)$ garbage polynomials in the first iteration, which the prover must compute. With our set of dot product constraints, the runtime of the prover in the k -th iteration is $O(n_k r_k + m_k)$. With $m_1 = O(N)^2$, we get a $O(N^2)$ runtime for the prover. Furthermore, since $n_1 \ll m_1$, the recursion does not start from a balanced state. Recursing after the first iteration, the new rank is $n_2 = \max(\frac{1}{\nu}, \frac{m_1}{\mu}) = \frac{m_1}{\mu}$, where ν and μ are the folding parameters when going from one iteration level to the next. Effectively, this means that recursion with the old formulation starts at rank $O(N^2)$.

F.2 Final constraints before moving to subring

Define $\vec{\boldsymbol{\delta}}_i \in \mathcal{R}_{q'}^n$ to be the vector with $(\vec{\boldsymbol{\delta}}_i)_j = \begin{cases} 1 & \text{if } i = j \\ 0 & \text{else} \end{cases}$.

Main constraints:

Fig. 5: Illustration of the padding scheme when ρ divides N . The columns of the left matrix are $\vec{u}_1, \dots, \vec{u}_{\lceil \frac{N}{\rho} \rceil} \in \mathcal{R}_{q'}^N$, and the columns of the right matrix are $\vec{u}'_1, \dots, \vec{u}'_{\rho} \in \mathcal{R}_{q'}^N$. When ρ does not divide N , the last couple of entries in the pattern is $\mathbf{0}$.

$$\begin{aligned}
 [\vec{u}_1, \dots, \vec{u}_{\lceil \frac{N}{\rho} \rceil}] &= \begin{bmatrix} w_1 & \mathbf{0} & \mathbf{0} & \cdots & \mathbf{0} \\ \vdots & \vdots & \vdots & \cdots & \vdots \\ w_{\rho} & \mathbf{0} & \vdots & \cdots & \vdots \\ \mathbf{0} & w_{\rho+1} & \vdots & \cdots & \vdots \\ \vdots & \vdots & \vdots & \cdots & \vdots \\ \vdots & w_{2\rho} & \mathbf{0} & \cdots & \vdots \\ \vdots & \mathbf{0} & w_{2\rho+1} & \cdots & \vdots \\ \vdots & \vdots & \vdots & \cdots & \vdots \\ \vdots & \vdots & w_{3\rho} & \cdots & \vdots \\ \vdots & \vdots & \mathbf{0} & \cdots & \vdots \\ \vdots & \vdots & \vdots & \cdots & \vdots \\ \vdots & \vdots & \vdots & \cdots & \mathbf{0} \\ \vdots & \vdots & \vdots & \cdots & w_{N-\rho+1} \\ \vdots & \vdots & \vdots & \cdots & \vdots \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \cdots & w_N \end{bmatrix}, \quad [\vec{u}'_1, \dots, \vec{u}'_{\rho}] = \begin{bmatrix} w'_1 & \mathbf{0} & \mathbf{0} & \cdots & \mathbf{0} \\ \mathbf{0} & w'_2 & \mathbf{0} & \cdots & \vdots \\ \vdots & \mathbf{0} & w'_3 & \cdots & \vdots \\ \vdots & \vdots & \mathbf{0} & \cdots & \vdots \\ \vdots & \vdots & \vdots & \cdots & \vdots \\ \vdots & \vdots & \vdots & \cdots & \mathbf{0} \\ \mathbf{0} & \vdots & \vdots & \cdots & w'_{\rho} \\ w'_{\rho+1} & \mathbf{0} & \vdots & \cdots & \mathbf{0} \\ \mathbf{0} & w'_{\rho+2} & \mathbf{0} & \cdots & \vdots \\ \vdots & \mathbf{0} & w'_{\rho+3} & \cdots & \vdots \\ \vdots & \vdots & \mathbf{0} & \cdots & \vdots \\ \vdots & \vdots & \vdots & \cdots & \vdots \\ \vdots & \vdots & \vdots & \cdots & \mathbf{0} \\ \mathbf{0} & \vdots & \vdots & \cdots & w'_{2\rho} \\ \vdots & \vdots & \vdots & \cdots & \vdots \\ w'_{N-\rho+1} & \mathbf{0} & \vdots & \cdots & \mathbf{0} \\ \mathbf{0} & w'_{N-\rho+2} & \mathbf{0} & \cdots & \vdots \\ \vdots & \mathbf{0} & w'_{N-\rho+3} & \cdots & \vdots \\ \vdots & \vdots & \mathbf{0} & \cdots & \vdots \\ \vdots & \vdots & \vdots & \cdots & \vdots \\ \vdots & \vdots & \vdots & \cdots & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \cdots & w'_N \end{bmatrix}
 \end{aligned}$$

For $i = 1$ to n :

$$\langle \vec{\delta}_i, \vec{y}_{\text{index}(i),1} \rangle + \langle \mathbf{h}_i \vec{\delta}_i, \vec{y}_{\text{index}(i),2} \rangle + \langle q \vec{\delta}_i, \vec{v} \rangle - H(m_i) = \mathbf{0} \bmod q' \quad (\text{Falcon eq})$$

$$\text{ct} \left(\left(\sum_{j=1}^2 \langle \vec{y}_{\text{index}(i),j}, \vec{y}'_{\text{index}'(i),j} \rangle \right) + \langle \vec{e}_{\text{index}(i)}, \vec{e}'_{\text{index}'(i)} \rangle - \beta^2 \right) = 0 \bmod q' \quad (4 \text{ squares})$$

$$\begin{aligned}
 \|(\vec{y}_{1,1}, \dots, \vec{y}_{\lceil \frac{n}{\rho} \rceil, 2}, \vec{y}'_{1,1}, \dots, \vec{y}'_{\lceil \frac{n}{\rho} \rceil, 2}, \vec{e}_1, \dots, \vec{e}_{\rho}, \vec{e}'_1, \dots, \vec{e}'_{\rho})\|_{\infty} &\leq \sqrt{\frac{q'}{2(2d+4)}} \\
 \|\vec{v}\|_{\infty} &\leq \frac{q'}{6q}
 \end{aligned}$$

Form constraints

Form constraints for the $\vec{y}_{i,j}$: We check that there has been padded with 0s correctly.

For $i = 1$ to $\lceil \frac{n}{\rho} \rceil$:

For $j = 1$ to 2 :

For $k = 1$ to n , $k \notin [(i-1)\rho + 1, i\rho]$:

$$\langle \vec{\delta}_k, \vec{y}_{i,j} \rangle = \mathbf{0} \bmod q'$$

Form constraints for the $\vec{y}'_{i,j}$:

For $i = 1$ to ρ :

For $j = 1$ to 2:

For $k = 1$ to n :

If $k \equiv i \pmod{\rho}$: We check that $\sigma_{-1}((\vec{y}_{i,j})_k) = (\vec{y}'_{i,j})_k$.

$$\text{ct}\left(\langle \vec{\delta}_k, \vec{y}_{i,j} \rangle + \langle -\vec{\delta}_k, \vec{y}'_{i,j} \rangle\right) = 0 \pmod{q'}$$

For $l = 1$ to $d - 1$:

$$\text{ct}\left(\langle \sigma_{-1}(X^l)\vec{\delta}_k, \vec{y}_{i,j} \rangle + \langle \sigma_{-1}(X^{d-l})\vec{\delta}_k, \vec{y}'_{i,j} \rangle\right) = 0 \pmod{q'}$$

Else:

$$\langle \vec{\delta}_k, \vec{y}'_{i,j} \rangle = 0 \pmod{q'}$$

Form constraints for the $\vec{e}_i$:

For $i = 1$ to $\lceil \frac{n}{\rho} \rceil$:

For $j = 1$ to n :

If $(i - 1)\rho < j \leq i\rho$:

For $k = 4$ to $d - 1$:

$$\text{ct}\left(\langle \sigma_{-1}(X^k)\vec{\delta}_j, \vec{e}_i \rangle\right) = 0 \pmod{q'}$$

Else:

$$\langle \vec{\delta}_j, \vec{e}_i \rangle = 0 \pmod{q'}$$

Form constraints for the $\vec{e}'_i$:

For $i = 1$ to ρ :

For $j = 1$ to n :

If $j \equiv i \pmod{\rho}$:

$$\text{ct}\left(\langle \vec{\delta}_j, \vec{e}_i \rangle + \langle -\vec{\delta}_j, \vec{e}'_i \rangle\right) = 0 \pmod{q'}$$

For $k = 1$ to 3:

$$\text{ct}\left(\langle \sigma_{-1}(X^k)\vec{\delta}_j, \vec{e}_i \rangle + \langle \sigma_{-1}(X^{d-k})\vec{\delta}_j, \vec{e}'_i \rangle\right) = 0 \pmod{q'}$$

(I'm not checking that the other coefficients are 0, it is enough to check that they are 0 in $\vec{e}_i$)

Else:

$$\langle \vec{\delta}_j, \vec{e}'_i \rangle = 0 \pmod{q'}$$

Projection constraints

There will also be 256 constant term constraints per projection that we compute. We compute two projections, using randomly sampled $\Pi_i \in \{0, \pm 1\}^{256 \times n}$.

$$\begin{aligned} \vec{p}_1 &= \Pi_1 \left(\sum_{i=1}^{\lceil \frac{n}{\rho} \rceil} \vec{y}_{i,1} \right) + \Pi_2 \left(\sum_{i=1}^{\lceil \frac{n}{\rho} \rceil} \vec{y}_{i,2} \right) + \Pi_3 \left(\sum_{i=1}^{\rho} \vec{y}'_{i,1} \right) + \Pi_4 \left(\sum_{i=1}^{\rho} \vec{y}'_{i,2} \right) + \Pi_5 \left(\sum_{i=1}^{\lceil \frac{n}{\rho} \rceil} \vec{e}_i \right) + \Pi_6 \left(\sum_{i=1}^{\rho} \vec{e}'_i \right) \\ \vec{p}_2 &= \Pi_7 \vec{v} \end{aligned}$$

Let $\vec{\pi}_i^{(j)}$ be the j -th row of Π_i . Then we will have the following projection constraints:

For $j = 1$ to 256:

$$\begin{aligned} &\text{ct}\left(\sum_{i=1}^{\lceil \frac{n}{\rho} \rceil} \langle \sigma_{-1}(\vec{\pi}_1^{(j)}), \vec{y}_{i,1} \rangle + \sum_{i=1}^{\lceil \frac{n}{\rho} \rceil} \langle \sigma_{-1}(\vec{\pi}_2^{(j)}), \vec{y}_{i,2} \rangle + \sum_{i=1}^{\rho} \langle \sigma_{-1}(\vec{\pi}_3^{(j)}), \vec{y}'_{i,1} \rangle + \sum_{i=1}^{\rho} \langle \sigma_{-1}(\vec{\pi}_4^{(j)}), \vec{y}'_{i,2} \rangle \right. \\ &\quad \left. + \sum_{i=1}^{\lceil \frac{n}{\rho} \rceil} \langle \sigma_{-1}(\vec{\pi}_5^{(j)}), \vec{e}_i \rangle + \sum_{i=1}^{\rho} \langle \sigma_{-1}(\vec{\pi}_6^{(j)}), \vec{e}'_i \rangle - (\vec{p}_1)_j \right) = 0 \pmod{q'} \\ &\text{ct}\left(\langle \sigma_{-1}(\vec{\pi}_7^{(j)}), \vec{v} \rangle - (\vec{p}_2)_j \right) = 0 \pmod{q'} \end{aligned}$$

The total number of dot product constraints

In total, this gives

$$|\mathcal{F}| = n + 3\left(\left\lceil \frac{n}{\rho} \right\rceil - 1\right)n + 3(\rho - 1)n \approx 6\sqrt{nn}$$

$$|\mathcal{F}'| = n + dn + (d - 4)n + 4n + 512 = (2d + 1)n + 512$$

G Coordinate-Wise PSS

Several lattice-based commit-and-open protocols, including LaBRADOR, only reveal an amortized opening to the verifier. With r witness vectors $\vec{w}_1, \dots, \vec{w}_r$ and some base challenge space S , the verifier sends $(\alpha_1, \dots, \alpha_r) \in S^r$ and the prover should respond with $\vec{z} = \sum_{i=1}^r \alpha_i \vec{w}_i$. The verifier checks that $\vec{z}$ is a short opening to the amortized commitment. Typically, to extract $\vec{w}_i$, the idea is to obtain two accepting transcripts with amortization challenges that only differ in the i -th coordinate, so that $\alpha_i \neq \alpha'_i$ but $\alpha_j = \alpha'_j$ for all $j \neq i$. If the challenge difference $\alpha_i - \alpha'_i$ is invertible, then a weak opening for the i -th commitment can be computed as $\vec{w}_i = (\alpha_i - \alpha'_i)^{-1}(\vec{z} - \vec{z}')$. Hence, there is a 2-special-sound structure for each of the r coordinates. This is captured by the notion of coordinate-wise special-soundness (CWSS) introduced in [FMN23]. For a recap on CWSS see Appendix B.5.

In this section, we analogously generalize predicate special soundness to coordinate-wise predicate special soundness. The new definitions are merely extensions of the previous ones, setting $r_1 = r_2 = \dots = r_\mu = 1$ yields regular predicate special soundness. We begin by generalizing our notation for trees of transcripts.

In the coordinate-wise setting, it is hard to define the notation for the predicates without having multiple levels of indexing. To improve readability, we use the convention that $\vec{s}$ denotes vectors over S_m and that $\vec{c}$ denotes vectors over $\mathcal{C}_m = S_m^{r_m}$. Thus, the entries of $\vec{c}$ are themselves vectors $\vec{s}_1, \dots, \vec{s}_n$. Observe that for all $(\vec{s}_1, \vec{c}_2, \dots, \vec{c}_\ell) \in \mathcal{C}_m^{(\ell)}$, $\{\vec{s}_1, \vec{s}_{2,1} \dots \vec{s}_{\ell, r_m}\} \in \text{SS}(S_m, r_m, \ell)$.

Definition G.1 (Coordinate-wise tree of transcripts). *Let $\mu, k_1, \dots, k_\mu, r_1, \dots, r_\mu \in \mathbb{N}_{\geq 0}$ and let $\Pi = (\mathcal{P}, \mathcal{V})$ be a $2\mu + 1$ -message public-coin argument of knowledge for a relation R_{pp} such that m -th challenge set is $\mathcal{C}_m = S_m^{r_m}$ for some set S_m . Additionally, let $m \in [\mu]$ and $\ell \in [k]$.*

- We define $\mathbb{T}_{\mu+1}$ to be the set of possible accepting transcripts for Π .
- We define $\mathbb{T}_{m+1}^{(\ell)}$ to be the set of possible accepting $(r_1, \dots, r_\mu)$ -coordinate-wise $(1, \dots, 1, \ell, k_{m+1}, k_\mu)$ -trees of transcripts, and denote $\mathbb{T}_m = \mathbb{T}_{m+1}^{(k_m)}$. We arrange $t \in \mathbb{T}_{m+1}^{(\ell)}$ such that $t = (t_1, \vec{t}_2, \dots, \vec{t}_\ell)$, with $t_1 \in \mathbb{T}_{m+1}$ and $\vec{t}_i \in \mathbb{T}_m^{r_m}$ for $i > 1$. For any $i \in [r_m]$, the m -th challenges of $t_1, t_{2,i}, \dots, t_{\ell,i}$ in $S_m^{r_m}$ are pairwise distinct in the i -th coordinate and equal in the other coordinates.
- For $t \in \mathbb{T}_{m+1}$, we define $\text{trunk}(t)$ to be the prefix $(a_1, c_1, a_2, c_2, \dots, a_m)$ shared by all the transcripts in t , $\text{chal}_m(t)$ to be their shared m -th challenge $\vec{s} \in S_m^{r_m}$, and $\text{chal}_{m,i}(t) = \vec{s}_i$ for $i \in [r_m]$. Furthermore, for $\vec{t} \in \mathbb{T}_m^{r_m}$ we define $\text{chal}_m(\vec{t})$ to be the vector $\vec{c} = (\vec{s}_1, \dots, \vec{s}_n) \in \mathcal{C}_m^n$ with $\vec{s}_i = \text{chal}_m(t_i)$.
- Finally, we let $\mathcal{C}_m^{(\ell)}$ denote the set of tuples $(\vec{s}_1, \vec{c}_2, \dots, \vec{c}_\ell)$ with $\vec{s}_1 \in \mathcal{C}_m$ and $\vec{c}_i = (\vec{s}_{i,1}, \dots, \vec{s}_{i,r_m}) \in \mathcal{C}_m^{r_m}$ such that $\vec{s}_1, \vec{s}_{2,i}, \dots, \vec{s}_{\ell,i}$ are pairwise distinct in the i -th coordinate and equal in the other coordinates. This is the set of all the tuples that may occur as $(\text{chal}_m(t_1), \text{chal}_m(\vec{t}_2), \dots, \text{chal}_m(\vec{t}_\ell))$ for some $(t_1, \vec{t}_2, \dots, \vec{t}_\ell) \in \mathbb{T}_{m+1}^{(\ell)}$.

For a $(t_1, \vec{t}_2, \dots, \vec{t}_\ell) \in \mathbb{T}_{m+1}^{(\ell)}$ and a coordinate $i \in [r_m]$, we refer to t_1 as the first tree for the i -th coordinate, $t_{2,i}$ as the second tree for that coordinate, $t_{3,i}$ the third, etc. Likewise, for a $(\vec{s}_1, \vec{c}_2, \dots, \vec{c}_\ell) \in \mathcal{C}_m^{(\ell)}$, we refer to $\vec{s}_1$ as the first challenge vector for the i -th coordinate, $\vec{s}_{2,i}$ the second, etc. Furthermore, we say that $s_{1,i}$ is the first challenge for the i -th coordinate, $s_{2,i,i}$ is the second, etc. Because of the asymmetry with the indexing, we will sometimes write $s_{1,i,i}$ to denote $s_{1,i}$ and $t_{1,i}$ to denote t_1 .

Definition G.2 (Coordinate-wise predicates). *Let $m \in [\mu]$ and $\ell \in [k_m]$.*

1. A challenge predicate on level m for the ℓ th challenge is a polynomial-time computable function

$$\Phi_{m,\ell}^{\text{chal}} : \mathcal{C}_m^{(\ell-1)} \times [r_m] \times S_m \rightarrow \{0, 1\}.$$

2. A commitment predicate on level m is a pair of polynomial-time computable functions $(\Phi_m^{\text{prop}}, \Phi_m^{\text{bind}})$, where

$$\Phi_m^{\text{prop}} : \mathbb{T}_{m+1}^{(k_m-1)} \rightarrow \{0, 1\} \quad \text{and} \quad \Phi_m^{\text{bind}} : \mathbb{T}_{m+1}^{(k_m-1)} \times [r_m] \times \mathbb{T}_{m+1} \rightarrow \{0, 1\}.$$

The ℓ -th challenge predicate $\Phi_{m,\ell}^{\text{chal}}$ is now evaluated in every coordinate. In regular predicate special soundness, the ℓ -th challenge predicate ensures that the ℓ -th challenge has the right properties with respect to the previous $\ell - 1$ challenges. A natural coordinate-wise generalization would be to evaluate $\Phi_{m,\ell}^{\text{chal}}$ locally in each coordinate i , giving it as input $s_{1,i}, s_{2,i,i}, \dots, s_{\ell,i,i}$ in the i -th coordinate. However, to increase the expressiveness of the framework, we allow the ℓ -th challenge predicate to take as input the first $\ell - 1$ challenge vectors from *every* coordinate. We do the same for the commitment predicates. The property predicate takes as input the $k_m - 1$ first trees for every coordinate, and then the binding predicate is evaluated in each coordinate i with the k_m -th tree $t_{k_m,i}$. Having information across coordinates was required for us to prove LaBRADOR knowledge sound. We move on to defining the validity of the coordinate-wise subtrees.

Definition G.3 (Coordinate-wise predicate system). A predicate system Φ for a $(r_1, \dots, r_\mu)$ -coordinate-wise $(k_1, \dots, k_\mu)$ -tree structure is a collection of predicates for each level in the tree. The m -th level has one commitment predicate $(\Phi_m^{\text{prop}}, \Phi_m^{\text{bind}})$, and k_m challenge predicates $\Phi_{m,1}^{\text{chal}}, \dots, \Phi_{m,k_m}^{\text{chal}}$. We recursively define a series of boolean functions Φ_m for $m \in [\mu + 1]$, describing whether a partial tree of transcripts satisfies the predicate system. For a single accepting transcript $t \in \mathbb{T}_{\mu+1}$ we let $\Phi_{\mu+1}(t) = 1$. For all larger subtrees $t = (t_1, \vec{t}_2, \dots, \vec{t}_{k_m}) \in \mathbb{T}_m$ with $m \in [\mu]$ then $\Phi_m(t) = 1$ if and only if

$$\bigwedge_{\ell \in [k_m]} \bigwedge_{i \in [r_m]} (\Phi_{m+1}(t_{\ell,i}) = 1 \wedge \Phi_{m,\ell}^{\text{chal}}((\vec{s}_1, \vec{c}_2, \dots, \vec{c}_{\ell-1}), i, s_{\ell,i,i}) = 1) \\ \wedge \Phi_m^{\text{prop}}(t_1, \vec{t}_2, \dots, \vec{t}_{k_m-1}) = 1 \wedge \bigwedge_{i \in [r_m]} \Phi_m^{\text{bind}}((t_1, \vec{t}_2, \dots, \vec{t}_{k_m-1}), i, t_{k_m,i}) = 1,$$

where $\vec{s}_1 = \text{chal}_m(t_1)$ and $\vec{c}_j = \text{chal}_m(\vec{t}_j)$ for $j > 1$. For notational convenience, we let $\Phi = \Phi_1$.

The definition of coordinate-wise predicate special soundness follows naturally.

Definition G.4 (Coordinate-Wise Predicate Special Soundness). Let $\Pi = (\mathcal{P}, \mathcal{V})$ be a $2\mu + 1$ -message public-coin argument of knowledge for a relation R_{pp} , with m -th challenge space $\mathcal{C}_m = S_m^{r_m}$. We say that Π is $(\mathbf{R}, \mathbf{K}, \Phi)$ -coordinate-wise predicate-special-sound for $\mathbf{R} = (r_1, \dots, r_\mu)$, $\mathbf{K} = (k_1, \dots, k_\mu)$ and a coordinate-wise predicate system Φ if there exists a polynomial time algorithm which given a statement x and a $\mathbf{R}$ -coordinate-wise $\mathbf{K}$ -tree of transcripts t for this statement with $\Phi(t) = 1$ always outputs a witness w such that $(x, w) \in R_{\text{pp}}$.

To analyze the knowledge soundness of coordinate-wise predicate-special-sound protocols, we define the failure density of our predicates, beginning with the challenge predicates.

Definition G.5 (Failure density of a challenge predicate). Let $m \in [\mu]$ and $\ell \in [k_m]$. Let $(\vec{s}_1, \vec{c}_2, \dots, \vec{c}_{\ell-1}) \in \mathcal{C}_m^{(\ell-1)}$ be any collection of challenges such that the first $\ell - 1$ challenge predicates are satisfied in every coordinate, meaning

$$\forall i \in [\ell - 1], \forall j \in [r_m] : \Phi_{m,i}^{\text{chal}}((\vec{s}_1, \vec{c}_2, \dots, \vec{c}_{i-1}), j, s_{i,j,j}) = 1.$$

For each coordinate $j \in [r_m]$, consider the set of possible ℓ -th challenges such that $\Phi_{m,\ell}^{\text{chal}}$ fails,

$$\text{BadChal}_{m,\ell}(\vec{s}_1, \vec{c}_2, \dots, \vec{c}_{\ell-1}, j) = \left\{ s \in S_m \mid \begin{array}{l} s \notin \{s_{1,j}, s_{2,j,j}, \dots, s_{\ell-1,j,j}\}, \\ \Phi_{m,\ell}^{\text{chal}}((\vec{c}_1, \dots, \vec{c}_{\ell-1}), j, s) = 0 \end{array} \right\}.$$

The challenge predicate $\Phi_{m,\ell}^{\text{chal}}$ has failure density $p_{m,\ell}^{\text{chal}}$ if it always holds that $|\text{BadChal}_{m,\ell}(\vec{s}_1, \vec{c}_2, \dots, \vec{c}_{\ell-1})| \leq p_{m,\ell}^{\text{chal}} |S_m|$.

Note that the failure density of the ℓ -th challenge predicate is defined locally for each coordinate. Even though the predicate takes as input the first $\ell - 1$ challenge vectors for every coordinate, we are counting the number of "bad" ℓ -th challenges for a particular coordinate. The same is true for the failure density of commitment predicates.

Definition G.6 (Failure density of a commitment predicate). Let $m \in [\mu]$. Define a set of bad subtrees

$$\text{PropUnsat}_m = \left\{ (t_1, \vec{t}_2, \dots, \vec{t}_{k_m-1}) \in \mathbb{T}_{m+1}^{(k_m-1)} \mid \begin{array}{l} \forall i \in [k_m - 1], j \in [r_m] : \\ \Phi_{m+1}(t_{i,j}) = 1, \\ \Phi_{m,i}^{\text{chal}}(\vec{s}_1, \vec{c}_2, \dots, \vec{c}_{i-1}, j, s_{i,j,j}) = 1, \\ \Phi_m^{\text{prop}}(t_1, \vec{t}_2, \dots, \vec{t}_{k_m-1}) = 0 \end{array} \right\}$$

using the shorthand $\vec{s}_1 = \text{chal}_m(t_1)$ and $\vec{c}_i = \text{chal}_m(\vec{t}_i)$ for $i > 1$. That is, subtrees in PropUnsat_m fail to satisfy the property predicate but otherwise satisfy the constraints. For each $(t_1, \vec{t}_2, \dots, \vec{t}_{k_m-1}) \in \mathbb{T}_{m+1}^{(k_m-1)}$, we define $\text{BindTree}_m(t_1, \dots, t_{k_m-1})$ to be the set of possible k_m -th subtrees that satisfy the binding predicate and the other constraints, using the shorthand $s = \text{chal}_{m,j}(t)$.

$$\text{BindTree}_m(t_1, \vec{t}_2, \dots, \vec{t}_{k_m-1}, j) = \left\{ t \in \mathbb{T}_{m+1} \mid \begin{array}{l} \forall i \in [k_m - 1] : s \neq s_{i,j}, \\ \text{trunk}(t) = \text{trunk}(t_1), \\ \Phi_{m+1}(t) = 1, \\ \Phi_{m,k_m}^{\text{chal}}((\vec{s}_1, \vec{c}_2, \dots, \vec{c}_{k_m-1}), j, s) = 1 \\ \Phi_m^{\text{bind}}((t_1, \vec{t}_2, \dots, \vec{t}_{k_m-1}), j, t) = 1 \end{array} \right\}$$

Consider the m -th level challenges for the j -th coordinate occuring for some tree in this set,

$$\text{BindChal}_m(t_1, \vec{t}_2, \dots, \vec{t}_{k_m-1}, j) = \{ \text{chal}_{m,j}(t) \mid t \in \text{BindTree}_m(t_1, \vec{t}_2, \dots, t_{k_m-1}, j) \}.$$

The commitment predicate $(\Phi_m^{\text{prop}}, \Phi_m^{\text{bind}})$ has failure density p_m^{com} if it always holds that there is some coordinate $j \in [r_m]$ such that

$$|\text{BindChal}_m(t_1, \vec{t}_2, \dots, \vec{t}_{k_m-1}, j)| \leq p_m^{\text{com}} |S_m|,$$

for all $(t_1, \vec{t}_2, \dots, \vec{t}_{k_m-1}) \in \text{PropUnsat}_m$.

In coordinate-wise special-sound protocols, one witness element $\vec{w}_i$ can typically be extracted from every coordinate i . When the property predicate does not hold, we require that there is some coordinate where we can apply the failure density for the binding predicate. In other words, there must be some coordinate i where only a small fraction of challenges allow the prover to use the same witness $\vec{w}_i$, and thereby not violate binding.

When the binding predicate is not satisfied in some coordinate, we must be able to obtain a witness for the binding relation.

Definition G.7 (Coordinate-wise binding relation). Let $\Pi = (\mathcal{P}, \mathcal{V})$ be a $(\mathbf{R}, \mathbf{K}, \Phi)$ -coordinate-wise predicate-special-sound argument of knowledge for a relation R_{pp} , and let $R_{\text{pp}}^{\text{bind}}$ be an additional relation. We say that Φ admits $R_{\text{pp}}^{\text{bind}}$ as a coordinate-wise binding relation if there exists a polynomial time algorithm $\mathcal{B}$, with the following property. Say it gets as input some statement x , trees for this statement $(t_1, \vec{t}_2, \dots, \vec{t}_{k_m-1}) \in \mathbb{T}_{m+1}^{(k_m-1)}$ and $t \in \mathbb{T}_{m+1}$, and an index $j \in [r_m]$ such that

$$\begin{aligned} & \Phi_m^{\text{bind}}((t_1, \vec{t}_2, \dots, \vec{t}_{k_m-1}), j, t) = 0 \text{ and} \\ & \forall \ell \in [k_m], i \in [r_m] : \Phi_{m+1}(t_{\ell,i}) = 1 \text{ and } \Phi_{m,\ell}^{\text{chal}}((\vec{s}_1, \vec{c}_2, \dots, \vec{c}_{\ell-1}), i, s_{\ell,i,i}) = 1, \end{aligned}$$

where $\vec{s}_1 = \text{chal}_m(t_1)$ and $\vec{c}_i = \text{chal}_m(\vec{t}_i)$ for $i > 1$. Then it always holds that $\mathcal{B}((t_1, \vec{t}_2, \dots, \vec{t}_{k_m-1}), j, t) \in R_{\text{pp}}^{\text{bind}}(x)$.

We may now finally describe the knowledge soundness of a coordinate-wise predicate-special-sound protocol.

Theorem G.1. Let $\Pi = (\mathcal{P}, \mathcal{V})$ be a $(\mathbf{R}, \mathbf{K}, \Phi)$ -coordinate-wise predicate-special-sound argument of knowledge for a relation R_{pp} . In addition, let $R_{\text{pp}}^{\text{bind}}$ be a coordinate-wise binding relation for Φ . Then the adaptive Fiat-Shamir transformation $FS[\Pi]$ is adaptively knowledge sound for the relation $R_{\text{pp}} \cup R_{\text{pp}}^{\text{bind}}$ with knowledge error

$$2(Q+1) \sum_{i=1}^{\mu} r_i \cdot \max \left(\frac{k_i - 1}{|\mathcal{C}_i|}, p_i^{\text{com}} + \sum_{\ell=1}^{k_i} p_{i,\ell}^{\text{chal}} \right).$$

The number of times that the knowledge extractor invokes the prover is in expectation at most $K + Q(K-1)$, where $K = \prod_{i=1}^{\mu} (r_i(k_i - 1) + 1)$.

H Knowledge Soundness Proof for PSS

In this section, we present our proof for Theorem 5.1 and Theorem G.1. First, we show that for the adaptive Fiat-Shamir transformation of any predicate-special-sound protocol, we can construct an efficient knowledge extractor that outputs a witness with knowledge error depending only on the tree structure and

the failure densities of the predicates. To this end, our approach is to extend the seminal work [AFK22] by Attema, Fehr and Klooß, which analyzes the knowledge soundness of multi-round special-sound protocols. Conceptually, our knowledge extractor is the same as theirs, except for some additional predicate checks. These predicate checks do not affect the expected runtime of the extractor, only its success probability. To capture the core of their extraction strategy, [AFK22] introduces an abstract sampling game. Analyzing the properties of this abstract game allows them to derive the knowledge error and expected runtime of their knowledge extractor without being bogged down with heavy notation. We follow their approach.

Next, we generalize our extractor construction to coordinate-wise predicate-special-sound protocols. The extractor for coordinate-wise special-sound protocols in [FMN23] essentially runs the [AFK22] extractor for special sound protocols in each coordinate. Our extractor is the [FMN23] extractor extended with predicate checks at the very end.

The rest of this section is organized as follows. In Section H.1, we present and analyze a new abstract sampling game for our predicate special soundness knowledge extractor. Then in Section H.2, we describe the knowledge extractor and analyze its efficiency and success probability using the abstract game. Finally in Section H.3, we generalize our analysis to coordinate-wise predicate special soundness.

H.1 Abstract Sampling Game

The abstract sampling game is presented in Game 3. Entries are sampled from the array M exactly like in [AFK22]. The only changes are that each entry of M has an extra t element and that we might additionally fail at the very end because of the predicate functions.

Game 3: The abstract sampling game with predicates

Parameters:

- $k, N, U \in \mathbb{N}$ and a set T .
- U -dimensional array M with entries $M(j_1, \dots, j_U) \in \{0, 1\} \times [U] \times T$ for all $(j_1, \dots, j_U) \in [N]^U$.
- Functions $f_\ell^{\text{chal}} : [N]^\ell \rightarrow \{0, 1\}$ for $\ell \in [k]$.
- A pair of functions $(f^{\text{prop}}, f^{\text{bind}})$ with $f^{\text{prop}} : T^{k-1} \rightarrow \{0, 1\}$ and $f^{\text{bind}} : T^k \rightarrow \{0, 1\}$.

Game:

1. Sample $(j_1, \dots, j_U) \xleftarrow{\$} [N]^U$ and compute $(v, i, t) = M(j_1, \dots, j_U)$.
2. If $v = 0$, abort with output 0.
3. Set $c_1 := j_i$, $t_1 := t$, $\ell := 1$ and $W := [N] \setminus \{c_1\}$.
4. Repeat the following until $\ell = k$ or $W = \emptyset$:
 - (a) Sample $j' \xleftarrow{\$} W$ and set $W := W \setminus \{j'\}$.
 - (b) Compute $(v', i', t') = M(j_1, \dots, j_{i-1}, j', j_{i+1}, \dots, j_U)$.
 - (c) If $v' = 1$ and $i' = i$, set $\ell := \ell + 1$, $c_\ell := j'$ and $t_\ell := t'$.
5. If $\ell < k$ or $f_n^{\text{chal}}(c_1, \dots, c_n) = 0$ for some $n \in [k]$, output 0.
6. Output $f^{\text{prop}}(t_1, \dots, t_{k-1}) \vee \neg f^{\text{bind}}(t_1, \dots, t_k)$.

Before analyzing the properties of the abstract sampling game, we will define some helpful notation. Following [AFK22], for all $i \in [U]$ we define the function

$$a_i : ([N]^r)^U \rightarrow \mathbb{N}_{\geq 0},$$

$$(\vec{j}_1, \dots, \vec{j}_U) \mapsto \left| \left\{ \vec{j} \in [N]^r \mid M(\vec{j}_1, \dots, \vec{j}_{i-1}, \vec{j}, \vec{j}_{i+1}, \dots, \vec{j}_U) = (1, i, \cdot) \right\} \right|.$$

This function counts the number of entries with $v = 1$ and index i in the 1-dimensional array $M(j_1, \dots, j_{i-1}, \cdot, j_{i+1}, \dots, j_U)$. Notice that the function does not depend on the i -th input j_i . Therefore we sometimes, by a slight abuse of notation, write $a_i(j_1, \dots, j_{i-1}, j_{i+1}, \dots, j_U)$.

Lemma H.1. Consider the game in Game 3. Let $\vec{J} = (J_1, \dots, J_U)$ be uniformly distributed in $[N]^U$, indicating the first entry sampled, and let $(V, I, T) = M(\vec{J})$. For each $i \in [U]$, let $A_i = a_i(\vec{J})$. Let F_ℓ^{chal} be the random variable indicating the outcome of f_ℓ^{chal} . Let F^{prop} and F^{bind} be the random variables for the outcome of f^{prop} and f^{bind} respectively. Let $d_1^{\text{chal}}, \dots, d_k^{\text{chal}}, d^{\text{com}} \in [0, 1]$ be numbers such that for all $i \in [U]$ and all $a \in \mathbb{N}$, $k \leq a \leq N$,

$$\begin{aligned} \frac{d_\ell^{\text{chal}} N}{a - \ell + 1} &\geq \Pr \left[F_\ell^{\text{chal}} = 0 \mid V = 1, I = i, A_i = a, \bigcap_{n=1}^{\ell-1} F_n^{\text{chal}} = 1 \right] \text{ and} \\ \frac{d^{\text{com}} N}{a - k + 1} &\geq \Pr \left[F_k^{\text{chal}} = 1, F^{\text{bind}} = 1 \mid \begin{array}{l} V = 1, I = i, A_i = a, \\ \bigcap_{\ell=1}^{k-1} F_\ell^{\text{chal}} = 1, F^{\text{prop}} = 0 \end{array} \right]. \end{aligned}$$

Finally, set some tuning parameter $1 \leq \sigma \leq N/k$ such that $\sigma k \in \mathbb{N}$. Then the game outputs 1 with probability at least

$$\Pr[V = 1] - A \cdot \max \left(\frac{\sigma k - 1}{N}, \frac{\sigma k}{\sigma k - k + 1} d^{\text{com}} + \sum_{\ell=1}^k \frac{\sigma k}{\sigma k - \ell + 1} d_\ell^{\text{chal}} \right),$$

where $A = \sum_{i=1}^U \Pr[A_i > 0]$.

Proof. By the description of the abstract game, we have that the probability that the game outputs 1 is

$$\begin{aligned} &\sum_{i=1}^U \Pr \left[V = 1, I = i, A_i \geq k, \bigcap_{\ell=1}^k F_\ell^{\text{chal}} = 1, (F^{\text{prop}} = 1 \cup F^{\text{bind}} = 0) \right] \\ &\geq \sum_{i=1}^U \Pr \left[V = 1, I = i, A_i \geq \sigma k, \bigcap_{\ell=1}^k F_\ell^{\text{chal}} = 1, (F^{\text{prop}} = 1 \cup F^{\text{bind}} = 0) \right]. \end{aligned}$$

We lower the success probability to allow bounding the impact of the failure density when sampling without replacement.

$$= \Pr[V = 1] - \sum_{i=1}^U \Pr[V = 1, I = i, A_i < \sigma k] \tag{11}$$

$$- \sum_{i=1}^U \Pr \left[V = 1, I = i, A_i \geq \sigma k, \bigcup_{\ell=1}^k F_\ell^{\text{chal}} = 0 \right] \tag{12}$$

$$- \sum_{i=1}^U \Pr \left[V = 1, I = i, A_i \geq \sigma k, \bigcap_{\ell=1}^k F_\ell^{\text{chal}} = 1, F^{\text{prop}} = 0, F^{\text{bind}} = 1 \right] \tag{13}$$

The equality holds by repeated application of $\Pr[A \cap B] = \Pr[A] - \Pr[A \cap \neg B]$ for events A, B . The expression contains three sums that represent each of the failure cases. We will individually upper bound each case, starting with (11).

$$\begin{aligned} \sum_{i=1}^U \Pr[V = 1, I = i, A_i < \sigma k] &= \sum_{i=1}^U \sum_{a=0}^{\sigma k-1} \Pr[A_i = a] \Pr[V = 1, I = i \mid A_i = a] \\ &= \sum_{i=1}^U \sum_{a=0}^{\sigma k-1} \Pr[A_i = a] \frac{a}{N} \\ &\leq \sum_{i=1}^U \sum_{a=1}^{\sigma k-1} \Pr[A_i = a] \frac{\sigma k - 1}{N} \end{aligned}$$

Next, we upper bound the second sum (12). For the sake of conciseness, we let $E_{i,a}$ denote the event that $V = 1, I = i, A_i = a$.

$$\begin{aligned}
& \sum_{i=1}^U \Pr \left[V = 1, I = i, A_i \geq \sigma k, \bigcup_{\ell=1}^k F_\ell^{\text{chal}} = 0 \right] \\
&= \sum_{i=1}^U \Pr \left[V = 1, I = i, A_i \geq \sigma k, \bigcup_{\ell=1}^k \left(F_\ell^{\text{chal}} = 0, \bigcap_{n=1}^{\ell-1} F_n^{\text{chal}} = 1 \right) \right] \\
&\leq \sum_{i=1}^U \sum_{\ell=1}^k \Pr \left[V = 1, I = i, A_i \geq \sigma k, F_\ell^{\text{chal}} = 0, \bigcap_{n=1}^{\ell-1} F_n^{\text{chal}} = 1 \right] \\
&= \sum_{i=1}^U \sum_{a=\sigma k}^N \Pr[A_i = a] \frac{a}{N} \sum_{\ell=1}^k \Pr \left[F_\ell^{\text{chal}} = 0, \bigcap_{n=1}^{\ell-1} F_n^{\text{chal}} = 1 \mid E_{i,a} \right] \\
&\leq \sum_{i=1}^U \sum_{a=\sigma k}^N \Pr[A_i = a] \frac{a}{N} \sum_{\ell=1}^k \Pr \left[F_\ell^{\text{chal}} = 0 \mid E_{i,a}, \bigcap_{n=1}^{\ell-1} F_n^{\text{chal}} = 1 \right] \\
&\leq \sum_{i=1}^U \sum_{a=\sigma k}^N \Pr[A_i = a] \frac{a}{N} \sum_{\ell=1}^k \frac{d_\ell^{\text{chal}} N}{a - \ell + 1} \\
&= \sum_{i=1}^U \sum_{a=\sigma k}^N \Pr[A_i = a] \sum_{\ell=1}^k \frac{a}{a - \ell + 1} d_\ell^{\text{chal}} \\
&\leq \sum_{i=1}^U \sum_{a=\sigma k}^N \Pr[A_i = a] \sum_{\ell=1}^k \frac{\sigma k}{\sigma k - \ell + 1} d_\ell^{\text{chal}}
\end{aligned}$$

For the last step, we exploited that $a \geq \sigma k \geq 1$ implies $a/(a - \ell + 1) \leq \sigma k/(\sigma k - \ell + 1)$. We will use a very similar approach to upper bound the third sum (13). Recall that $F^{\text{prop}} = 0$ is determined by the first $k - 1$ successes sampled.

$$\begin{aligned}
& \sum_{i=1}^U \Pr \left[V = 1, I = i, A_i \geq \sigma k, \bigcap_{\ell=1}^k F_\ell^{\text{chal}} = 1, F^{\text{prop}} = 0, F^{\text{bind}} = 1 \right] \\
&\leq \sum_{i=1}^U \sum_{a=\sigma k}^N \Pr[A_i = a] \frac{a}{N} \\
&\quad \cdot \Pr \left[F_k^{\text{chal}} = 1, F^{\text{bind}} = 1 \mid E_{i,a}, \bigcap_{\ell=1}^{k-1} F_\ell^{\text{chal}} = 1, F^{\text{prop}} = 0 \right] \\
&\leq \sum_{i=1}^U \sum_{a=\sigma k}^N \Pr[A_i = a] \frac{a}{N} \cdot \frac{d^{\text{com}} N}{a - k + 1} \\
&\leq \sum_{i=1}^U \sum_{a=\sigma k}^N \Pr[A_i = a] \frac{\sigma k}{\sigma k - k + 1} d^{\text{com}}
\end{aligned}$$

Using the bounds for the three sums (11,12,13), the lemma follows.

$$\begin{aligned}
& \Pr[V = 1] - \sum_{i=1}^U \sum_{a=1}^{\sigma k-1} \Pr[A_i = a] \frac{\sigma k - 1}{N} \\
&\quad - \sum_{i=1}^U \sum_{a=\sigma k}^N \Pr[A_i = a] \left(\frac{\sigma k}{\sigma k - k + 1} d^{\text{com}} + \sum_{\ell=1}^k \frac{\sigma k}{\sigma k - \ell + 1} d_\ell^{\text{chal}} \right) \\
&\geq \Pr[V = 1] - A \cdot \max \left(\frac{\sigma k - 1}{N}, \frac{\sigma k}{\sigma k - k + 1} d^{\text{com}} + \sum_{\ell=1}^k \frac{\sigma k}{\sigma k - \ell + 1} d_\ell^{\text{chal}} \right)
\end{aligned}$$

□

The rest of the analysis of the abstract sampling game is identical to the one in [AFK22]. For completeness, we include the relevant results below.

Lemma H.2 (Expected cost [AFK22, Lemma 5]). Consider the game in Game 3, as well as a cost function $\Gamma : [N]^U \rightarrow \mathbb{R}_{\geq 0}$ and a constant cost $\gamma \in \mathbb{R}_{\geq 0}$. Let $\vec{J} = (J_1, \dots, J_U)$ be uniformly distributed in $[N]^U$, indicating the first entry sampled, and let $(V, I, T) = M(\vec{J})$. For each $i \in [U]$, let $A_i = a_i(\vec{J})$. We define the cost of sampling an entry $M(j_1, \dots, j_U) = (v, i, t)$ with index $i = I$ to be $\Gamma(j_1, \dots, j_U)$ and the cost of sampling an entry $M(j_1, \dots, j_U) = (v, i, t)$ with $i \neq I$ to be γ . Let Δ be the total cost of playing this game. Then

$$\mathbb{E}[\Delta] \leq k \cdot \mathbb{E}[\Gamma(\vec{J})] + (k-1) \cdot A' \cdot \gamma,$$

where $A' = \sum_{i=1}^U \Pr[I \neq i, A_i > 0] \leq A$.

Lemma H.3 ([AFK22, Lemma 3 & 6]). Consider the game in Game 3. Let v , idx and $tree$ be functions such that $M(\vec{j}) = (v(\vec{j}), idx(\vec{j}), tree(\vec{j}))$ for all $\vec{j} \in [N]^U$. Furthermore, let $\vec{J} = (J_1, \dots, J_U)$ be uniformly distributed in $[N]^U$ and set $A_i = a_i(J)$ for all $i \in [U]$. Assume that for all $\vec{j} \in [N]^U$ there exists a subset $S(\vec{j}) \subseteq [U]$ of cardinality at most Q such that $idx(\vec{j}) = idx(\vec{j}')$ for all $\vec{j}, \vec{j}'$ with $j_\ell = j'_\ell$ for all $\ell \in S(\vec{j})$. Then

$$A = \sum_{i=1}^U \Pr[A_i > 0] \leq Q + 1 \text{ and}$$

$$A' = \sum_{i=1}^U \Pr[idx(J) \neq i, A_i > 0] \leq Q.$$

H.2 Knowledge Extractor

We move on to discuss how to construct a knowledge extractor for the adaptive Fiat-Shamir transformation $\text{FS}[\Pi]$ of some $(\mathbf{K}, \Phi)$ -predicate-special-sound interactive argument Π . To simplify the presentation, like [AFK22] we will only present the case where all the rounds of Π use the same challenge set $\mathcal{C}$ of cardinality N . The analysis can easily be generalized to the case where each round i has a different challenge set $\mathcal{C}_i$ of cardinality N_i . The proof is the same as before, except that we need to handle the additional bookkeeping from having a different random oracle $\text{RO}_i : \{0, 1\}^{\leq u} \rightarrow \mathcal{C}_i$ for each round.

Let $\mathcal{P}^*$ be some adaptive Q -query random oracle prover for $\text{FS}[\Pi]$. By Remark B.1, we can assume that $\mathcal{P}^*$ is deterministic. After making at most Q queries to the random oracle, the prover outputs $(x, \pi, \text{aux}) \leftarrow \mathcal{P}^*$, where x is a statement, $\pi = (a_1, \dots, a_{\mu+1})$ a proof and aux some auxiliary information. We reformat the output of $\mathcal{P}^*$ to be an index vector $\vec{I} = (I_1, \dots, I_{\mu+1})$, where

$$I_1 := (x, a_1), I_2 := (x, a_1, a_2), \dots, I_{\mu+1} := (x, a_1, \dots, a_{\mu+1}).$$

On top of that, we extend $\mathcal{P}^*$ to a $(Q + \mu)$ -query algorithm $\mathcal{A}$ that checks the validity of the proof. It computes $c_i = \text{RO}(I_i)$ for $i \in [\mu]$ and outputs

$$\vec{I}, t := (a_1, c_1, \dots, a_\mu, c_\mu, a_{\mu+1}), v := \mathcal{V}(x, t), b := 1 \text{ and } \text{aux}.$$

The bit b indicates whether t satisfies the binding constraints or not. For $\mathcal{A}$ we trivially have that $b = 1$, but we include it to make the output of $\mathcal{A}$ consistent with our extractor.

The extractor is constructed recursively as a sequence of subextractors $\mathcal{E}_1, \dots, \mathcal{E}_{\mu+1}$. The goal of $\mathcal{E}_m$ is to output a tree $t \in \mathbb{T}_m$ such that either $\Phi(t) = 1$ or such that t encodes a witness for the binding relation R_{bind} . To do so, it essentially plays an instantiation of the abstract sampling game with the previous subextractor $\mathcal{E}_{m+1}$. At the bottom of the recursion, $\mathcal{E}_{\mu+1} = \mathcal{A}$. For the recursive argument to work, we need the subextractors to be the same kind of object as $\mathcal{A}$, namely a random oracle algorithm making the same number of queries. We define $\mathcal{E}_m$ in Extractor 1. The final extractor $\mathcal{E}$ for $\text{FS}[\Pi]$ is obtained by running $\mathcal{E}_1$ and using lazy-sampling to answer its random oracle queries.

Extractor 1: The subextractor $\mathcal{E}_m$

Parameters:

- $k_m, Q \in \mathbb{N}$.
- Challenge predicates $\Phi_{m,\ell}^{\text{chal}} : \mathcal{C}^\ell \rightarrow \{0, 1\}$ for $\ell \in [k_m]$.
- A commitment predicate $(\Phi_m^{\text{prop}}, \Phi_m^{\text{bind}})$, where $\Phi_m^{\text{prop}} : \mathbb{T}_{m+1}^{(k_m-1)} \rightarrow \{0, 1\}$ and $\Phi_m^{\text{bind}} : \mathbb{T}_{m+1}^{(k_m-1)} \times \mathbb{T}_{m+1} \rightarrow \{0, 1\}$.

Black-box access to: $\mathcal{E}_{m+1}$.

Random oracle queries: $Q + \mu$.

1. Run $\mathcal{E}_{m+1}$ as follows to obtain $(\vec{I}, t, v, b, \text{aux})$: Relay the $Q + \mu$ queries to the random oracle and record all query-response pairs. Set $i := I_m$ and let $j_i \in \mathcal{C}$ be the response to query i .
2. If $v = 0$, the extractor fails with output $v := 1$.
3. Set $b_1 := b$, $c_1 := j_i$, $t_1 := t$, $W := \mathcal{C} \setminus \{c_1\}$ and $\ell := 1$.
4. Repeat the following until $\ell = k_m$ or $W = \emptyset$:
 - (a) Sample $j' \xleftarrow{\$} W$ and set $W := W \setminus \{j'\}$.
 - (b) Run $\mathcal{E}_{m+1}$ as follows to obtain $(\vec{I}', t', v', b', \text{aux}')$:
 - i. Stop after the initial run of $\mathcal{P}^*$ if $I'_m \neq I_m$.
 - ii. Answer the query to i with j' .
 - iii. Answer other queries from $\mathcal{E}_{m+1}$ consistently with the responses stored. Each new query is answered with a locally sampled fresh random value from $\mathcal{C}$, and recorded.
 - (c) If $v' = 1$ and $I'_m = I_m$, set $\ell := \ell + 1$, $b_\ell = b'$, $c_\ell := j'$ and $t_\ell := t'$.
5. If $\ell < k_m$ or $\Phi_{m,n}^{\text{chal}}(c_1, \dots, c_n) = 0$ for some $n \in [k_m]$, the extractor fails with output $v := 0$.
6. If $b_n = 0$ for some $n \in [k_m]$ or $\Phi_m^{\text{bind}}((t_1, \dots, t_{k_m-1}), t_{k_m}) = 0$, let $b := 0$. Else, let $b := 1$.
7. If $\Phi_m^{\text{prop}}(t_1, \dots, t_{k_m-1}) = 0$ and $b = 1$, the extractor fails with output $v := 0$. Else, it succeeds with output $\vec{I}, (t_1, \dots, t_{k_m}), v := 1, b, \text{aux}$.

For the sake of a simpler description of $\mathcal{E}_m$ that is more consistent with the abstract sampling game, $\mathcal{E}_m$ does not terminate immediately when $\mathcal{E}_{m+1}$ outputs a tuple with $v = 1$ and $b = 0$, even though it has found a valid witness for R_{bind} . Note that this optimization would not lead to an improved result for the success probability or the expected run time, as in the worst case we always have that $\mathcal{E}_{m+1}$ outputs $b = 1$.

For adaptive security, the statement is considered part of the first prover message a_1 . If the extractor $\mathcal{E}$ is able to output a tree of transcripts, it holds immediately that they all have the same statement. To fit Definition B.11, the output format of $\mathcal{E}$ should be modified to the form $(x, \pi, \text{aux}, v, w)$, where (x, π, aux) is identically distributed to the output of $\mathcal{P}^*$ and $v = \mathcal{V}(x, \pi)$. This change can easily be implemented in Extractor 1, but we omit it to make the presentation more clear.

Lemma H.4 (Correctness and consistency [AFK22, Lemma 7 & Proposition 1]). *For any fixed choice of the random oracle RO, let $(\vec{I}, (t^{(1)}, \dots, t^{(k_m)}), v, b) \leftarrow \mathcal{E}_m^{\text{RO}}$. If $v = 1$, then $(t^{(1)}, \dots, t^{(k_m)}) \in \mathbb{T}_m$. Furthermore, the index vector $\vec{I}$ and the auxilliary information aux equal those output by $\mathcal{P}^*$ querying RO.*

The subextractor $\mathcal{E}_m$ samples entries in exactly the same way as in [AFK22], but it might additionally fail because of the predicates. Hence, we can use the same argument for the expected run time of $\mathcal{E}_m$ as in [AFK22], but the success probability analysis now relies on Lemma H.1.

Lemma H.5 (Run Time and Success Probability). *Let $N = |\mathcal{C}|$. The extractor $\mathcal{E}_m$ succeeds and outputs $v = 1$ with probability at least*

$$\varepsilon(\mathcal{P}^*) - (Q + 1) \sum_{i=\mu}^m \min_{\sigma_i \in [1, \frac{N}{k_i}]: \sigma_i k_i \in \mathbb{N}} \kappa_i(\sigma_i),$$

where

$$\kappa_i(\sigma) = \max \left(\frac{\sigma k_i - 1}{N}, \frac{\sigma k_i}{\sigma k_i - k_i + 1} p_i^{\text{com}} + \sum_{\ell=1}^{k_i} \frac{\sigma k_i}{\sigma k_i - \ell + 1} p_{i,\ell}^{\text{chal}} \right).$$

The number of times it invokes $\mathcal{P}^*$ is in expectation at most $K_m + Q(K_m - 1)$, where $K_m = \prod_{i=m}^\mu k_i$.

Proof. The proof is by induction in m . The base case $m = \mu + 1$ holds trivially. For the induction step, we assume that the lemma holds for $m + 1$. The idea of the proof is to demonstrate how one can view $\mathcal{E}_m$ as playing an instantiation of the abstract sampling game in Game 3 with $\mathcal{E}_{m+1}$, allowing us to derive the properties of $\mathcal{E}_m$ from the properties of the game.

Let $U = |\{0, 1\}^{\leq u}|$ denote the cardinality of the domain of the random oracle $\text{RO} : \{0, 1\}^{\leq u} \rightarrow \{0, 1\}$. Fix an arbitrary ordering $\xi_1, \dots, \xi_U$ for the bitstrings $\xi_i \in \{0, 1\}^{\leq u}$. Then a vector $\vec{j} \in \mathcal{C}^U$ encodes the function table of the random oracle with $\text{RO}(\xi_i) = j_i$. Within a run of $\mathcal{E}_m$, all queries made by the different invocations of $\mathcal{E}_{m+1}$ are answered consistently using lazy sampling, except for the queries to the index i , where we reprogram the response. This is indistinguishable from how it is done in the abstract sampling game, where the entire function table $\vec{j}$ of the random oracle is sampled initially. For the purpose of analysis, we modify $\mathcal{E}_m$ to handle the random oracle queries like in the game. This change has no impact on its success probability or its expected number of prover invocations.

Next, let us define the array M . Observe that since $\mathcal{A}$ is deterministic, we can view it as a function from a choice of random oracle to its output given access to that random oracle. Then for $\mathcal{E}_\mu$, we can define $M(\vec{c}) = (v, I_\mu, t)$, where $(\vec{I}, t, v, b, \text{aux}) \leftarrow \mathcal{A}(\vec{c})$. However, for $\mathcal{E}_m$, the subextractor it invokes $\mathcal{E}_{m+1}$ is not a deterministic algorithm. Defining the entries of M by the output of $\mathcal{E}_{m+1}$ will not give us a deterministic array like in the game.

The trick to still be able to use the abstract game is to analyze the case when $\mathcal{E}_{m+1}$ is using some fixed random tape. In this case, M is a deterministic array like in the sampling game. Averaging over the choice of random tape for $\mathcal{E}_{m+1}$ will, by the linearity of the success probability and the expected run time, yield the desired result. Formally, to allow for fresh randomness in the different runs of $\mathcal{E}_{m+1}$ within $\mathcal{E}_m$, we actually fix a choice of function $f : \mathcal{C}^U \rightarrow \{0, 1\}^{\leq r}$. The invocation of $\mathcal{E}_{m+1}$ with random oracle $\vec{j} \in \mathcal{C}^U$ will use $f(\vec{j})$ as its random tape.

We instantiate f_ℓ^{chal} with $\Phi_{m,\ell}^{\text{chal}}$ for $\ell \in [k_m]$ and $(f^{\text{prop}}, f^{\text{bind}})$ with $(\Phi_m^{\text{prop}}, \Phi_m^{\text{bind}})$. A difference between $\mathcal{E}_m$ and the abstract game is when the predicates lead to failure. Namely, the difference is the b bits output by $\mathcal{E}_{m+1}$, which are not present in the abstract game. If $b_n = 0$ for some $n \in [k]$, then $\mathcal{E}_m$ will succeed even though $\Phi_m^{\text{prop}}(t_1, \dots, t_{k_m-1})$ and $\Phi_m^{\text{bind}}((t_1, \dots, t_{k_m-1}), t_{k_m}) = 1$, so that it would fail in the game. Luckily, we are lower bounding the success probability of $\mathcal{E}_m$, so we will simply assume that we always have $b_n = 1$ for all $n \in [k_m]$. Clearly, this can only decrease the success probability of $\mathcal{E}_m$. As a consequence, the b output of $\mathcal{E}_{m+1}$ can be ignored completely, so that $\mathcal{E}_m$ is truly playing the abstract game.

Next, we need to bound the parameters $A = \sum_{i=1}^U \Pr[A_i > 0]$ and $d_1^{\text{chal}}, \dots, d_k^{\text{chal}}, d^{\text{com}}$ from Lemma H.1. We bound A using Lemma H.3. The observation is that the set $S(\vec{c}) \subseteq \{0, 1\}^{\leq u}$ corresponds to the set of at most Q indices that the prover queries to the random oracle when the random oracle is $\vec{c}$. Since $\mathcal{P}^*$ is deterministic, its output can only change when the random oracle is reprogrammed at one of the indices in $S(\vec{c})$. This then also holds for the index vector output by $\mathcal{E}_{m+1}$, since $\mathcal{E}_{m+1}$ outputs the same index vector $\vec{I}$ as $\mathcal{P}^*$ for any random oracle $\vec{c}$ by Lemma H.4. It follows that we can apply Lemma H.3 to bound $A \leq Q + 1$.

We can use Definition 5.2 to bound $d_1^{\text{chal}}, \dots, d_{k_m}^{\text{chal}}$. The first observation is that if $\mathcal{E}_{m+1}$ outputs an entry with $v = 1$, $b = 1$ and subtree $t \in \mathbb{T}_{m+1}$, then by construction $\Phi_{m+1}(t) = 1$. Next, given that $\mathcal{E}_m$ has obtained a first success with index i and is sampling from a 1-dimensional array with $a \geq k_m$, it will obtain a uniform random size k_m subset of these a successes. Fix any choice for $c_1, \dots, c_{\ell-1}$ such that the first $\ell - 1$ challenge predicates are satisfied. Then we know that the ℓ -th challenge is uniformly distributed among the $a - \ell + 1$ remaining successes. Out of these, at most $p_{m,\ell}^{\text{chal}}N$ are such that Φ_ℓ^{chal} fails. Thus, we can set $d_\ell^{\text{chal}} = p_\ell^{\text{chal}}$ for all $\ell \in [k_m]$.

Similarly, we can use Definition 5.6 to set $d^{\text{com}} = p_m^{\text{com}}$. Fix any choice for the first $k - 1$ successes such that the challenge predicates are satisfied, but the property predicate is not. Then the k_m -th challenge is uniformly distributed among the remaining $a - k_m + 1$ successes. Out of these, we know that there are at most $p_m^{\text{com}}N$ challenges such that the k_m -th challenge predicate and the binding predicate are satisfied. Thus, we can set $d^{\text{com}} = p_m^{\text{com}}$.

Let us now derive the lower bound for the success probability of $\mathcal{E}_m$. Let F be the random variable indicating the choice of random tape function, let $\vec{J}$ denote the initial choice of random oracle in $\mathcal{C}^U$ and let $S_{m'}$ for $n \in \{m, m + 1\}$ be the event that $\mathcal{E}_n$ outputs $v = 1$ when $\mathcal{E}_{m+1}$ is run with the random oracle

Game 4: The coordinate-wise abstract sampling game

Parameters:

- $k, r, N, U \in \mathbb{N}$ and a set T .
- rU -dimensional array M with entries $M(\vec{j}_1, \dots, \vec{j}_U) \in \{0, 1\} \times [U] \times T$ for all $(\vec{j}_1, \dots, \vec{j}_U) \in ([N]^r)^U$.
- Functions $f_{\ell, n}^{\text{chal}} : ([N]^r)^{\ell-1} \times [N] \rightarrow \{0, 1\}$ for all $\ell \in [k], n \in [r]$.
- Functions $(f_1^{\text{prop}}, f_1^{\text{bind}}, \dots, f_r^{\text{bind}})$ with $f^{\text{prop}} : T \times (T^r)^{k-2} \rightarrow \{0, 1\}$ and $f_n^{\text{bind}} : T \times (T^r)^{k-2} \times T \rightarrow \{0, 1\}$ for all $n \in [r]$.

Game:

1. Sample $(\vec{j}_1, \dots, \vec{j}_U) \xleftarrow{\$} ([N]^r)^U$ and compute $(v, i, t) = M(\vec{j}_1, \dots, \vec{j}_U)$.
2. If $v = 0$, abort with output 0.
3. Set $\vec{s}_1 := \vec{j}_i$, $t_1 := t$. For each $n \in [r]$, set $\ell_n := 1$ and $W_n := [N] \setminus \{s_{1,n}\}$.
4. For each $n \in [r]$, repeat the following until $\ell_n = k$ or $W_n = \emptyset$:
 - (a) Sample $j' \xleftarrow{\$} W_n$ and set $W_n := W_n \setminus \{j'\}$.
 - (b) Set $\vec{j}' := (j_{i,1}, \dots, j_{i,n-1}, j', j_{i,n+1}, \dots, j_{i,r})$.
 - (c) Compute $(v', i', t') = M(\vec{j}_1, \dots, \vec{j}_{i-1}, \vec{j}', \vec{j}_{i+1}, \dots, \vec{j}_U)$.
 - (d) If $v' = 1$ and $i' = i$, set $\ell_n := \ell_n + 1$, $s_{\ell_n, n} := j'$ and $t_{\ell_n, n} := t'$.
5. If $\ell_n < k$ for some $n \in [r]$, output 0.
6. For each $\ell \in [k] \setminus \{1\}$, set $\vec{s}_\ell := (s_{\ell,1}, \dots, s_{\ell,r})$ and $\vec{t}_\ell := (t_{\ell,1}, \dots, t_{\ell,r})$.
7. If $f_{\ell, n}^{\text{chal}}((\vec{s}_1, \dots, \vec{s}_{\ell-1}), s_{\ell, n}) = 0$ for some $\ell \in [k], n \in [r]$, output 0.
8. If $f^{\text{prop}}(t_1, (\vec{t}_2, \dots, \vec{t}_{k-1})) = (0, y)$ for some y and $\bigvee_{n=1}^r f_n^{\text{bind}}(t_1, (\vec{t}_2, \dots, \vec{t}_{k-1}), t_{k,n}) = 1$, output 0. Else, output 1.

$\vec{J}$ and random tape $F(\vec{J})$. For any choice of $1 \leq \sigma_i \leq N/k_i$ such that $\sigma_i k_i \in \mathbb{N}$ for $i \in [\mu]$, we have that

$$\begin{aligned}
\Pr[S_m] &= \sum_f \Pr[F = f] \Pr[S_m \mid F = f] \\
&\geq \sum_f \Pr[F = f] (\Pr[S_{m+1} \mid F = f] - (Q+1)\kappa_m(\sigma_m)) \\
&= \Pr[S_{m+1}] - (Q+1)\kappa_m(\sigma_m) \\
&\geq \varepsilon(\mathcal{P}^*) - (Q+1) \sum_{i=\mu+1}^m \kappa_i(\sigma_i).
\end{aligned}$$

The first inequality follows by Lemma H.1 and Lemma H.3, the second inequality by the induction hypothesis.

The argument for the expected number of invocations of $\mathcal{A}$ follows similarly as for the success probability, using Lemma H.2 and that we stop an invocation of $\mathcal{E}_{m+1}$ after a single invocation of $\mathcal{A}$ if the indices do not match. $\square$

To simplify the presentation of our result in Theorem 5.1, we set $\sigma_i = 2$ for all $i \in [\mu]$. Then $\sigma k_i / ((\sigma_i - 1)k_i + 1) \leq 2$.

H.3 Coordinate-Wise Extension

In this section, we generalize our extractor to the coordinate-wise setting. The extractor for coordinate-wise special-sound protocols in [FMN23] essentially runs the [AFK22] extractor for special sound protocols in each coordinate. We obtain our extractor by modifying their coordinate-wise extractor to potentially fail at the very end because of the predicates. The coordinate-wise abstract sampling game with predicates is presented in Game 4. As in Game 3, the predicates do not affect the expected number of entries sampled in the abstract game. We therefore only need to focus on analyzing the probability of winning the game.

For all $i \in [U], n \in [r]$ we define the functions

$$\begin{aligned} a_i : ([N]^r)^U &\rightarrow \mathbb{N}_{\geq 0}, \\ (\vec{j}_1, \dots, \vec{j}_U) &\mapsto \left| \left\{ \vec{j} \in [N]^r \mid M(\vec{j}_1, \dots, \vec{j}_{i-1}, \vec{j}, \vec{j}_{i+1}, \dots, \vec{j}_U) = (1, i, \cdot) \right\} \right|, \\ a_{i,n} : ([N]^r)^U &\rightarrow \mathbb{N}_{\geq 0}, \\ (\vec{j}_1, \dots, \vec{j}_U) &\mapsto \left| \left\{ \vec{j} \in [N]^r \mid \begin{array}{l} \vec{j} = (j_{i,1}, \dots, j_{i,n-1}, j, j_{i,n+1}, \dots, j_{i,r}), \\ M(\vec{j}_1, \dots, \vec{j}_{i-1}, \vec{j}, \vec{j}_{i+1}, \dots, \vec{j}_U) = (1, i, \cdot) \end{array} \right\} \right|. \end{aligned}$$

$a_i(\vec{j}_1, \dots, \vec{j}_U)$ counts the number of entries with $v = 1$ and index i in the r -dimensional array $M(\vec{j}_1, \dots, \vec{j}_{i-1}, \cdot, \vec{j}_{i+1}, \dots, \vec{j}_U)$, while $a_{i,n}(\vec{j}_1, \dots, \vec{j}_U)$ counts the number of such entries in the 1-dimensional subarray where all coordinates except the n -th are fixed. Hence, $a_{i,n}(\vec{j}_1, \dots, \vec{j}_U) \leq a_i(\vec{j}_1, \dots, \vec{j}_U)$ for all $n \in [r]$.

Lemma H.6. *Consider the game in Game 4. Let $\vec{J} = (\vec{J}_1, \dots, \vec{J}_U)$ be uniformly distributed in $([N]^r)^U$, indicating the first entry sampled, and let $(V, I, T) = M(\vec{J})$. For each $i \in [U]$ and $n \in [r]$, let $A_i = a_i(\vec{J})$ and $A_{i,n} = a_{i,n}(\vec{J})$. Let $F_{\ell,n}^{\text{chal}}$ be the random variable indicating the outcome of $f_{\ell,n}^{\text{chal}}$. Let (F^{prop}, Y) denote the outcome of f^{prop} and F_n^{bind} the outcome of f_n^{bind} . Let $d_1^{\text{chal}}, \dots, d_k^{\text{chal}}, d^{\text{com}} \in [0, 1]$ be numbers such that for all $i \in [U], n \in [r]$ and all $a \in \mathbb{N}, k \leq a \leq N$,*

$$\begin{aligned} \frac{d_{\ell}^{\text{chal}} N}{a - \ell + 1} &\geq \Pr \left[F_{\ell,n}^{\text{chal}} = 0 \mid \begin{array}{l} V = 1, I = i, A_{i,n} = a, \bigcap_{n' \neq n} A_{i,n'} \geq k, \\ \bigcap_{\ell'=1}^{\ell-1} \bigcap_{n'=1}^r F_{\ell',n'}^{\text{chal}} = 1 \end{array} \right] \text{ and} \\ \frac{d^{\text{com}} N}{a - k + 1} &\geq \Pr \left[F_n^{\text{bind}} = 1 \mid \begin{array}{l} V = 1, I = i, A_{i,n} = a, \bigcap_{n' \neq n} A_{i,n'} \geq k, \\ \bigcap_{\ell=1}^k \bigcap_{n'=1}^r F_{\ell,n'}^{\text{chal}} = 1, F^{\text{prop}} = 0, Y = n \end{array} \right]. \end{aligned}$$

Finally, set some tuning parameter $1 \leq \sigma \leq N/k$ such that $\sigma k \in \mathbb{N}$. Then the game outputs 1 with probability at least

$$\Pr[V = 1] - rA \cdot \max \left(\frac{\sigma k - 1}{N}, \frac{\sigma k}{\sigma k - k + 1} d^{\text{com}} + \sum_{\ell=1}^k \frac{\sigma k}{\sigma k - \ell + 1} d_{\ell}^{\text{chal}} \right),$$

where $A = \sum_{i=1}^U \Pr[A_i > 0]$.

Proof. For the sake of conciseness, we let E_i denote the event that $V = 1, I = i$. By the description of the abstract game, we have that the probability that the game outputs 1 is

$$\sum_{i=1}^U \Pr \left[E_i, \bigcap_{n=1}^r A_{i,n} \geq k, \bigcap_{\ell=1}^k \bigcap_{n=1}^r F_{\ell,n}^{\text{chal}} = 1, \left(F^{\text{prop}} = 1 \cup \bigcup_{n=1}^r F_n^{\text{bind}} = 0 \right) \right]$$

This probability can be decomposed as follows.

$$= \Pr[V = 1] - \sum_{i=1}^U \Pr \left[E_i, \bigcup_{n=1}^r A_{i,n} < \sigma k \right] \tag{14}$$

$$- \sum_{i=1}^U \Pr \left[E_i, \bigcap_{n=1}^r A_{i,n} \geq \sigma k, \bigcup_{\ell=1}^k \bigcup_{n=1}^r F_{\ell,n}^{\text{chal}} = 0 \right] \tag{15}$$

$$- \sum_{i=1}^U \Pr \left[E_i, \bigcap_{n=1}^r A_{i,n} \geq \sigma k, \bigcap_{\ell=1}^k \bigcap_{n=1}^r F_{\ell,n}^{\text{chal}} = 1, f^{\text{prop}} = 0, \bigcap_{n=1}^r f_n^{\text{bind}} = 1 \right] \tag{16}$$

We upper bound each of the three sums individually. For the first sum (14), after applying a union bound, the argument is the same for each $A_{i,n}$.

$$\begin{aligned}
\sum_{i=1}^U \Pr \left[E_i, \bigcup_{n=1}^r A_{i,n} < \sigma k \right] &\leq \sum_{n=1}^r \sum_{i=1}^U \Pr [E_i, A_{i,n} < \sigma k] \\
&= \sum_{n=1}^r \sum_{i=1}^U \sum_{a=0}^{\sigma k-1} \Pr [A_{i,n} = a] \frac{a}{N} \\
&\leq \sum_{n=1}^r \sum_{i=1}^U \sum_{a=1}^{\sigma k-1} \Pr [A_{i,n} = a] \frac{\sigma k - 1}{N}
\end{aligned}$$

Next we bound the second sum (15).

$$\begin{aligned}
&\sum_{i=1}^U \Pr \left[E_i, \bigcap_{n=1}^r A_{i,n} \geq \sigma k, \bigcup_{\ell=1}^k \bigcup_{n=1}^r F_{\ell,n}^{\text{chal}} = 0 \right] \\
&= \sum_{i=1}^U \Pr \left[E_i, \bigcap_{n=1}^r A_{i,n} \geq \sigma k, \bigcup_{\ell=1}^k \bigcup_{n=1}^r \left(F_{\ell,n}^{\text{chal}} = 0, \bigcap_{\ell'=1}^{\ell-1} \bigcap_{n'=1}^r F_{\ell',n'}^{\text{chal}} = 1 \right) \right] \\
&\leq \sum_{\ell=1}^k \sum_{n=1}^r \sum_{i=1}^U \Pr \left[E_i, \bigcap_{n'=1}^r A_{i,n'} \geq \sigma k, \bigcap_{\ell'=1}^{\ell-1} \bigcap_{n'=1}^r F_{\ell',n'}^{\text{chal}} = 1, F_{\ell,n}^{\text{chal}} = 0 \right] \\
&\leq \sum_{\ell=1}^k \sum_{n=1}^r \sum_{i=1}^U \Pr \left[E_i, A_{i,n} \geq \sigma k, \bigcap_{n' \neq n} A_{i,n'} \geq k, \bigcap_{\ell'=1}^{\ell-1} \bigcap_{n'=1}^r F_{\ell',n'}^{\text{chal}} = 1, F_{\ell,n}^{\text{chal}} = 0 \right] \\
&\leq \sum_{n=1}^r \sum_{i=1}^U \sum_{a=\sigma k}^N \Pr [A_{i,n} = a] \cdot \Pr [E_i \mid A_{i,n} = a] \\
&\quad \cdot \sum_{\ell=1}^k \Pr \left[\bigcap_{n' \neq n} A_{i,n'} \geq k, \bigcap_{\ell'=1}^{\ell-1} \bigcap_{n'=1}^r F_{\ell',n'}^{\text{chal}} = 1 \mid E_i, A_{i,n} = a \right] \\
&\quad \cdot \Pr \left[F_{\ell,n}^{\text{chal}} = 0 \mid E_i, A_{i,n} = a, \bigcap_{n' \neq n} A_{i,n'} \geq k, \bigcap_{\ell'=1}^{\ell-1} \bigcap_{n'=1}^r F_{\ell',n'}^{\text{chal}} = 1 \right] \\
&\leq \sum_{n=1}^r \sum_{i=1}^U \sum_{a=\sigma k}^N \Pr [A_{i,n} = a] \cdot \frac{a}{N} \sum_{\ell=1}^k \frac{d_{\ell}^{\text{chal}} N}{a - \ell + 1} \\
&\leq \sum_{n=1}^r \sum_{i=1}^U \sum_{a=\sigma k}^N \Pr [A_{i,n} = a] \sum_{\ell=1}^k \frac{\sigma k}{\sigma k - \ell + 1} d_{\ell}^{\text{chal}}
\end{aligned}$$

Finally, we bound the third sum (16).

$$\begin{aligned}
& \sum_{y=1}^r \sum_{i=1}^U \Pr \left[E_i, \bigcap_{n=1}^r A_{i,n} \geq \sigma k, \bigcap_{\ell=1}^k \bigcap_{n=1}^r F_{\ell,n}^{\text{chal}} = 1, f^{\text{prop}} = 0, Y = y, \bigcap_{n=1}^r f_n^{\text{bind}} = 1 \right] \\
& \leq \sum_{y=1}^r \sum_{i=1}^U \sum_{a=\sigma k}^N \Pr [A_{i,y} = a] \cdot \Pr [E_i \mid A_{i,y} = a] \\
& \quad \cdot \Pr \left[F_y^{\text{bind}} = 1 \mid E_i, A_{i,y} = a, \bigcap_{n \neq y} A_{i,n} \geq k, \bigcap_{\ell=1}^k \bigcap_{n=1}^r F_{\ell,n}^{\text{chal}} = 1, F^{\text{prop}} = 0, Y = y \right] \\
& \leq \sum_{y=1}^r \sum_{i=1}^U \sum_{a=\sigma k}^N \Pr [A_{i,y} = a] \frac{a}{N} \cdot \frac{d^{\text{com}} N}{a - k + 1} \\
& \leq \sum_{y=1}^r \sum_{i=1}^U \sum_{a=\sigma k}^N \Pr [A_{i,y} = a] \frac{\sigma k}{\sigma k - k + 1} d^{\text{com}} \\
& = \sum_{n=1}^r \sum_{i=1}^U \sum_{a=\sigma k}^N \Pr [A_{i,n} = a] \frac{\sigma k}{\sigma k - k + 1} d^{\text{com}}
\end{aligned}$$

Combining the bounds for the three sums (14, 15, 16) and using that $\Pr [A_{i,n} > 0] \leq \Pr [A_i > 0]$ for all $n \in [r]$, the lemma follows. $\square$

Lemma H.7 (Expected cost [FMN23, Lemma 8.1]). *Consider the game in Game 4, as well as a cost function $\Gamma : [N]^U \rightarrow \mathbb{R}_{\geq 0}$ and a constant cost $\gamma \in \mathbb{R}_{\geq 0}$. Let $\vec{J} = (\vec{J}_1, \dots, \vec{J}_U)$ be uniformly distributed in $([N]^r)^U$, indicating the first entry sampled, and let $(V, I, T) = M(\vec{J})$. For each $i \in [U]$, let $A_i = a_i(\vec{J})$. We define the cost of sampling an entry $M(j_1, \dots, j_U) = (v, i, t)$ with index $i = I$ to be $\Gamma(j_1, \dots, j_U)$ and the cost of sampling an entry $M(j_1, \dots, j_U) = (v, i, t)$ with $i \neq I$ to be γ . Let Δ be the total cost of playing this game. Then*

$$\mathbb{E}[\Delta] \leq (1 + r(k-1)) \cdot \mathbb{E}[\Gamma(\vec{J})] + r(k-1) \cdot A' \cdot \gamma,$$

where $A' = \sum_{i=1}^U \Pr [I \neq i, A_i > 0] \leq A$.

From this abstract game, an extractor for coordinate-wise predicate-special-sound protocols can be constructed in the same way as in Section H.2. We let $N = |S|$ be the cardinality of the base challenge space. $f_{\ell,n}^{\text{chal}}$ is instantiated as the ℓ -th challenge predicate evaluated in the n -th coordinate. Note that there is a slight mismatch in their inputs. In the framework, the ℓ -th challenge predicate gets the $\ell - 1$ first challenge vectors for every coordinate, $(\vec{s}_1, \vec{c}_2, \dots, \vec{c}_{\ell-1})$. However, in the abstract game, $f_{\ell,n}^{\text{chal}}$ only gets $\vec{s}_1$ and $s_{\ell',n,n}$ for every $\ell' \in [\ell] \setminus \{1\}, n \in [r]$, it does not get the rest of $\vec{s}_{\ell',n}$. But since all the other entries of $\vec{s}_{\ell',n}$ have to be equal to those in $\vec{s}_1$, we do not need to pass them to the challenge predicate.

f_n^{bind} is the binding predicate evaluated in the n -th coordinate. Using Definition G.6, we let the $y \in [r]$ output by f^{prop} indicate the coordinate where we can use the failure density. Thus, Definition G.5 and Definition G.6 imply that we can set $d_\ell^{\text{chal}} = p_{m,\ell}^{\text{chal}}$ for all $\ell \in [k_m]$ and $d_m^{\text{com}} = p_m^{\text{com}}$. In addition, we can once again bound $A = \sum_{i=1}^U \Pr [A_i > 0] \leq (Q+1)$ using Lemma H.3. In conclusion, by using Lemma H.6 and Lemma H.7, we obtain the following.

Proposition H.1. *Let $\Pi = (\mathcal{P}, \mathcal{V})$ be a $(\mathbf{R}, \mathbf{K}, \Phi)$ -coordinate-wise predicate-special-sound argument of knowledge for a relation R_{pp} . In addition, let $R_{\text{bind,pp}}$ be a coordinate-wise binding relation for Φ . Consider the adaptive Fiat-Shamir transformation $\text{FS}[\Pi]$ of Π . There exists a knowledge extractor for the relation $R_{\text{pp}} \cup R_{\text{bind,pp}}$, which given black-box access to an adaptive Q -query random oracle prover $\mathcal{P}^*$ for $\text{FS}[\Pi]$, succeeds with probability at least*

$$\varepsilon(\mathcal{P}^*) - (Q+1) \sum_{i=1}^{\mu} r_i \min_{\sigma_i \in [1, \frac{N}{k_i}]: \sigma_i k_i \in \mathbb{N}} \kappa_i(\sigma_i),$$

where

$$\kappa_i(\sigma) = \max \left(\frac{\sigma k_i - 1}{N}, \frac{\sigma k_i}{\sigma k_i - k_i + 1} p_i^{\text{com}} + \sum_{\ell=1}^{k_i} \frac{\sigma k_i}{\sigma k_i - \ell + 1} p_{i,\ell}^{\text{chal}} \right).$$

The number of times that the knowledge extractor invokes $\mathcal{P}^*$ is in expectation at most $K_m + Q(K_m - 1)$, where $K = \prod_{i=1}^{\mu} (r_i(k_i - 1) + 1)$

I Proof of Non-Interactive Knowledge Soundness of LaBRADOR

In this section, we provide the first *full* proof of non-interactive knowledge soundness of LaBRADOR [BS23]. In our soundness analysis of LaBRADOR, we initially focus on a single iteration. Due to the structure of PSS, the soundness of recursively composed protocols then follows easily. A single iteration of LaBRADOR is $(\mathbf{R}, \mathbf{K}, \Phi)$ -coordinate-wise predicate-special-sound, with $\mathbf{R} = (1, 1, 1, r)$, $\mathbf{K} = (2, 2, 2, 3)$ and r the number of witnesses. We now define the predicate system Φ from the bottom up, for the sake of simplicity we treat rounds which only have one coordinate as if they were simply predicate-special-sound.

We are interested in the case where LaBRADOR is instantiated with a ring $\mathcal{R}_q$ of degree d with splitting factor l and a B -well-spread challenge set, where each challenge has operator norm at most T_{op} . Our proof may be easily adapted for a ring that permits a challenge space where challenge differences are always invertible, as it was the case for the original LaBRADOR [BS23]. For our statements we require an ℓ_2 -norm bound $\sqrt{\lambda/C_2}\beta$ at most q/C_1 , where C_1, C_2 are the parameters of Lemma 2.2.

I.1 Analysing Levels

Recall that LaBRADOR is a $(2\mu + 1)$ -message public-key argument of knowledge with $\mu = 4$. Working bottom up, we define predicates for one layer at a time. To analyze the failure density of the predicates on some level, we may by definition assume that the predicates on the levels below it are satisfied. Thus, we incrementally enforce stronger properties on the extracted candidate witness.

We begin with a single transcript $t_5 \in \mathbb{T}_5$. As always, if t_5 is an accepting transcript then $\Phi_5(t_5) = 1$. For all remaining levels ($i = 1, 2, 3, 4$), we set $\Phi_i^{\text{com}} = (\Phi_i^{\text{prop}}, \Phi_i^{\text{bind}})$.

I.1.1 Level 4. This round contains the amortized opening to the commitment, and is the only level which makes use of the coordinate-wise extension of PSS and non-trivial challenge predicates. In particular, we have one coordinate for each witness vector we wish to extract, giving r coordinates in total. Let $(t_1, \vec{t}_2, \vec{t}_3) \in \mathbb{T}_4$ be the extracted tree with $\vec{t}_2 = (t_{2,1}, \dots, t_{2,r}) \in \mathbb{T}_5^r$ and $\vec{t}_3 = (t_{3,1}, \dots, t_{3,r}) \in \mathbb{T}_5^r$, such that $t_1, t_{2,i}, t_{3,i}$ are the transcripts for the i th coordinate. We let $\mathbf{c}_{1,1}, \dots, \mathbf{c}_{1,r}$ be the level 4 challenges of t_1 , and let $\mathbf{c}_{i,j}$ be the j -th challenge of $t_{i,j}$ for $i \in \{2, 3\}, j \in [r]$. Note that the other challenges of the $t_{i,j}$ are the same as those in t_1 .

To later enforce that the $\mathbf{g}_{i,j}$ garbage polynomials are computed correctly, our trick is to enforce that all the first challenges are units. For $i \in [r]$,

$$\Phi_{4,1}^{\text{chal}}(i, \mathbf{c}_{1,i}) = 1 \Leftrightarrow \mathbf{c}_{1,i} \in \mathcal{R}_q^\times.$$

We also require that the difference between the first two challenges in each coordinate is invertible, so that we can compute weak openings for the inner commitments. For $i \in [r]$,

$$\Phi_{4,2}^{\text{chal}}((\mathbf{c}_{1,1}, \dots, \mathbf{c}_{1,r}), i, \mathbf{c}_{2,i}) = 1 \Leftrightarrow (\mathbf{c}_{1,i} - \mathbf{c}_{2,i}) \in \mathcal{R}_q^\times.$$

Next, we define the commitment predicate to ensure that each garbage polynomial is computed correctly. We introduce some definition to describe the elements in each transcript. For $t \in \mathbb{T}_5$, we let $\vec{z}(t)$, $\vec{v}_i(t)$, $\mathbf{g}_{i,j}(t)$, $\mathbf{h}_{i,j}(t)$ and $\vec{\varphi}_i(t)$ denote the corresponding values in the transcript t . Let

$$\bar{\mathbf{c}}_i = \mathbf{c}_{1,i} - \mathbf{c}_{2,i} \text{ and } \vec{\mathbf{w}}_i^* = \bar{\mathbf{c}}_i^{-1} (\vec{z}(t_1) - \vec{z}(t_{2,i})) \quad \forall i \in [r].$$

Recall that by the LaBRADOR verification algorithm Protocol 3, $\|\vec{z}(t)\|_2 \leq (b+1)\beta'$. When the inner commitments are the same in t_1 and $t_{2,i}$, $\bar{\mathbf{c}}_i$ and $\vec{\mathbf{w}}_i^*$ form a weak opening for $\vec{v}_i$ of norm $2(b+1)\beta'$. Furthermore, for $i, j \in [r]$, we need the prover to be bound to

$$\begin{aligned} \vec{\mathbf{y}}_i(t) &= \vec{z}(t) - \text{chal}_{4,i}(t) \vec{\mathbf{w}}_i^*, \\ \vec{\mathbf{y}}_{i,j}(t) &= \vec{z}(t) - \text{chal}_{4,i}(t) \vec{\mathbf{w}}_i^* - \text{chal}_{4,j}(t) \vec{\mathbf{w}}_j^*, \end{aligned}$$

where $\text{chal}_{4,i}(t)$ outputs the i th coordinate challenge for the fourth level. Finally, we define an extraction algorithm for each coordinate $\ell \in [r]$,

$$E_5(t, \ell) = \left((\vec{v}_i(t), \mathbf{g}_{i,j}(t), \mathbf{h}_{i,j}(t))_{i,j \in [r]}, \vec{\mathbf{y}}_\ell(t), (\vec{\mathbf{y}}_{\ell,j}(t))_{j \in [r], \ell \neq j} \right).$$

The property predicate ensures that the garbage polynomials are computed correctly with respect to the $\vec{\mathbf{w}}_1^*, \dots, \vec{\mathbf{w}}_r^*$ computed from t_1 and $\vec{t}_2$.

$$\Phi_4^{\text{prop}}(t_1, \vec{t}_2) = 1 \Leftrightarrow \forall i, j \in [r] : \mathbf{h}_{i,i}(t_1) = \langle \vec{\varphi}_i(t_1), \vec{\mathbf{w}}_i^* \rangle \wedge \mathbf{g}_{i,j}(t_1) = \langle \vec{\mathbf{w}}_i^*, \vec{\mathbf{w}}_j^* \rangle.$$

The binding predicate states that the relevant openings should be consistent across transcripts.

$$\Phi_4^{\text{bind}}((t_1, \vec{t}_2), \ell, t_{3,\ell}) = 1 \Leftrightarrow E_5(t_1, \ell) = E_5(t_{2,\ell}, \ell) = E_5(t_{3,\ell}, \ell).$$

In preparation for the next level, for $t_4 = (t_1, \vec{t}_2, \vec{t}_3)$ we let $E_4(t_4) = (\vec{w}_i^*, \vec{c}_i)_{i \in [r]}$.

Lemma I.1. *In the fourth level $\Phi_{4,1}^{\text{chal}}$ has failure density $p_{4,1}^{\text{chal}} = lB$, $\Phi_{4,2}^{\text{chal}}$ has failure density $p_{4,2}^{\text{chal}} = lB$ and Φ_4^{com} has failure density $p_4^{\text{com}} = 5B$.*

Proof. We address each of the predicates individually, starting with $\Phi_{4,1}^{\text{chal}}$. Given a fixed $\mathbf{c}^* \in \mathcal{R}_q$, Lemma 3.1 tells us that

$$\Pr_{\mathbf{c} \leftarrow \mathcal{C}} [\mathbf{c}^* - \mathbf{c} \notin \mathcal{R}_q^\times] \leq lB.$$

Letting $\mathbf{c}^* = \mathbf{0}$, it follows that a uniformly sampled challenge in $\mathcal{C}$ is a unit with probability less than lB . Hence, there are at most $lB|\mathcal{C}|$ non-units in $\mathcal{C}$, meaning that the failure density is $p_{4,1}^{\text{chal}} = lB$.

A similar argument may be applied for $\Phi_{4,2}^{\text{chal}}$ in each coordinate. Let $\mathbf{c}^* = \mathbf{c}_{1,i}$, then the lemma implies that at most $lB|\mathcal{C}|$ of the challenges $\mathbf{c} \in \mathcal{C}$ are such that $\mathbf{c}_{1,i} - \mathbf{c} \notin \mathcal{R}_q^\times$. One of these is $\mathbf{c}_{1,i}$ itself, which we can exclude as it will not be sampled again by the extractor. Thus, we obtain the failure density $lB - 1/|\mathcal{C}|$. However, for simplicity, we upper bound the failure density of $\Phi_{4,2}^{\text{chal}}$ by $p_{4,2}^{\text{chal}} = lB$.

We now proceed to Φ_4^{com} . To derive the failure density of this predicate, we analyze the $\mathbf{h}_{i,i}$ and $\mathbf{g}_{i,j}$ separately. By a union bound, the failure density of this predicate is upper bounded by the sum of the failure density of each case.

Assume for some index $i_0 \in [r]$, $\mathbf{h}_{i_0,i_0}(t_1) \neq \langle \vec{\varphi}_{i_0}(t_1), \vec{w}_{i_0}^* \rangle$. Then we upper bound the number of choices of the challenge $\mathbf{c}_{i_0}$ in the i_0 th coordinate which could lead to an accepting transcript without breaking binding. To do this, we are going to use the Schwartz-Zippel lemma (Lemma 3.3). We are going to redefine the verification equation for the $\mathbf{h}_{i,j}$ as a non-zero polynomial which has $\mathbf{c}_{i_0}$ as a root. The verification algorithm for a $t \in \mathbb{T}_5$ has the following check for the $\mathbf{h}_{i,j}$.

$$\sum_{i=1}^r \langle \vec{\varphi}_i(t), \vec{z}(t) \rangle \mathbf{c}_i \stackrel{?}{=} \sum_{i,j=1}^r \mathbf{h}_{i,j}(t) \mathbf{c}_i \mathbf{c}_j,$$

with $\mathbf{c}_i = \text{chal}_{4,i}(t)$. The polynomial for the Schwartz-Zippel test is obtained by rearranging this equation, substituting $\vec{z}(t)$ with $\vec{\mathbf{y}}_{i_0}(t) + \mathbf{c}_{i_0} \vec{w}_{i_0}^*$ and setting the i_0 -th challenge to be the variable X_{i_0} .

$$\begin{aligned} & \left(\sum_{\substack{1 \leq i,j \leq r \\ i,j \neq i_0}} \mathbf{h}_{i,j}(t) \mathbf{c}_i \mathbf{c}_j - \sum_{\substack{1 \leq i \leq r \\ i \neq i_0}} \langle \vec{\varphi}_i(t), \vec{\mathbf{y}}_{i_0}(t) \rangle \mathbf{c}_i \right) \\ & + \left(\sum_{\substack{1 \leq i \leq r \\ i \neq i_0}} (\mathbf{h}_{i,i_0}(t) + \mathbf{h}_{i_0,i}(t)) \mathbf{c}_i - \langle \vec{\varphi}_{i_0}(t), \vec{\mathbf{y}}_{i_0}(t) \rangle - \sum_{\substack{1 \leq i \leq r \\ i \neq i_0}} \langle \vec{\varphi}_i(t), \vec{w}_{i_0}^* \rangle \mathbf{c}_i \right) X_{i_0} \\ & + (\mathbf{h}_{i_0,i_0}(t) - \langle \vec{\varphi}_{i_0}(t), \vec{w}_{i_0}^* \rangle) X_{i_0}^2 \stackrel{?}{=} \mathbf{0}. \end{aligned}$$

The left hand side is a polynomial over $\mathcal{R}_q[X_{i_0}]$ of degree 2, as the leading coefficient by assumption is not $\mathbf{0}$. For t to be accepting, the i_0 -th challenge $\mathbf{c}_{i_0}$ must be a root of this polynomial. Given $(t_1, \vec{t}_2)$, we have fixed all the values in the coefficients of this polynomial. The challenges $\mathbf{c}_j$ for $j \neq i_0$ are fixed by the coordinate-wise extraction. The elements $\vec{w}_1^*, \dots, \vec{w}_r^*$ are just the constants we chose to define $\vec{\mathbf{y}}_i$. The other values are fixed by the binding predicate. Thus, if the third transcript t_{3,i_0} succeeds without breaking binding, it must do so using a $\mathbf{c}_{i_0}$ that is a root of this *fixed* polynomial.

By Lemma 3.3, the probability that a challenge that was sampled independently and uniformly at random from $\mathcal{C}$ is a root of a fixed polynomial of degree 2 is at most $2B$. It follows that there can be at most $2B \cdot |\mathcal{C}|$ roots of the polynomial. Thus, the failure density for this case is $2B$.

Next, assume $\mathbf{g}_{i_0,i_0} \neq \langle \vec{w}_{i_0}^*, \vec{w}_{i_0}^* \rangle$. Then the argument is exactly the same as before. The verification algorithm checks that the $\mathbf{g}_{i,j}$ in a transcript $t \in \mathbb{T}_5$ satisfy the equation

$$\langle \vec{z}(t), \vec{z}(t) \rangle \stackrel{?}{=} \sum_{i,j=1}^r \mathbf{g}_{i,j}(t) \mathbf{c}_i \mathbf{c}_j, \tag{17}$$

where $\mathbf{c}_i = \text{chal}_{4,i}(t)$. This equation can be reformulated to a non-zero polynomial of degree 2 which must have $\mathbf{c}_{i_0}$ as a root in order for the transcript to be accepting. Thus, we get the failure density $2B$ for this case too.

Finally, assume $\mathbf{g}_{i_0,j_0}(t_1) \neq \langle \vec{\mathbf{w}}_{i_0}^*, \vec{\mathbf{w}}_{j_0}^* \rangle$ for a $j_0 \neq i_0$, but that all the $\mathbf{g}_{i,i}(t_1)$ were computed correctly. By reordering (17), substituting $\vec{\mathbf{z}}(t)$ with $\vec{\mathbf{y}}_{i_0,j_0}(t) + \mathbf{c}_{i_0}\vec{\mathbf{w}}_{i_0}^* + \mathbf{c}_{j_0}\vec{\mathbf{w}}_{j_0}^*$ and setting $\mathbf{c}_{i_0}$ to be X_{i_0} and $\mathbf{c}_{j_0}$ to be X_{j_0} , we obtain the following bivariate polynomial.

$$\begin{aligned} & \left(\sum_{i,j \in \{i_0,j_0\}} \mathbf{g}_{i,j}(t) \mathbf{c}_i \mathbf{c}_j - \langle \vec{\mathbf{y}}_{i_0,j_0}(t), \vec{\mathbf{y}}_{i_0,j_0}(t) \rangle \right) \\ & + 2 \left(\sum_{i \in \{i_0,j_0\}} \mathbf{g}_{i_0,i}(t) \mathbf{c}_i - \langle \vec{\mathbf{y}}_{i_0,j_0}(t), \vec{\mathbf{w}}_{i_0}^* \rangle \right) X_{i_0} \\ & + 2 \left(\sum_{i \in \{i_0,j_0\}} \mathbf{g}_{j_0,i}(t) \mathbf{c}_i - \langle \vec{\mathbf{y}}_{i_0,j_0}(t), \vec{\mathbf{w}}_{j_0}^* \rangle \right) X_{j_0} \\ & + 2 (\mathbf{g}_{i_0,j_0}(t) - \langle \vec{\mathbf{w}}_{i_0}^*, \vec{\mathbf{w}}_{j_0}^* \rangle) X_{i_0} X_{j_0} \\ & + (\mathbf{g}_{i_0,i_0}(t) - \langle \vec{\mathbf{w}}_{i_0}^*, \vec{\mathbf{w}}_{i_0}^* \rangle) X_{i_0}^2 + (\mathbf{g}_{j_0,j_0}(t) - \langle \vec{\mathbf{w}}_{j_0}^*, \vec{\mathbf{w}}_{j_0}^* \rangle) X_{j_0}^2. \end{aligned}$$

The coefficients of this polynomial are once again fixed by binding and coordinate-wise extraction.

The degree of the polynomial is 2, since $\mathbf{g}_{i_0,j_0}(t_1) - \langle \vec{\mathbf{w}}_{i_0}^*, \vec{\mathbf{w}}_{j_0}^* \rangle \neq 0$. However, when extracting in the i_0 -th coordinate, we always use the same j_0 -th coordinate $\mathbf{c}_{j_0}$. Thus, we do not evaluate the polynomial in a j_0 -th challenge chosen independently of the polynomial, which is a requirement to use the Schwartz-Zippel lemma.

We circumvent this by considering the polynomial with X_{j_0} evaluated in $\mathbf{c}_{j_0}$. Recall, by $\Phi_{4,1}^{\text{chal}}$ we know that the challenge in each coordinate is a unit $\mathbf{c}_{j_0} \in \mathcal{R}_q^\times$. As a result $(\mathbf{g}_{i_0,j_0}(t) - \langle \vec{\mathbf{w}}_{i_0}^*, \vec{\mathbf{w}}_{j_0}^* \rangle) \mathbf{c}_{j_0} \neq 0$, meaning that the remaining polynomial is not the zero polynomial. This allows us to consider a polynomial in $\mathcal{R}_q[X_{i_0}]$ of degree 1 with coefficients independent of $\mathbf{c}_{i_0}$. By the Schwartz-Zippel lemma, it follows that the failure density in this case is B . Summing over all cases, we obtain $p_4^{\text{com}} = 5B$. $\square$

I.1.2 Level 3. Now we proceed to the third level. Here individual constraints are aggregated to form one single function F . It is computed as the random linear combination of the K constraints in $\mathcal{F}$ and the $K'' = \lceil \lambda / \log q \rceil$ aggregated constant term dot product constraints.

$$\begin{aligned} F(\vec{\mathbf{w}}_1^*, \dots, \vec{\mathbf{w}}_r^*) &= \sum_{k=1}^K \alpha_k f^{(k)}(\vec{\mathbf{w}}_1^*, \dots, \vec{\mathbf{w}}_r^*) + \sum_{k=1}^{K''} \beta_k f''^{(k)}(\vec{\mathbf{w}}_1^*, \dots, \vec{\mathbf{w}}_r^*) \\ &= \sum_{i,j}^r \mathbf{a}_{i,j} \langle \vec{\mathbf{w}}_i^*, \vec{\mathbf{w}}_j^* \rangle + \sum_{i=1}^r \langle \vec{\boldsymbol{\varphi}}_i, \vec{\mathbf{w}}_i^* \rangle - \mathbf{b}. \end{aligned}$$

The verification algorithm has the following check.

$$\sum_{i,j=1}^r \mathbf{a}_{i,j} \mathbf{g}_{i,j} + \sum_{i=1}^r \mathbf{h}_{i,i} - \mathbf{b} \stackrel{?}{=} \mathbf{0}. \quad (18)$$

We wish to ensure that all of the aggregated functions evaluate to $\mathbf{0}$. Given $t \in \mathbb{T}_4$ where $\Phi(t) = 1$, we know E_4 will extract weak openings for the inner commitments.

$$\Phi_3^{\text{prop}}(t_1) = 1 \Leftrightarrow \begin{cases} (\vec{\mathbf{w}}_i^*, \vec{\mathbf{c}}_i)_{i \in [r]} \leftarrow E_4(t_1), \\ \forall f \in \mathcal{F} : f(\vec{\mathbf{w}}_1^*, \dots, \vec{\mathbf{w}}_r^*) = \mathbf{0}, \\ \forall f'' \in \mathcal{F}'' : f''(\vec{\mathbf{w}}_1^*, \dots, \vec{\mathbf{w}}_r^*) = \mathbf{0}. \end{cases}$$

Binding holds when the extracted witnesses and inner commitments are the same in each subtree. We make the same requirements for rounds $\ell = 1, 2, 3$:

$$\Phi_\ell^{\text{bind}}(t_{\ell+1}^1, t_{\ell+1}^2) = 1 \Leftrightarrow \begin{cases} (\vec{\mathbf{w}}_{i,k}^*, \vec{\mathbf{c}}_{i,k})_{i \in [r]} \leftarrow E_{\ell+1}(t_{\ell+1}^k) \text{ for } k \in \{1, 2\} \\ \forall i \in [r], \vec{\mathbf{w}}_{i,1}^* = \vec{\mathbf{w}}_{i,2}^*, \vec{\mathbf{v}}_{i,1} = \vec{\mathbf{v}}_{i,2} \end{cases}$$

Here $\vec{v}_{i,1}, \vec{v}_{i,2}$ denotes the i th inner commitment in respectively $t_{\ell+1}^1$ and $t_{\ell+1}^2$. Note that our analysis assumes the predicates on the lower levels are satisfied, so that in particular, the prover does not violate binding with respect to the inner commitments in $t_{\ell+1}^1$ or $t_{\ell+1}^2$. Hence, there is a unique inner commitment in each subtree. where for $t_\ell = (t_{\ell+1}^1, t_{\ell+1}^2)$ we let $E_\ell(t_\ell) = E_{\ell+1}(t_{\ell+1}^1)$.

Lemma I.2. *The failure density of Φ_3^{com} in Φ is $p_3^{\text{com}} = q^{-d/l}$.*

Proof. Assume Φ_3^{prop} does not hold. The analysis of the functions in $\mathcal{F}$ and $\mathcal{F}'$ is the same, so without loss of generality, we focus on the former. Then there is some $f^{(k_0)} \in \mathcal{F}$ such that $f^{(k_0)}(\vec{w}_1^*, \dots, \vec{w}_r^*) \neq \mathbf{0}$.

Let us bound the fraction of challenges that can be used to produce a second valid tree without breaking binding. The evaluation of $f^{(k_0)}(\vec{w}_1^*, \dots, \vec{w}_r^*)$ is fixed by the binding predicate Φ_4^{bind} . Let

$$F(\vec{w}_1^*, \dots, \vec{w}_r^*) = \alpha_{k_0} f^{(k_0)}(\vec{w}_1^*, \dots, \vec{w}_r^*) + F'(\vec{w}_1^*, \dots, \vec{w}_r^*),$$

where F' is the sum of the other terms. Due to Φ_4^{com} we know that if $\Phi_4(t_2) = 1$, we must have $\mathbf{g}_{i,j} = \langle \vec{w}_i^*, \vec{w}_j^* \rangle$ and $\mathbf{h}_{i,i} = \langle \vec{\varphi}_i, \vec{w}_i^* \rangle$. Thus, Equation 18 must hold for t_2 , meaning $F(\vec{w}_1^*, \dots, \vec{w}_r^*) = \mathbf{0}$.

For F to evaluate to $\mathbf{0}$ over $\mathcal{R}_q$, it must evaluate to 0 in every CRT slot. $f^{(k_0)}(\vec{w}^*) \neq \mathbf{0}$ over $\mathcal{R}_q$ means that there is some $i \in \{1, \dots, l\}$ such that its i th CRT slot is non-zero. Since the evaluation of $f^{(k_0)}$ is fixed by binding, this slot i is fixed.

Imagine that $\vec{\alpha}, \vec{\beta}$ were chosen independently and uniformly at random. The probability over the choice of $\vec{\alpha}, \vec{\beta}$ that F evaluates to $\mathbf{0}$ in $\mathcal{R}_q$ is upper bounded by the probability that it evaluates to $\mathbf{0}$ in the i th CRT slot. Each CRT slot is a field of order $q^{d/l}$. In a field, the product of an independent uniformly random element with a non-zero constant is still independently uniformly random. Hence, $\alpha_{k_0} f^{(k_0)}(\vec{w}_1^*, \dots, \vec{w}_r^*)$ is an independently distributed uniformly random term of $F(\vec{w}_1^*, \dots, \vec{w}_r^*)$ in the i th CRT slot. This means that the evaluation of F in the i th CRT slot is uniformly random over the choice of $\vec{\alpha}, \vec{\beta}$. Thus, $F(\vec{w}_1^*, \dots, \vec{w}_r^*) = \mathbf{0} \pmod{(X^{d/l} - \zeta_i)}$ with probability $q^{-d/l}$. It follows that there are at most $q^{-d/l} \cdot |\mathbb{Z}_q^K \times \mathbb{Z}_q^{K''}|$ choices of $\vec{\alpha}, \vec{\beta}$ such that F evaluates to $\mathbf{0}$. We conclude that the failure density of this predicate is $q^{-d/l}$. $\square$

I.1.3 Level 2. On this level we wish to ensure that our extracted witnesses satisfy the constant term constraint functions in $\mathcal{F}'$ and that the projection was computed correctly.

$$\Phi_2^{\text{prop}}(t_1) = 1 \Leftrightarrow \begin{cases} (\vec{w}_i^*, \vec{c}_i)_{i \in [r]} \leftarrow E_3(t_1), \\ \forall f' \in \mathcal{F}' : \text{ct}(f'(\vec{w}_1^*, \dots, \vec{w}_r^*)) = 0 \pmod{q}, \\ \forall l \in [2\lambda] : p_l = \text{ct}(\langle \sigma_{-1}(\vec{\pi}_i^{(l)}), \vec{w}_i^* \rangle) \pmod{q} \end{cases}$$

Lemma I.3. *The failure density of Φ_2^{com} in Φ is $p_2^{\text{com}} = q^{-\lceil \lambda / \log q \rceil}$.*

Proof. In the first aggregation step, the constraints in $\mathcal{F}'$ are aggregated to $K'' = \lceil \lambda / \log_2 q \rceil$ functions by computing random linear combinations. Let $L = |\mathcal{F}'|$. For $k = 1, \dots, K''$,

$$\begin{aligned} f''^{(k)}(\vec{w}_1^*, \dots, \vec{w}_r^*) &= \sum_{l=1}^L \psi_l^{(k)} f'^{(l)}(\vec{w}_1^*, \dots, \vec{w}_r^*) + \sum_{l=1}^{2\lambda} \omega_l^{(k)} (\langle \sigma_{-1}(\vec{\pi}_i^{(l)}), \vec{w}_i^* \rangle - p_l) \\ &= \sum_{i,j=1}^r \mathbf{a}''^{(k)} \langle \vec{w}_i^*, \vec{w}_j^* \rangle + \sum_{i=1}^r \langle \vec{\varphi}_i''^{(k)}, \vec{w}_i^* \rangle - b_0''^{(k)}. \end{aligned}$$

Then the prover should extend $b_0''^{(k)}$ to $\mathbf{b}''^{(k)}$ so that $f''^{(k)}$ evaluates to $\mathbf{0}$ over $\mathcal{R}_q$ in the witness. The verification algorithm checks that every $b_0''^{(k)}$ is computed correctly. Because of Φ_3^{prop} , we know that $f''^{(k)}(\vec{w}_1^*, \dots, \vec{w}_r^*) = \mathbf{0}$ for all k , meaning that the rest of $\mathbf{b}''^{(k)}$ was computed correctly as well.

To compute the failure density of this predicate, assume without loss of generality that some $f'^{(l)} \in \mathcal{F}'$ does not evaluate to $\mathbf{0}$. The prover is bound to this evaluation by the binding predicate. Imagine that for the second subtree, $\vec{\psi}^{(k)} \xleftarrow{\$} \mathbb{Z}_q^L$ and $\vec{\omega}^{(k)} \xleftarrow{\$} \mathbb{Z}_q^{2\lambda}$. $\mathbf{b}''^{(k)}$ was computed honestly, so $b_0''^{(k)}$ contains the term $\psi_l^{(k)} f'(\vec{w}_1^*, \dots, \vec{w}_r^*)$, which is independently uniformly random since $\mathbb{Z}_q$ is a field. Thus, $b_0''^{(k)}$ is distributed uniformly at random and would only be the correct with probability $1/q$. The probability that all of the $\lceil \lambda / \log_2 q \rceil$ repetitions will have the correct value is then $q^{-\lceil \lambda / \log q \rceil}$. It follows that the proportion of choices of $\vec{\psi}^{(k)}, \vec{\omega}^{(k)}$ such that there is a valid subtree is $q^{-\lceil \lambda / \log q \rceil}$. $\square$

I.1.4 Level 1. For the top level, we enforce that the witness is short.

$$\Phi_1^{\text{prop}}(t_1) = 1 \Leftrightarrow (\vec{w}_i^*, \vec{c}_i)_{i \in [r]} \leftarrow E_2(t_1), \sum_{i=1}^r \|\vec{w}_i^*\| \leq \sqrt{\lambda/C_2}\beta.$$

Here C_2 is a constant defined as in Lemma 2.2.

For an honest prover the projection will be smaller than the required bound with probability $1/2$. When this is not the case the prover will fail, giving completeness error $1/2$. To address this we require the prover to start over by slightly adjusting the first round of Fiat-Shamir transform: (1) prover initializes $\text{ctr} = 1$, (2) query H_1 with $(x, \vec{u}_1, \text{ctr})$ (with x the statement to be proven) to obtain a random projection challenge, (3) compute $\vec{p}$, (4) if $\vec{p}$ exceeds the verification bound, set $\text{ctr} = \text{ctr} + 1$ and go to (2); else, proceed to the second round. The verifier must ensure $\text{ctr} \in \{1, \dots, \eta\}$, where η denotes a completeness parameter. Then we obtain completeness error $2^{-\eta}$. By viewing a tuple $(\vec{u}_1, \text{ctr})$ as a first-round message, the failure density for a single projection may be used without modification.

Lemma I.4. *The failure density of Φ_1^{com} in Φ is $p_1^{\text{com}} = 2^{-\lambda}$.*

Proof. Assume that $\sum_{i=1}^r \|\vec{w}_i^*\|_2^2 > (\lambda/C_2)\beta^2$. By the previous predicates, we know that the prover computed their projection $\vec{p}$ honestly, so that $\vec{p} = \sum_{i=1}^r \Pi_i \vec{w}_i^*$. Because the subtree is valid, it must be the case that $\|\vec{p}\|_2 \leq \sqrt{\lambda}\beta$. The modular Johnson-Lindenstrauss lemma (Lemma 2.2) states the following. For a fixed vector in $\mathbb{Z}_q^m$ with ℓ_2 -norm strictly greater than $\sqrt{\lambda/C_2}\beta$, the probability that an independently sampled projection challenge gives $\|\vec{p}\|_2 \leq \sqrt{\lambda}\beta$ is at most $2^{-\lambda}$. Thus, the failure density of this predicate is $2^{-\lambda}$. $\square$

I.2 PSS of LaBRADOR.

We modify the primary LaBRADOR relation to allow some norm slack σ ,

$$R_\sigma = \left\{ \left(\begin{pmatrix} (\mathcal{F}, \mathcal{F}', \beta), \\ (\vec{w}_1^*, \dots, \vec{w}_r^*) \end{pmatrix} \right) \middle| \begin{array}{l} \forall f \in \mathcal{F}, f(\vec{w}_1^*, \dots, \vec{w}_r^*) = \mathbf{0}, \\ \forall f' \in \mathcal{F}', \text{ct}(f'(\vec{w}_1^*, \dots, \vec{w}_r^*)) = 0 \pmod{q}, \\ \sum_{i=1}^r \|\vec{w}_i^*\|_2^2 \leq \sigma\beta^2 \end{array} \right\}. \quad (19)$$

Lemma I.5. *Let Π be the base LaBRADOR protocol as described in Protocol 2 and 3. Let $\mathbf{K} = (2, 2, 2, 3)$, $\mathbf{R} = (1, 1, 1, r)$, and let Φ be the predicate system consisting of commitment predicates $\Phi_1^{\text{com}}, \Phi_2^{\text{com}}, \Phi_3^{\text{com}}, \Phi_4^{\text{com}}$ and the challenge predicates $\Phi_{4,1}^{\text{chal}}, \Phi_{4,2}^{\text{chal}}$. Then the protocol Π is $(\mathbf{K}, \mathbf{R}, \Phi)$ -coordinate-wise predicate-special-sound for the primary LaBRADOR relation R_σ with $\sigma = \sqrt{\lambda/C_2}$.*

Proof. Given a $t \in \mathbb{T}_1$ for a statement $(\mathcal{F}, \mathcal{F}', \beta)$ such that $\Phi(t) = 1$, we know from $\Phi_1^{\text{prop}}, \Phi_2^{\text{prop}}$ and Φ_3^{prop} that a witness can be computed from t such that $((\mathcal{F}, \mathcal{F}', \beta), (\vec{w}_1^*, \dots, \vec{w}_r^*)) \in R_\sigma$. $\square$

I.3 Binding Relation

In the first round LaBRADOR has both inner and outer commitments. Inner commitments are all produced with respect to $\mathbf{A} \in \mathcal{R}_{q'}^{\kappa \times n}$. The outer commitments are the sum of commitments using the matrices $\mathbf{B}_i \in \mathcal{R}_{q'}^{\kappa_1 \times \kappa}$ for $i \in [1, r]$ and $\mathbf{C}_{ijk} \in \mathcal{R}_{q'}^{\kappa_2 \times 1}$ for $i \in [1, r]$, $j \in [i, r]$, $k \in [0, t_2 - 1]$. The norms of openings to outer commitments are only ever checked together, therefore, when $\kappa_1 = \kappa_2$ we may therefore treat the commitment as one large Ajtai commitment for a matrix $\mathbf{B}$. Similarly, for the second outer commitment we may consider $\mathbf{D}$ rather than $\mathbf{D}_{i,j,k} \in \mathcal{R}_{q'}^{\kappa_2 \times 1}$ for $i \in [1, r]$, $j \in [i, r]$, $k \in [0, t_1 - 1]$. For $n_1 = \left(r\kappa + \frac{r^2+r}{2}t_2\right)$ and $n_2 = \left(\frac{r^2+r}{2}t_1\right)$, $\mathbf{B} \in \mathcal{R}_q^{\kappa_1 \times n_1}$, $\mathbf{D} \in \mathcal{R}_q^{\kappa_1 \times n_2}$. In the case where binding is broken we wish to extract solutions to the Module-SIS problem, in particular we consider the relation

$$R_{\kappa, n, \mathbf{A}, \beta}^{\text{M-SIS}} = \{ \vec{v} \mid \vec{v} \in \mathcal{R}_q^n, \mathbf{A} \cdot \vec{v} = \vec{0}, 0 < \|\vec{v}\| \leq \beta^* \}.$$

Lemma I.6. *The predicate system Φ admits*

$$R_{\text{pp}}^{\text{M-SIS}} = R_{\kappa, n, 8T_{\text{op}}(b+1)\beta', \text{pp}, \mathbf{A}}^{\text{M-SIS}} \cup R_{\kappa_1, n_1, 2\beta', \text{pp}, \mathbf{B}}^{\text{M-SIS}} \cup R_{\kappa_1, n_2, 2\beta', \text{pp}, \mathbf{D}}^{\text{M-SIS}}$$

as a binding relation.

Proof. First, we consider $\Phi_3^{\text{bind}}(t_4^1, t_4^2)$ where $\Phi(t_4^i) = 1$ for $i = 1, 2$. Recall for the predicate to be satisfied the following conditions must hold

$$\begin{aligned} (\vec{w}_{i,1}^*, \vec{c}_{i,1})_{i \in [r]} &\leftarrow E_4(t_4^1), (\vec{w}_{i,2}^*, \vec{c}_{i,2})_{i \in [r]} \leftarrow E_4(t_4^2) \\ \forall i \in [r], \vec{w}_{i,1}^* &= \vec{w}_{i,2}^*, \vec{v}_{i,1} = \vec{v}_{i,2}. \end{aligned}$$

If the predicate is violated there must be some $i \in [r]$ such that $\vec{w}_{i,1}^* \neq \vec{w}_{i,2}^*$ or $\vec{v}_{i,1} \neq \vec{v}_{i,2}$.

Let us first address the case where $\vec{w}_{i,1}^* \neq \vec{w}_{i,2}^*$ but $\vec{v}_{j,1} = \vec{v}_{j,2}$ for all $j \in [r]$. In this case we have a two distinct weak openings for $\vec{v}_{i,1} = \vec{v}_{i,2}$, allowing us to find a short solution $\vec{v} = \vec{c}_{i,1}\vec{c}_{i,2}(\vec{w}_{i,1}^* - \vec{w}_{i,2}^*)$. This may be seen as,

$$\mathbf{A}\vec{w}_{i,1}^* = \vec{v}_{i,1} = \mathbf{A}\vec{w}_{i,2}^*,$$

giving $\mathbf{A}(\vec{w}_{i,1}^* - \vec{w}_{i,2}^*) = \vec{0}$, which in turn implies $\mathbf{A}\vec{c}_{i,1}\vec{c}_{i,2}(\vec{w}_{i,1}^* - \vec{w}_{i,2}^*) = \vec{0}$. This solution is indeed short as, when $\Phi(t_4^1) = \Phi(t_4^2) = 1$ then

$$\begin{aligned} \|\vec{c}_{i,1}\vec{c}_{i,2}(\vec{w}_{i,1}^* - \vec{w}_{i,2}^*)\|_2 &\leq \|\vec{c}_{i,2}(\vec{c}_{i,1}\vec{w}_{i,1}^*)\|_2 + \|\vec{c}_{i,1}(\vec{c}_{i,2}\vec{w}_{i,2}^*)\|_2 \\ &\leq 2T_{\text{op}} \|\vec{c}_{i,1}\vec{w}_{i,1}^*\|_2 + 2T_{\text{op}} \|\vec{c}_{i,2}\vec{w}_{i,2}^*\|_2 \leq 4T_{\text{op}}(2(b+1)\beta'). \end{aligned}$$

Now we must consider the case where $\vec{v}_{i,1} \neq \vec{v}_{i,2}$ for some $i \in [r]$. In this case we obtain two distinct openings with norm bound β' for a commitment using the matrix $\mathbf{B}$, from which we may compute a solution of norm $2\beta'$. The analysis for Φ_3^{bind} may be applied to the predicates $\Phi_1^{\text{bind}}, \Phi_2^{\text{bind}}$.

If Φ_4^{bind} is violated there must either be distinct openings to the outer commitments or distinct $\vec{y}_{i,j} = \vec{z} - \mathbf{c}_i\vec{w}_i^* - \mathbf{c}_j\vec{w}_j^*$ values. Note that for the transcripts collected for the i -th coordinate,

$$\vec{y}_i(t) \neq \vec{y}_i(t') \Rightarrow \vec{y}_i(t) - \mathbf{c}_{1,j}\vec{w}_j^* \neq \vec{y}_i(t') - \mathbf{c}_{1,j}\vec{w}_j^* \Rightarrow \vec{y}_{i,j}(t) \neq \vec{y}_{i,j}(t'),$$

so the $\vec{y}_i$ are already covered by the $\vec{y}_{i,j}$ case. Assume two subtrees t and t' extracted for the same coordinate i have $\vec{y}_{i,j} \neq \vec{y}'_{i,j}$ for some $j \in [r] \setminus \{i\}$. By the verification check, $\mathbf{A}(\vec{z} - \vec{z}') = (\mathbf{c}_i - \mathbf{c}'_i)\vec{v}_i$, giving

$$\mathbf{A}(\vec{y}_{i,j} + \mathbf{c}'_i\vec{w}_i^* - \vec{y}'_{i,j} - \mathbf{c}_i\vec{w}_i^*) = (\mathbf{c}_i - \mathbf{c}'_i)\vec{v}_i.$$

We know $\vec{w}_i^*$ and $\vec{c}_i$ make up a weak opening, so we have $\vec{v} = \vec{c}_i(\vec{y}_{i,j} - \vec{y}'_{i,j})$ where $\mathbf{A}\vec{v} = \vec{0}$ and $\|\vec{v}\|_2 \leq 8T_{\text{op}}(b+1)\beta'$.

Consider an outer commitment with two distinct openings, i.e. we have trees t, t' where for some i, j one of the following hold

$$\vec{v}_i(t) \neq \vec{v}_i(t'), \quad \mathbf{g}_{i,j}(t) \neq \mathbf{g}_{i,j}(t'), \quad \mathbf{h}_{i,j}(t) \neq \mathbf{h}_{i,j}(t').$$

By verification, these are openings of norm less than β' for matrix $\mathbf{B}$ or $\mathbf{D}$, giving a solution of norm at most $2\beta'$. $\square$

I.4 Knowledge Soundness

Combining our results we conclude that LaBRADOR is knowledge sound.

Theorem I.1. *Let Π be the base LaBRADOR protocol as described in Protocol 2 and 3. We consider the case with a ring $\mathcal{R}_q$ of degree d with splitting factor l and a B -well-spread challenge set, where each challenge has operator norm at most T_{op} . Restrict statements in the LaBRADOR relation to have ℓ_2 -norm bound β at most q/C_1 , where C_1, C_2 are the parameters of Lemma 2.2. The Fiat-Shamir transformation of LaBRADOR is knowledge sound for the relation $R_\sigma \cup R_{\text{pp}}^{\text{M-SIS}}$ for $\sigma = \sqrt{\lambda}/C_2$ with knowledge error¹⁴*

$$2(Q+1)(2^{-\lambda} + q^{-\lceil \lambda/\log q \rceil} + q^{-d/l} + (5+2l)rB),$$

where the extractor in expectation makes at most $K + Q(K-1)$ queries to the prover for $K = 2^3 \cdot (2r+1)$ and the prover makes at most Q oracle queries.

Proof. This follows by Theorem G.1 and Lemmata I.1 to I.6.

¹⁴ For the sake of simplicity we assume the failure density terms dominate $(k_i - 1)/|\mathcal{C}_i|$ for each round i .

Remark I.1. To plug in Fiat-Shamir LaBRADOR into [Lemma C.2](#) to realize an aggregate signature scheme, we must make sure that it also satisfies $\mathcal{Z}$ -auxiliary input knowledge soundness ([Definition B.12](#)). It is easy to see that the existence of `aux-in` does not interfere with extraction at all. Observe that the distribution of `aux-in` output by $\mathcal{Z}(1^\lambda)$ is independent of any random oracle responses in the abstract sampling game, because it merely consists of a sequence of random preimage-image pairs and a public key of the underlying hash-then-sign signature scheme. Since `aux-in` is already fixed before the abstract sampling game starts, the above analysis goes through without modification even in the presence of `aux-in`, by considering a prover with their input tape filled with `aux-in` as another fixed cheating prover with no auxiliary input. Note also that `aux-in` is independent of the LaBRADOR public parameters pp_Π , which allows one to construct a reduction solving M-SIS without any problem whenever a M-SIS instance has to be embedded in pp_Π .

I.5 Recursive Composition

Let us now consider recursively composing the LaBRADOR protocol t times. This means that the prover no longer responds with $\vec{z}, \vec{v}, \vec{g}, \vec{h}$ after the first iteration, but instead commits to it as the first message of the next iteration, only leaving the opening of the last iteration.

For a single LaBRADOR iteration an accepting transcript directly guarantees that the checks performed by an honest verifier hold, thus the guarantees for $\vec{z}, \vec{v}, \vec{g}, \vec{h}$ must now instead be enforced by the predicates on the subsequent iteration. We may see a sequence of t compositions as $(\mathbf{K}, \mathbf{R}, \Phi)$ -coordinate-wise predicate-special-sound, for

$$\mathbf{K} = (2, 2, 2, 3, \dots, 2, 2, 2, 3), \quad \mathbf{R} = (1, 1, 1, r_1, \dots, 1, 1, 1, r_t).$$

The predicate system Φ must now be obtained by applying the same predicates as for a single iteration repeatedly, going left to right starting from the first level:

$$\Phi_1^{\text{com}}, \Phi_2^{\text{com}}, \Phi_3^{\text{com}}, (\Phi_{4,1}^{\text{chal}}, \Phi_{4,2}^{\text{chal}}, \Phi_4^{\text{com}}), \dots, \Phi_1^{\text{com}}, \Phi_2^{\text{com}}, \Phi_3^{\text{com}}, (\Phi_{4,1}^{\text{chal}}, \Phi_{4,2}^{\text{chal}}, \Phi_4^{\text{com}}).$$

Note the norm checks enforced by Φ_1^{com} all have slack $\sqrt{\lambda/C_2}$. Thus, the norms of the M-SIS must tolerate this additional slack, too. The soundness error now accumulates additively for each iteration,

$$2(Q+1) \sum_{i=1}^t \left(2^{-\lambda} + q^{-\lceil \lambda / \log q \rceil} + q^{-d/l} + (5+2l)r_i B \right), \quad (20)$$

while the runtime composes multiplicatively, giving $K + Q(K-1)$ queries to the prover for $K = \prod_{i=1}^t (2^3 \cdot (2r_i + 1))$. Recall, the sequence $r_1, \dots, r_t$ is exponentially decreasing.

I.6 Last Iteration Optimizations

In its original presentation, the final iteration of LaBRADOR was optimized to reduce the proof size; as the final message is not given as input to another iteration the garbage polynomials may instead be distributed across r prover messages, where r is the number of witness vectors. Using techniques from [\[NS22\]](#), this allows garbage polynomials to depend on previous challenges, giving a smaller number of polynomials in total. Previously we have used 3-special-sound trees to extract witnesses and perform Schwartz-Zippel checks. Extending this approach naïvely, extracting one witness for each level in the tree quickly proves problematic, when considering the norms of the extracted openings. For each new witness the norm of its weak opening will depend on the norms of all previous openings with an extra factor depending on the operator norm. This causes the norms of the weak openings to have slack proportional to $(T_{\text{op}})^r$. To ensure binding still holds for weak openings with this added slack, the rank of the commitments must grow significantly. The size impact of larger commitments quickly outweighs the benefits of the optimization as r grows.

In the interactive setting it was possible to extract each witness $\vec{w}_i^*$ independently, by finding transcripts where the challenge only differs in the i th round. However, it seems unclear how such an extraction strategy would work in the non-interactive setting. One alternative solution could be adding a final round with an extra amortized opening for new challenges. Witnesses could then be extracted from this new opening as in a normal LaBRADOR iteration, and used to enforce binding with respect to the other opening, avoiding the exponential slack in r . The second opening may outweigh the concrete gains made by applying the optimization.